# Warehouse Metrics Repo

```
/*
================================================================================
================

WAREHOUSE SKIPPED LINE REPORT — UAE & KSA WAREHOUSES

Purpose : tracks skipped, ops-marked-missing, and picked item
          counts per warehouse category and business date,
          combining both legacy (GOMS) and new (CW Picking)
          picking systems into a single unified view
Output  : one row per date × warehouse category
          with skipped_items, ops_marked_missing,
          ops_marked_damage, ops_marked_expiry, and picked_items
Author  : noon Minutes Ops
================================================================================
================

DEFINITIONS
-----------
date          : operational business date derived from the picking
                job timestamp, offset by -6h30m (UAE) or -6h (KSA)
                to align with the warehouse business day start.
                e.g. a job accepted at 06:30 Dubai maps to the
                same business date as one accepted at 07:00
wh_code       : partner_warehouse_code — human-readable WH identifier
                e.g. DXBID01, RUHID01
warehouse     : logical category resolved from the WH code mapping
                table (noonbinimops.fulfillment.wh_code_wh_category_mapping).
                Falls back to 'Other' if code is unmapped.
skipped_items : distinct picking lines/items skipped due to
                Damaged, Missing, or Expired reasons
ops_marked_missing : subset of skipped items where the item is
                in mp_status=31 (unfulfillable), representing
                items flagged by ops as missing/damaged/expired
ops_marked_damage  : ops_marked_missing filtered to Damaged reason
ops_marked_expiry  : ops_marked_missing filtered to Expired reason
picked_items  : distinct items/lines successfully picked
                (not skipped, not cancelled)

SYSTEMS COVERED
---------------
old_base  : legacy GOMS picking system
            source: noondwh.scgoms_goms (picking_job, picking_job_line,
                item, warehouse, reason)
```

```
            skip reasons identified by name_en IN
              ('Damaged Items', 'Missing Items', 'Expired Items')
            skip flag: pjl.id_status = 11
            picked flag: pjl.id_status NOT IN (11, 51)

  new_base  : new CW Picking system
            source: noondwh.scgoms_cwpicking (picking_job, picklist_line,
                  reason, warehouse, picking_container_line)
            skip reasons identified by r.code IN
              ('missing', 'damaged', 'expired')
            skip flag: pllst.code = 'skipped'
            picked flag: picking_container_line join is non-null
            NOTE: item-level data joined via transfer_request_line
                 using transfer_nr + catalog_sku as composite key

  FILTERS
  ———————
  Rolling 31-day window on job created/accepted timestamp
  Only closed jobs (id_status=2 in GOMS / pjst.code='closed' in CW)
  GOMS: id_mp = 9 (noon marketplace)
  Final output limited to last 30 business days
═══════════════════════════════════════════════════════════════════════════════
═══════
*/

WITH old_base AS (
    --
———————————————————————————————————————————————————————————————————————————
—
    -- LEGACY GOMS: Aggregate skipped and picked items per warehouse
    -- and business date from the old picking system.
    -- Business date is shifted back 6h30m (UAE) or 6h (KSA) from
    -- job accepted_at to align with warehouse operational day.
    --
———————————————————————————————————————————————————————————————————————————
—
    SELECT
        src_w.partner_warehouse_code AS wh_code,
        name_en,                        -- skip reason label (e.g. 'Damaged Items')
        CASE
          WHEN src_w.id_country = 1 THEN date(DATEtime(pj.accepted_at, 'Asia/Dubai') - interval
'6' hour - interval '30' minute)
          WHEN src_w.id_country = 2 THEN date(DATEtime(pj.accepted_at, 'Asia/Riyadh') -
interval '6' hour)
```

```
        END AS date,

        -- Items skipped for Damaged / Missing / Expired reasons
        COUNT(DISTINCT case when pjl.id_status = 11 AND name_en IN ('Damaged Items',
'Missing Items', 'Expired Items') then i.id_item end) AS skipped_items,

        -- Subset of skipped items that ops flagged as unfulfillable (missing/ops reason)
        COUNT(
            DISTINCT CASE
                WHEN pjl.id_status = 11 AND name_en IN ('Damaged Items', 'Missing Items',
'Expired Items') and i.id_mp_status = 31
                    AND JSON_EXTRACT_SCALAR(i.misc, '$.unfulfillable_reason') in
('ops_marked_missing','ops')
                THEN i.id_item
            END
        ) AS ops_marked_missing,

        -- Items successfully picked (not skipped, not in a terminal skip/cancel state)
        COUNT(DISTINCT case when pjl.id_status <> 11 AND pjl.id_status <> 51 then i.id_item
end) AS picked_items
    FROM
        noondwh.scgoms_goms.picking_job pj
        LEFT JOIN noondwh.scgoms_goms.picking_job_line pjl USING (id_picking_job)
        LEFT JOIN noondwh.scgoms_goms.item i ON pjl.id_item = i.id_item
        LEFT JOIN noondwh.scgoms_goms.user u ON pj.id_user = u.id_user
        LEFT JOIN noondwh.scgoms_goms.warehouse src_w ON src_w.id_warehouse =
pj.id_warehouse
        LEFT JOIN noondwh.scgoms_goms.reason r ON r.id_reason = pjl.id_skip_reason
    WHERE
        DATE(pj.accepted_at, 'Asia/Dubai') >= CURRENT_DATE - 31  -- rolling 31-day window
        AND pj.id_status = 2    -- closed jobs only
        AND pj.id_mp = 9        -- noon marketplace only
    GROUP BY
        ALL
),

new_base AS (
    --
-------------------------------------------------------------------------------
-
    -- NEW CW PICKING: Aggregate skipped and picked lines per
    -- warehouse and business date from the new picking system.
    -- Business date offset same as old_base (6h30m UAE / 6h KSA).
    -- Item-level data linked via transfer_request + transfer_nr.
```

```
  --
_____________________________________________________________________________
_
  SELECT
    src_w.partner_warehouse_code AS wh_code,
      r.code as name_en,              -- skip reason code (e.g. 'missing', 'damaged')
      CASE
        WHEN src_w.id_country = 1 THEN date(DATEtime(pj.created_at, 'Asia/Dubai') - interval
'6' hour - interval '30' minute)
        WHEN src_w.id_country = 2 THEN date(DATEtime(pj.created_at, 'Asia/Riyadh') - interval
'6' hour)
      END AS date,

      -- Picklist lines skipped for missing / damaged / expired reasons
      count(distinct case when pllst.code = 'skipped' AND r.code IN ('missing', 'damaged',
'expired') then pll.id_picklist_line end) AS skipped_items,

      -- Subset of skipped items where item is unfulfillable (mp_status=31),
      -- representing ops-confirmed missing/damaged/expired inventory
      COUNT(
        DISTINCT CASE
          WHEN pllst.code = 'skipped' AND r.code IN ('missing', 'damaged', 'expired') and
i.id_mp_status = 31
            --  AND JSON_EXTRACT_SCALAR(i.misc, '$.unfulfillable_reason') in
('ops_marked_missing','ops')
          THEN i.id_item
        END
      ) AS ops_marked_missing,

      -- Picklist lines successfully picked (confirmed by presence in picking container)
      count(distinct case when plc.id_picking_container_line is not null then pll.id_picklist_line
end) AS picked_items
  FROM noondwh.scgoms_cwpicking.picking_job pj
LEFT JOIN noondwh.scgoms_cwpicking.status pjst ON pj.id_status = pjst.id_status
LEFT JOIN noondwh.scgoms_cwpicking.picklist_line pll USING (id_picking_job)
-- Transfer request lines linked within a 32-day window to bound partition scans
left join noondwh.scgoms_cwpicking.transfer_request_line t on t.id_transfer_request_line =
pll.id_transfer_request_line and DATE(t.created_at, 'Asia/Dubai') >= CURRENT_DATE - 32
left join noondwh.scgoms_cwpicking.transfer_request tr on tr.id_transfer_request =
t.id_transfer_request and DATE(tr.created_at, 'Asia/Dubai') >= CURRENT_DATE - 32
-- Item resolved via composite key: transfer_nr + catalog_sku
LEFT JOIN noondwh.scgoms_goms.item i ON tr.transfer_nr = i.transfer_nr and i.catalog_sku =
t.sku  and DATE(i.created_at, 'Asia/Dubai') >= CURRENT_DATE - 32
LEFT JOIN noondwh.scgoms_cwpicking.status pllst ON pll.id_status = pllst.id_status
```

```
LEFT JOIN noondwh.scgoms_cwpicking.warehouse src_w ON pll.id_warehouse =
src_w.id_warehouse
LEFT JOIN noondwh.scgoms_cwpicking.reason r ON r.id_reason = pll.id_skip_reason
LEFT JOIN noondwh.scgoms_cwpicking.warehouse_zone wz ON pll.id_warehouse_zone =
wz.id_warehouse_zone
LEFT JOIN noondwh.scgoms_cwpicking.warehouse dst_w ON pll.id_destination =
dst_w.id_warehouse
LEFT JOIN noondwh.scgoms_cwpicking.picking_container_line plc USING (id_picklist_line)
LEFT JOIN noondwh.scgoms_cwpicking.picking_container pcr ON pcr.id_picking_container =
plc.id_picking_container
LEFT JOIN noondwh.scgoms_cwpicking.cluster cl ON (cl.id_cluster = pll.id_cluster)
WHERE
  -- Business date filter: offset by 6h30m to align with WH operational day
  DATE(DATETIME(pj.created_at, 'Asia/Dubai') - INTERVAL '6' HOUR - interval '30' minute) >=
CURRENT_DATE - 31
  and pjst.code = 'closed'   -- closed jobs only
group by all
),

final_base as
(
--
-------------------------------------------------------------------------------------
-------
-- UNION: Merge new CW Picking and legacy GOMS results, resolve
-- warehouse category from the WH code mapping table, and
-- aggregate by date × warehouse. Last 30 days only.
--
-------------------------------------------------------------------------------------
-------

-- New CW Picking system aggregated by warehouse category
SELECT
    date,
    COALESCE(c.warehouse_catrgory, 'Other') AS warehouse,
    SUM(skipped_items) AS skipped_items,
    SUM(ops_marked_missing) AS ops_marked_missing,
    sum(case when name_en in ('Damaged Items') then ops_marked_missing else 0 end ) as
ops_marked_damage,
    sum(case when name_en in ('Expired Items') then ops_marked_missing else 0 end ) as
ops_marked_expiry,
    sum(picked_items) as picked_items
FROM
    new_base b
```

```
    LEFT JOIN noonbinimops.fulfillment.wh_code_wh_category_mapping c ON b.wh_code =
c.warehouse_code
    where date >= CURRENT_DATE - 30
GROUP BY ALL

union all

-- Legacy GOMS system aggregated by warehouse category
SELECT
    date,
    COALESCE(c.warehouse_catrgory, 'Other') AS warehouse,
    SUM(skipped_items) AS skipped_items,
    SUM(ops_marked_missing) AS ops_marked_missing,
    sum(case when name_en in ('Damaged Items') then ops_marked_missing else 0 end ) as
ops_marked_damage,
    sum(case when name_en in ('Expired Items') then ops_marked_missing else 0 end ) as
ops_marked_expiry,
    sum(picked_items) as picked_items
FROM
    old_base b
    LEFT JOIN noonbinimops.fulfillment.wh_code_wh_category_mapping c ON b.wh_code =
c.warehouse_code
    where date >= CURRENT_DATE - 30
GROUP BY ALL
)

--
-------------------------------------------------------------------------------
-- FINAL OUTPUT: Collapse any duplicates across the two systems
-- by summing all metrics at date × warehouse grain.
--
-------------------------------------------------------------------------------
select
    date,
    warehouse,
    sum(skipped_items) as skipped_items,
    sum(ops_marked_missing) as ops_marked_missing,
    sum(ops_marked_damage) as ops_marked_damage,
    sum(ops_marked_expiry) as ops_marked_expiry,
    sum(picked_items) as picked_items
from final_base
group by all
```

```
/*
================================================================================
=
```

# INCIDENT STOCKTAKE REPORT — UAE & KSA WAREHOUSES

```
  Purpose : tracks job completion metrics for incident stocktake
            processes per warehouse category and business date
  Output  : one row per created_date × warehouse
            with total_jobs, pending_jobs, pending_confirmation_jobs,
            skipped_jobs, and completed_jobs
  Author  : noon Minutes Ops
================================================================================
=
  DEFINITIONS
  ----------------
  created_date            : local calendar date the stocktake job
                            was created (Dubai TZ for UAE,
                            Riyadh TZ for KSA)
  grouping_key            : job grouping identifier — filtered to
                            'incident_stocktake' (ad-hoc counts)
  wh_code                 : partner_warehouse_code — human-readable
                            WH identifier (e.g. DXBID01, RUHID01)
  warehouse               : logical category resolved from the WH
                            code mapping table. Falls back to
                            'Other' if code is unmapped.
  skipped_jobs            : jobs in 'force_closed' state —
                            stocktake jobs closed without completion
  pending_jobs            : jobs still in 'pending' state
  pending_confirmation_jobs : jobs in 'pending_confirmation' state —
                            awaiting supervisor/ops sign-off
  completed_jobs          : jobs in 'closed' state — fully counted
                            and confirmed
  total_jobs              : all jobs regardless of state
  completion_percentage   : completed_jobs / total_jobs × 100

  SOURCE TABLES (noondwh.mxdcss_dcss)
  ------------------------------------------------------------

  job          : stocktake job header with created_at and status
  status       : lookup for job state codes
  warehouse    : warehouse master with country and partner_warehouse_code
  process      : process definition linked to the job
  process_type : process type lookup — filtered to 'stock_take'
```

```
  FILTERS
  ———————
  process_type.code = 'stock_take'
  grouping_key = 'incident_stocktake'
  Rolling 31-day window on job created_at
  Final output limited to last 30 business days
═══════════════════════════════════════════════════════════════════════════════════════════════
=
*/

with base as
(
  SELECT
    -- Localise job creation date to warehouse country timezone
    CASE
         WHEN w.id_country = 1 THEN DATE(j.created_at, 'Asia/Dubai')
         WHEN w.id_country = 2 THEN DATE(j.created_at, "Asia/Riyadh")
        END AS created_date,

    j.grouping_key,
    w.partner_warehouse_code as wh_code,
    COUNT(CASE WHEN s.code = 'force_closed' THEN j.job_nr END) as skipped_jobs,            -- jobs
closed without completion
    COUNT(CASE WHEN s.code = 'pending' THEN j.job_nr END) AS pending_jobs,                 -- not yet
started
    COUNT(CASE WHEN s.code = 'pending_confirmation' THEN j.job_nr END) AS
pending_confirmation_jobs,  -- awaiting sign-off
    COUNT(CASE WHEN s.code = 'closed' THEN j.job_nr END) AS completed_jobs,                -- fully
completed
    COUNT(j.job_nr) AS total_jobs,
    COUNT(CASE WHEN s.code = 'closed' THEN j.job_nr END)* 100/COUNT(j.job_nr) AS
completion_percentage  -- % complete

FROM noondwh.mxdcss_dcss.job j
JOIN noondwh.mxdcss_dcss.status s USING (id_status)
JOIN noondwh.mxdcss_dcss.warehouse w using (id_warehouse)
JOIN noondwh.mxdcss_dcss.process p USING (id_process)
JOIN noondwh.mxdcss_dcss.process_type pt USING (id_process_type)
WHERE
  pt.code = 'stock_take' and  j.grouping_key = 'incident_stocktake'  -- incident stocktakes only
  AND j.created_at >= TIMESTAMP(DATE_SUB(CURRENT_DATE(), INTERVAL 31 DAY))  -- rolling
31-day window
  --and s.code != 'force_closed'

GROUP BY all
)
-- ———————————————————————————————————————————————————————————————————————————————
-- Final output: resolve warehouse category from WH code mapping,
-- aggregate job counts at date × warehouse grain.
```

```
-- Last 30 days only.
-- ──────────────────────────────────────────────────────────────────────────────────────
select
  created_date,
  coalesce(c.warehouse_catrgory,'Other') AS warehouse,  -- falls back to 'Other' if wh_code unmapped
  sum(total_jobs) total_jobs,
  sum(pending_jobs) as pending_jobs,
  sum(pending_confirmation_jobs) as pending_confirmation_jobs,
  sum(skipped_jobs) as skipped_jobs,
  sum(completed_jobs) as completed_jobs
 from base b left join noonbinimops.fulfillment.wh_code_wh_category_mapping c on b.wh_code =
c.warehouse_code
 where created_date >= CURRENT_DATE - 30
 group by 1,2
```

/*
═══════════════════════════════════════════════════════════════════════
═══════

# MP PRODUCTIVITY REPORT — UAE CENTRAL WAREHOUSES

Purpose : measures per-user throughput and active time for
6 warehouse functions across all UAE central WHs
for a single business day
Output : one row per user × wh_code × biz_date
with units, time (seconds), and total throughput
Author : noon Minutes
═══════════════════════════════════════════════════════════════════════
═══════

DEFINITIONS
———————————

biz_date : operational business date, defined as the 24h window
starting 06:00 Dubai time (02:00 UTC).
e.g. biz_date 2026-04-06 = UTC 2026-04-06 02:00 → 2026-04-07 02:00
computed as: DATE(DATETIME(ts,'Asia/Dubai') - INTERVAL 6 HOUR)
so a shift starting at 06:00 Dubai = midnight on biz_date

wh_code : partner_warehouse_code — the human-readable WH identifier
e.g. DXBID01, AUHID01, DXSID03

warehouse_group : logical grouping of WH codes
TP = DXBID01-08 (Technopark, Dubai)
Kezad = AUHID01-02 (Khalifa Economic Zone, Abu Dhabi)
FZ Kezad = AUHFKZ01-03 (Free Zone Kezad)
Dubai South = DXSID01-07 + DXSLQD01
F&V = DXBFNV01 (Fruit & Veg dedicated WH)

id_user : internal numeric user ID across goms/dcss/inbound systems
username : resolved from scgoms_cwpicking.user + scgoms_goms.user.
prefers ops.idp.noon.team usernames (local ops accounts)
over noon.com SSO usernames.
falls back to raw id_user string if not found in either table.
NOTE: dcss-only users (stocktake/movement) may not resolve

FUNCTIONS COVERED
—————————————————

pick : units = distinct picklist lines completed in a closed picking job

time  = job duration (job_start → job_end), but if first scan
occurs >600s after job start, time starts from first scan
(strips idle/setup time at job open)
source: noondwh.scgoms_cwpicking

put : putaway jobs only (id_job_subtype IN 4,5,6)
units = distinct completed job lines (id_status=3)
time  = MIN(job.created_at) → MAX(job_line.updated_at)
source: noondwh.mxdcss_dcss.job + job_line

receive : inbound box receipt (id_status IN 13=received, 14=putaway)
units = distinct inbound boxes
time  = MIN(created_at) → MAX(updated_at) per user/wh/day
source: noondwh.scinbound_inbound.inbound_box
NOTE: joins on partner_code (not partner_warehouse_code)

sort : goms sort scan events (id_audit_action=21,
to_state='pending_putaway')
units = scan count
time  = sum of inter-scan gaps, capped at 300s per gap
(prevents inflating time when sorter is idle)
source: noondwh.scgoms_goms.audit_log

stocktake : dcss stocktake jobs (id_job_subtype=10, process_type LIKE '%stock%take%')
only closed or force_closed-with-pending-confirmation jobs
units = distinct job lines
time  = opened_at → pending_confirmation_at (or closed_at if no pending)
source: noondwh.mxdcss_dcss.job + audit_log

movement : dcss internal_movement or migration jobs on deep locations only
(is_deep='1' on source location)
units = SUM(transaction_line.qty)
time  = MIN(tl.created_at) → MAX(tl.created_at) per user/wh/day
id_user sourced from job table (not transaction_line)
source: noondwh.mxdcss_dcss.transaction_line + job + process_type

total_throughput : pick + sort + put + recv + stocktake + movement units
(movement_units excluded from time calculations but included
in throughput count)

KEY DESIGN DECISIONS
---

1. biz_date shift: 06:00 Dubai start aligns with operational day.
the -6h offset converts Dubai time to a "logical midnight" so

DATE() returns the correct biz_date for night-shift workers.

2. pick time: the >600s first-scan rule removes cases where a job
   was opened but the picker wasn't active yet (e.g. job pre-assigned
   before shift start). uses first actual scan as clock start.

3. sort gap cap: LEAST(..., 300) caps each inter-scan gap at 5 min.
   without this, a sorter's idle break inflates sort_sec massively.
   300s is a conservative max reasonable scan cycle time.

4. stocktake pre-aggregation: audit log timestamps are aggregated
   into stocktake_job_times CTE first, then SUM'd — avoids BQ
   multi-level aggregation error (SUM(MIN(...)) not allowed).

5. movement user: transaction_line has no id_user; pulled from
   parent job table via tl.id_job = j.id_job.

6. movement time: MIN→MAX across all tl rows for user/wh/day —
   not per-job. understates time for users with multiple movement jobs.

KNOWN LIMITATIONS
———————————————

- sort only covers goms Sort WHs (DXBID01 is Non-Sort, uses cwpicking)
- JLM (skip/skipped_flow) not excluded from pick units — these are
  failed picks logged in scgoms_goms, not cwpicking, so not in scope here
- username resolution misses dcss-only users → falls back to id_user int
- movement time is day-level not job-level (see point 6 above)

═══════════════════════════════════════════════════════════════════════
═══════
*/

-- ── VARIABLES
———————————————————————————————————————————————————————————
———

-- biz_date_start / biz_date_end define the UTC window for the target biz date
-- to change date: update TIMESTAMP('2026-04-06 02:00:00 UTC') → next day 02:00 UTC
-- e.g. for 2026-04-07: use TIMESTAMP('2026-04-07 02:00:00 UTC') and '2026-04-08 02:00:00
UTC'
DECLARE biz_date_start TIMESTAMP DEFAULT TIMESTAMP('2026-04-06 02:00:00 UTC');
DECLARE biz_date_end   TIMESTAMP DEFAULT TIMESTAMP('2026-04-07 02:00:00 UTC');

WITH

```
-- ── WH SCOPE ──────────────────────────────────────────────────────────────────────────────
-- explicit allowlist of UAE central WH codes to include.
-- add/remove codes here to adjust scope.
-- excludes NIM dark stores and KSA WHs by design.
wh_scope AS (
  SELECT wh_code FROM UNNEST([
    'AUHID01','AUHID02',                        -- Kezad
    'AUHFKZ01','AUHFKZ02','AUHFKZ03',           -- FZ Kezad
    'DXBID01','DXBID02','DXBID03','DXBID04','DXBID07', -- TP
    'DXSID01','DXSID02','DXSID03','DXSID04','DXSID06','DXSID07', -- Dubai South
    'DXSLQD01',                                 -- Dubai South LQ
    'DXBFNV01'                                  -- F&V
  ]) AS wh_code
),

-- ── PICK: job_base ────────────────────────────────────────────────────────────────────────
-- raw picking data: one row per picking job.
-- qty = distinct picklist lines (= units picked in the job)
-- first_scan = earliest scan time in the job (used for idle-strip logic)
-- only includes jobs with at least one container line (pcl.id_picking_container_line IS NOT
NULL)
-- only closed jobs (pjst.code = 'closed')
job_base AS (
  SELECT
    src_w.partner_warehouse_code                          AS wh_code,
    pj.id_user,
    DATE(DATETIME(pj.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)   AS biz_date,
    DATETIME(pj.created_at,'Asia/Dubai')                  AS job_start,
    DATETIME(pj.finsihed_picking_at,'Asia/Dubai')         AS job_end,  -- note: typo in
source table (finsihed), do not correct
    COUNT(DISTINCT pll.id_picklist_line)                  AS qty,
    MIN(DATETIME(pcl.created_at,'Asia/Dubai'))            AS first_scan
  FROM noondwh.scgoms_cwpicking.picking_job pj
  JOIN noondwh.scgoms_cwpicking.status pjst
    ON pj.id_status = pjst.id_status AND pjst.code = 'closed'
  JOIN noondwh.scgoms_cwpicking.picklist_line pll
    ON pll.id_picking_job = pj.id_picking_job
  JOIN noondwh.scgoms_cwpicking.warehouse src_w
    ON pll.id_warehouse = src_w.id_warehouse
  JOIN noondwh.scgoms_cwpicking.picking_container_line pcl
    ON pcl.id_picklist_line = pll.id_picklist_line
```

```
  JOIN wh_scope ws ON src_w.partner_warehouse_code = ws.wh_code
  WHERE pj.created_at >= biz_date_start AND pj.created_at < biz_date_end
    AND pcl.id_picking_container_line IS NOT NULL
  GROUP BY 1,2,3,4,5
),

-- ── PICK: aggregated to user/wh/date ──────────────────────────────────
-- applies the >600s idle-strip rule:
-- if first scan is >10min after job open → start clock from first_scan
-- otherwise use full job duration (job_start → job_end)
pick AS (
  SELECT wh_code, id_user, biz_date, 'pick' AS fn,
    SUM(qty) AS units,
    SUM(CASE WHEN DATETIME_DIFF(first_scan,job_start,SECOND) > 600
             THEN DATETIME_DIFF(job_end,first_scan,SECOND)
             ELSE DATETIME_DIFF(job_end,job_start,SECOND) END) AS time_sec
  FROM job_base GROUP BY 1,2,3,4
),

-- ── PUTAWAY ──────────────────────────────────────────────────────────────────────────────
-- id_job_subtype IN (4,5,6): putaway subtypes only (excludes stocktake, movement, etc.)
-- jl.id_status = 3: completed lines only
-- time = job open → last line update (captures actual putaway duration)
dcss AS (
  SELECT
    w.partner_warehouse_code                                    AS wh_code,
    j.id_user,
    DATE(DATETIME(j.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)   AS biz_date,
    'put'                                                       AS fn,
    COUNT(DISTINCT jl.id_job_line)                              AS units,
    DATETIME_DIFF(
      MAX(DATETIME(jl.updated_at,'Asia/Dubai')),
      MIN(DATETIME(j.created_at,'Asia/Dubai')), SECOND)         AS time_sec
  FROM noondwh.mxdcss_dcss.job j
  JOIN noondwh.mxdcss_dcss.job_line jl ON jl.id_job = j.id_job
  JOIN noondwh.mxdcss_dcss.warehouse w  ON j.id_warehouse = w.id_warehouse
  JOIN wh_scope ws ON w.partner_warehouse_code = ws.wh_code
  WHERE j.id_job_subtype IN (4,5,6)
    AND jl.id_status = 3
    AND j.created_at >= biz_date_start AND j.created_at < biz_date_end
    AND j.id_user IS NOT NULL
```

```
  GROUP BY 1,2,3,4
),

-- ── RECEIVE ──────────────────────────────────────────────────────────────────────────────────
-- id_status IN (13,14): received or putaway boxes (both count as received)
-- NOTE: inbound.warehouse uses partner_code not partner_warehouse_code
-- time = earliest box created → latest box updated for the user/wh/day session
recv AS (
  SELECT
    w.partner_code                                      AS wh_code,
    ib.id_user,
    DATE(DATETIME(ib.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)  AS biz_date,
    'receive'                                           AS fn,
    COUNT(DISTINCT ib.id_inbound_box)                   AS units,
    DATETIME_DIFF(
      MAX(DATETIME(ib.updated_at,'Asia/Dubai')),
      MIN(DATETIME(ib.created_at,'Asia/Dubai')), SECOND)          AS time_sec
  FROM noondwh.scinbound_inbound.inbound_box ib
  JOIN noondwh.scinbound_inbound.warehouse w ON ib.id_warehouse = w.id_warehouse
  JOIN wh_scope ws ON w.partner_code = ws.wh_code
  WHERE ib.id_status IN (13,14)
    AND ib.created_at >= biz_date_start AND ib.created_at < biz_date_end
    AND ib.id_user IS NOT NULL
  GROUP BY 1,2,3,4
),

-- ── SORT: raw scan events ────────────────────────────────────────────────────────────────────
-- id_audit_action = 21: sort scan event in goms
-- to_state = 'pending_putaway': marks a completed sort scan
-- LEAD() gets the next scan time for the same user/wh/date
-- used to compute per-scan gap for time calculation
sort_base AS (
  SELECT
    w.partner_warehouse_code                            AS wh_code,
    al.id_user,
    DATE(DATETIME(al.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)  AS biz_date,
    DATETIME(al.created_at,'Asia/Dubai')                AS scan_at,
    LEAD(DATETIME(al.created_at,'Asia/Dubai')) OVER (
      PARTITION BY al.id_user, w.partner_warehouse_code,
        DATE(DATETIME(al.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)
      ORDER BY al.created_at)                           AS next_scan
```

```
  FROM noondwh.scgoms_goms.audit_log al
  JOIN noondwh.scgoms_goms.input_container_line icl
    ON icl.id_input_container_line = al.action_key
  JOIN noondwh.scgoms_goms.input_container ic
    ON ic.id_input_container = icl.id_input_container
  JOIN noondwh.scgoms_goms.warehouse w ON w.id_warehouse = ic.id_warehouse
  JOIN wh_scope ws ON w.partner_warehouse_code = ws.wh_code
  WHERE al.id_audit_action = 21
    AND JSON_EXTRACT_SCALAR(al.misc,'$.to') = 'pending_putaway'
    AND al.created_at >= biz_date_start AND al.created_at < biz_date_end
    AND al.id_user IS NOT NULL
),

-- ── SORT: aggregated ──────────────────────────────────────────────────────────
-- COALESCE(gap, 300): last scan of the day has no next_scan → default to 300s
-- LEAST(..., 300): cap each gap at 300s to exclude idle/break time
sort AS (
  SELECT wh_code, id_user, biz_date, 'sort' AS fn,
    COUNT(*) AS units,
    SUM(LEAST(COALESCE(DATETIME_DIFF(next_scan,scan_at,SECOND),300),300)) AS
time_sec
  FROM sort_base GROUP BY 1,2,3,4
),

-- ── STOCKTAKE: pre-aggregate audit timestamps per job ─────────────────────────
-- resolves process_type per job via explicit joins (BQ disallows subquery in JOIN)
-- aggregates opened/pending_confirmation/closed timestamps per job in one pass
-- has_pending: count of pending_confirmation events (used to validate force_closed jobs)
-- status_code: closed or force_closed
job_process_type AS (
  SELECT j.id_job, pt.code AS pt_code
  FROM noondwh.mxdcss_dcss.job j
  JOIN noondwh.mxdcss_dcss.process p     ON p.id_process = j.id_process
  JOIN noondwh.mxdcss_dcss.process_type pt ON pt.id_process_type = p.id_process_type
  WHERE j.id_job_subtype = 10
    AND pt.code LIKE '%stock%take%'
    AND j.created_at >= biz_date_start AND j.created_at < biz_date_end
),
stocktake_job_times AS (
  SELECT
    j.id_job,
    j.id_user,
```

```
    j.id_warehouse,
    MAX(s.code)                                                 AS status_code,
    MIN(CASE WHEN JSON_EXTRACT_SCALAR(al1.misc,'$.to') = 'opened'              THEN
al1.created_at END) AS opened_at,
    MIN(CASE WHEN JSON_EXTRACT_SCALAR(al1.misc,'$.to') = 'pending_confirmation'
THEN al1.created_at END) AS pending_at,
    MIN(CASE WHEN JSON_EXTRACT_SCALAR(al1.misc,'$.to') = 'closed'              THEN
al1.created_at END) AS closed_at,
    COUNTIF(JSON_EXTRACT_SCALAR(al1.misc,'$.to') = 'pending_confirmation')      AS
has_pending
  FROM noondwh.mxdcss_dcss.job j
  JOIN noondwh.mxdcss_dcss.status s ON s.id_status = j.id_status
  JOIN job_process_type jpt         ON jpt.id_job = j.id_job
  JOIN noondwh.mxdcss_dcss.audit_log al1
    ON al1.id_audit_action = 1
    AND al1.action_key = j.id_job
    AND al1.created_at >= biz_date_start AND al1.created_at < biz_date_end
  WHERE (s.code = 'closed' OR s.code = 'force_closed')
    AND j.created_at >= biz_date_start AND j.created_at < biz_date_end
    AND j.id_user IS NOT NULL
  GROUP BY 1,2,3
),

-- ── STOCKTAKE: aggregated
──────────────────────────────────────────────────────────────────────
-- time = opened_at → pending_confirmation_at (preferred)
--        falls back to closed_at if no pending_confirmation event exists
-- excludes force_closed jobs with no pending_confirmation (incomplete counts)
inv_stocktake AS (
  SELECT
    w.partner_warehouse_code                                    AS wh_code,
    jt.id_user,
    DATE(DATETIME(jt.opened_at,'Asia/Dubai') - INTERVAL 6 HOUR)   AS biz_date,
    'stocktake'                                 AS fn,
    COUNT(DISTINCT jl.id_job_line)                               AS units,
    SUM(TIMESTAMP_DIFF(
      COALESCE(jt.pending_at, jt.closed_at),
      jt.opened_at, SECOND))                                    AS time_sec
  FROM stocktake_job_times jt
  LEFT JOIN noondwh.mxdcss_dcss.job_line jl ON jl.id_job = jt.id_job
  LEFT JOIN noondwh.mxdcss_dcss.warehouse w  ON w.id_warehouse = jt.id_warehouse
  JOIN wh_scope ws ON w.partner_warehouse_code = ws.wh_code
  WHERE jt.opened_at IS NOT NULL
    AND (jt.pending_at IS NOT NULL OR jt.closed_at IS NOT NULL)
```

```
    AND NOT (jt.status_code = 'force_closed' AND jt.has_pending = 0)
  GROUP BY 1,2,3,4
),

-- ── INTERNAL MOVEMENT & MIGRATION ──────────────────────────────────────────
-- process_type IN ('internal_movement','migration'): stock moves between locations
-- is_deep='1': only deep storage locations (excludes forward pick face moves)
-- units = qty moved (not job lines), can be > 1 per tl row
-- time = MIN→MAX of tl.created_at across all movement lines for user/wh/day
--        NOTE: day-level aggregation, not per-job — understates time if multi-job
-- id_user joined from job table (transaction_line has no id_user column)
inv_movement AS (
  SELECT
    w.partner_warehouse_code                                    AS wh_code,
    j.id_user,
    DATE(DATETIME(tl.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)  AS biz_date,
    'movement'                                                  AS fn,
    SUM(tl.qty)                                                 AS units,
    DATETIME_DIFF(
      MAX(DATETIME(tl.created_at,'Asia/Dubai')),
      MIN(DATETIME(tl.created_at,'Asia/Dubai')), SECOND)        AS time_sec
  FROM noondwh.mxdcss_dcss.transaction_line tl
  JOIN noondwh.mxdcss_dcss.location l
    ON ('location', l.id_location) = (tl.src_key, CAST(tl.src_value AS INT64))
  LEFT JOIN noondwh.mxdcss_dcss.warehouse w    ON w.id_warehouse = l.id_warehouse
  JOIN noondwh.mxdcss_dcss.job j               ON j.id_job = tl.id_job
  JOIN noondwh.mxdcss_dcss.process p           ON p.id_process = j.id_process
  JOIN noondwh.mxdcss_dcss.process_type pt     ON pt.id_process_type = p.id_process_type
  JOIN wh_scope ws ON w.partner_warehouse_code = ws.wh_code
  WHERE pt.code IN ('internal_movement','migration')
    AND tl.created_at >= biz_date_start AND tl.created_at < biz_date_end
    AND JSON_EXTRACT_SCALAR(l.misc,'$.is_deep') = '1'
    AND j.id_user IS NOT NULL
  GROUP BY 1,2,3,4
),

-- ── UNION ALL 6 FUNCTIONS ──────────────────────────────────────────────────
-- common schema: wh_code, id_user, biz_date, fn (label), units, time_sec
all_fns AS (
  SELECT * FROM pick
  UNION ALL SELECT * FROM dcss
  UNION ALL SELECT * FROM recv
```

```
  UNION ALL SELECT * FROM sort
  UNION ALL SELECT * FROM inv_stocktake
  UNION ALL SELECT * FROM inv_movement
),

-- ── PIVOT
──────────────────────────────────────────────────────────────────────────────
──────────
-- collapses fn rows into columns per user/wh/date
-- CASE WHEN fn=X THEN ... ELSE 0 pattern ensures no NULLs in output
pivoted AS (
  SELECT
    wh_code, id_user, biz_date,
    SUM(CASE WHEN fn='pick'      THEN units    ELSE 0 END) AS pick_units,
    SUM(CASE WHEN fn='pick'      THEN time_sec ELSE 0 END) AS pick_sec,
    SUM(CASE WHEN fn='put'       THEN units    ELSE 0 END) AS put_lines,
    SUM(CASE WHEN fn='put'       THEN time_sec ELSE 0 END) AS put_sec,
    SUM(CASE WHEN fn='receive'   THEN units    ELSE 0 END) AS recv_boxes,
    SUM(CASE WHEN fn='receive'   THEN time_sec ELSE 0 END) AS recv_sec,
    SUM(CASE WHEN fn='sort'      THEN units    ELSE 0 END) AS sort_items,
    SUM(CASE WHEN fn='sort'      THEN time_sec ELSE 0 END) AS sort_sec,
    SUM(CASE WHEN fn='stocktake' THEN units    ELSE 0 END) AS stocktake_units,
    SUM(CASE WHEN fn='stocktake' THEN time_sec ELSE 0 END) AS stocktake_sec,
    SUM(CASE WHEN fn='movement'  THEN units    ELSE 0 END) AS movement_units,
    SUM(CASE WHEN fn='movement'  THEN time_sec ELSE 0 END) AS movement_sec
  FROM all_fns
  GROUP BY 1,2,3
),

-- ── USERNAMES
──────────────────────────────────────────────────────────────────────────────
─────
-- deduplicates users who appear in both cwpicking and goms user tables
-- COALESCE logic: prefer ops.idp.noon.team accounts (local WH logins)
--                 over noon.com SSO accounts
-- both sources split on '@' to get just the username prefix
usernames AS (
  SELECT id_user,
    COALESCE(
      MIN(CASE WHEN username LIKE '%ops.idp.noon.team%' THEN
SPLIT(username,'@')[OFFSET(0)] END),
      MIN(SPLIT(username,'@')[OFFSET(0)])
    ) AS username
  FROM (
```

```
    SELECT id_user, username FROM noondwh.scgoms_cwpicking.user
    UNION ALL
    SELECT id_user, username FROM noondwh.scgoms_goms.user
  )
  GROUP BY id_user
)

-- ── FINAL OUTPUT
──────────────────────────────────────────────────────────────────────────────
─
-- one row per user × wh_code × biz_date
-- warehouse_group derived from wh_code prefix
-- username falls back to CAST(id_user AS STRING) if not in username table
-- total_throughput = sum of all unit types (movement included in count)
SELECT
  p.biz_date,
  p.wh_code,
  CASE
    WHEN p.wh_code LIKE 'AUHFKZ%'                          THEN 'FZ Kezad'
    WHEN p.wh_code LIKE 'AUHID%'                           THEN 'Kezad'
    WHEN p.wh_code LIKE 'DXBID%'                           THEN 'TP'
    WHEN p.wh_code LIKE 'DXSID%' OR p.wh_code LIKE 'DXSLQ%'  THEN 'Dubai South'
    WHEN p.wh_code = 'DXBFNV01'                            THEN 'F&V'
    ELSE 'Other'
  END                                          AS warehouse_group,
  p.id_user,
  COALESCE(u.username, CAST(p.id_user AS STRING))          AS username,
  p.pick_units,
  p.pick_sec,
  p.sort_items,
  p.sort_sec,
  p.put_lines,
  p.put_sec,
  p.recv_boxes,
  p.recv_sec,
  p.stocktake_units,
  p.stocktake_sec,
  p.movement_units,
  p.movement_sec,
  p.pick_units + p.sort_items + p.put_lines
    + p.recv_boxes + p.stocktake_units + p.movement_units     AS total_throughput
FROM pivoted p
LEFT JOIN usernames u ON u.id_user = p.id_user
ORDER BY 1,3,2,5
```

```
/*
═══════════════════════════════════════════════════════════════════════════
```

# WAREHOUSE IPH — WITH AND WITHOUT UTILIZATION

```
Purpose : Per-user, per-function IPH at two granularities:
            • IPH_effective   – job time only (no idle; >600s scan-gap rule
                                for pick, 300s gap cap for sort)
            • IPH_utilization– total shift window: first_scan → last_scan
                                capped at 12h (43200s); cases exceeding 12h
                                flagged for ops review

Source  : Raw tables (~30min refresh) – noondwh.scgoms_cwpicking,
          noondwh.mxdcss_dcss, noondwh.scinbound_inbound,
          noondwh.scgoms_goms
Grain   : user × wh_code × function × biz_date  (detail layer)
          wh_code × function × biz_date          (summary layer)
Biz day : 06:00 local time = midnight on biz_date
          (DATE(DATETIME(ts,'Asia/Dubai') - INTERVAL 6 HOUR))
Date    : Change biz_date_start / biz_date_end for a different day.
          Current window = yesterday.

DEFINITIONS
___________

effective_time_sec  : Active working time per job, excluding idle.
                      pick    → if first scan >600s after job open,
                                clock starts from first_scan, else full job
                      sort    → sum of per-scan gaps capped at 300s each
                      put     → job open → last completed line
                      receive → first box created → last box updated
                      stocktake → job opened → pending_confirmation
                                  (or closed if no pending_conf)
                      movement → MIN(tl.created_at) → MAX(tl.created_at)
```

```
 utilization_sec     : DATETIME_DIFF(last_event, first_event) per user/wh/day
                       across ALL functions combined. Capped at 43200s (12h).
                       Over-12h flag raised when raw window > 43200s.

 IPH_effective       : units / (effective_time_sec / 3600)
                       Only active job time in denominator.

 IPH_utilization     : units / (utilization_sec / 3600)
                       Total shift window in denominator.
                       Lower than IPH_effective — penalises idle time.

 NOTES
 ─────
 • movement time: day-level MIN→MAX, not per-job (see MP productivity notes)
 • sort: only GOMS Sort WHs; DXBID01 is Non-Sort / cwpicking
 • receive: scinbound.warehouse uses partner_code not partner_warehouse_code
 • stocktake: force_closed with no pending_confirmation excluded (incomplete)
 • username: prefers ops.idp.noon.team over noon.com SSO accounts
 • Source: directly from raw noondwh.* tables (not wh_picker_metrics_pace
   derived table) — real-time, reflects jobs completed up to ~30min ago

 REF: picker_raw_v2_wh.toml, wh_picker_metrics_pace.toml,
      MP productivity query (mp_productivity.sql)
═══════════════════════════════════════════════════════════════════════════════
*/

-- ── DATE WINDOW ──────────────────────────────────────────────────────────────
-- Change these two values to shift the analysis date.
-- biz_date_start = 02:00 UTC = 06:00 Dubai = business day start
-- For yesterday: start = DATE_SUB(CURRENT_DATE(), 1) 02:00 UTC
DECLARE biz_date_start TIMESTAMP DEFAULT TIMESTAMP(
   CONCAT(FORMAT_DATE('%Y-%m-%d', DATE_SUB(CURRENT_DATE('Asia/Dubai'), INTERVAL 1
DAY)), ' 02:00:00 UTC')
);
DECLARE biz_date_end TIMESTAMP DEFAULT TIMESTAMP(
```

```
   CONCAT(FORMAT_DATE('%Y-%m-%d', CURRENT_DATE('Asia/Dubai')), ' 02:00:00 UTC')
);

-- ── WH SCOPE ──────────────────────────────────────────────────────────────
-- Explicit allowlist. Mirrors the MP Productivity query scope.
-- Add/remove codes here to adjust coverage.
WITH wh_scope AS (
 SELECT wh_code FROM UNNEST([
   'AUHID01','AUHID02',
   'AUHFKZ01','AUHFKZ02','AUHFKZ03',
   'DXBID01','DXBID02','DXBID03','DXBID04','DXBID07',
   'DXSID01','DXSID02','DXSID03','DXSID04','DXSID06','DXSID07',
   'DXSLQD01',
   'DXBFNV01'
 ]) AS wh_code
),

/*
═══════════════════════════════════════════════════════════════
 FUNCTION 1 — PICK
 Source : noondwh.scgoms_cwpicking
 Units  : distinct picklist lines with a container scan
 Time   : job_start→job_end; if first scan >600s after job open,
          clock starts from first_scan (idle-strip rule)
 REF    : picker_raw_v2_wh.toml, wh_picker_metrics_pace.toml
═══════════════════════════════════════════════════════════════
*/
pick_jobs AS (
 SELECT
   src_w.partner_warehouse_code                                    AS wh_code,
   pj.id_user,
   DATE(DATETIME(pj.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)    AS biz_date,
   DATETIME(pj.created_at,'Asia/Dubai')                            AS job_start,
   DATETIME(pj.finsihed_picking_at,'Asia/Dubai')                   AS job_end,
   -- note: 'finsihed_picking_at' is a typo in the source schema — do not correct
   COUNT(DISTINCT pll.id_picklist_line)                            AS units,
```

```
    MIN(DATETIME(pcl.created_at,'Asia/Dubai'))                          AS first_scan,
    MAX(DATETIME(pcl.created_at,'Asia/Dubai'))                          AS last_scan
  FROM noondwh.scgoms_cwpicking.picking_job pj
  JOIN noondwh.scgoms_cwpicking.status pjst
      ON pj.id_status = pjst.id_status AND pjst.code = 'closed'
  JOIN noondwh.scgoms_cwpicking.picklist_line pll
      ON pll.id_picking_job = pj.id_picking_job
  JOIN noondwh.scgoms_cwpicking.warehouse src_w
      ON pll.id_warehouse = src_w.id_warehouse
  JOIN noondwh.scgoms_cwpicking.picking_container_line pcl
      ON pcl.id_picklist_line = pll.id_picklist_line
      AND pcl.id_picking_container_line IS NOT NULL
  JOIN wh_scope ws ON src_w.partner_warehouse_code = ws.wh_code
  WHERE pj.created_at >= biz_date_start AND pj.created_at < biz_date_end
  GROUP BY 1,2,3,4,5
),

pick AS (
  SELECT
    wh_code,
    id_user,
    biz_date,
    'pick'                                                              AS fn,
    SUM(units)                                                          AS units,
    -- effective time: if first scan >10min after job open, strip idle prefix
    SUM(
      CASE
        WHEN DATETIME_DIFF(first_scan, job_start, SECOND) > 600
        THEN DATETIME_DIFF(job_end, first_scan, SECOND)
        ELSE DATETIME_DIFF(job_end, job_start, SECOND)
      END
    )                                                                   AS
effective_sec,
    MIN(CASE
        WHEN DATETIME_DIFF(first_scan, job_start, SECOND) > 600
        THEN first_scan ELSE job_start
```

```
    END)                                                                AS
day_first_event,
    MAX(job_end)                                                        AS
day_last_event
  FROM pick_jobs
  GROUP BY 1,2,3,4
),

/*
═══════════════════════════════════════════════════════════════
  FUNCTION 2 — PUTAWAY
  Source : noondwh.mxdcss_dcss
  Units  : distinct completed job lines (id_status = 3)
  Time   : job open → last completed line updated_at
  Subtypes: 4=putaway_item, 5=putaway_case, 6=putaway_carton
  REF    : mp_productivity.sql (dcss CTE)
═══════════════════════════════════════════════════════════════
*/
put AS (
  SELECT
    w.partner_warehouse_code                                            AS wh_code,
    j.id_user,
    DATE(DATETIME(j.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)         AS biz_date,
    'put'                                                               AS fn,
    COUNT(DISTINCT jl.id_job_line)                                      AS units,
    DATETIME_DIFF(
      MAX(DATETIME(jl.updated_at,'Asia/Dubai')),
      MIN(DATETIME(j.created_at,'Asia/Dubai')),
      SECOND
    )                                                                   AS
effective_sec,
    MIN(DATETIME(j.created_at,'Asia/Dubai'))                            AS
day_first_event,
    MAX(DATETIME(jl.updated_at,'Asia/Dubai'))                           AS
day_last_event
  FROM noondwh.mxdcss_dcss.job j
```

```
 JOIN noondwh.mxdcss_dcss.job_line jl  ON jl.id_job = j.id_job
 JOIN noondwh.mxdcss_dcss.warehouse w  ON j.id_warehouse = w.id_warehouse
 JOIN wh_scope ws ON w.partner_warehouse_code = ws.wh_code
 WHERE j.id_job_subtype IN (4,5,6)     -- putaway subtypes only
   AND jl.id_status = 3                -- completed lines only
   AND j.created_at >= biz_date_start AND j.created_at < biz_date_end
   AND j.id_user IS NOT NULL
 GROUP BY 1,2,3,4
),

/*
═══════════════════════════════════════════════════════════════
 FUNCTION 3 — RECEIVE (INBOUND)
 Source : noondwh.scinbound_inbound
 Units  : distinct inbound boxes (id_status IN 13, 14)
 Time   : first box created → last box updated for user/wh/day
 NOTE   : scinbound.warehouse.partner_code maps to wh_scope.wh_code
          (not partner_warehouse_code — different column name)
 REF    : mp_productivity.sql (recv CTE)
═══════════════════════════════════════════════════════════════
*/
recv AS (
 SELECT
   w.partner_code                                                  AS wh_code,
   ib.id_user,
   DATE(DATETIME(ib.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)    AS biz_date,
   'receive'                                                       AS fn,
   COUNT(DISTINCT ib.id_inbound_box)                               AS units,
   DATETIME_DIFF(
     MAX(DATETIME(ib.updated_at,'Asia/Dubai')),
     MIN(DATETIME(ib.created_at,'Asia/Dubai')),
     SECOND
   )                                                               AS
effective_sec,
   MIN(DATETIME(ib.created_at,'Asia/Dubai'))                       AS
day_first_event,
```

```
   MAX(DATETIME(ib.updated_at,'Asia/Dubai'))                              AS
day_last_event
 FROM noondwh.scinbound_inbound.inbound_box ib
 JOIN noondwh.scinbound_inbound.warehouse w ON ib.id_warehouse = w.id_warehouse
 JOIN wh_scope ws ON w.partner_code = ws.wh_code -- note: partner_code, not
partner_warehouse_code
 WHERE ib.id_status IN (13,14)
   AND ib.created_at >= biz_date_start AND ib.created_at < biz_date_end
   AND ib.id_user IS NOT NULL
 GROUP BY 1,2,3,4
),

/*
═══════════════════════════════════════════════════════════════
 FUNCTION 4 — SORT
 Source : noondwh.scgoms_goms.audit_log
 Units  : sort scan events (id_audit_action=21, to='pending_putaway')
 Time   : sum of per-scan gaps capped at 300s each
          (LEAST(gap, 300) prevents idle/break inflation)
          last scan of day gets COALESCE(gap, 300) = 300s default
 NOTE   : only covers GOMS Sort WHs; DXBID01 uses cwpicking Non-Sort
 REF    : mp_productivity.sql (sort / sort_base CTEs)
═══════════════════════════════════════════════════════════════
*/
sort_raw AS (
 SELECT
   w.partner_warehouse_code                                              AS wh_code,
   al.id_user,
   DATE(DATETIME(al.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)       AS biz_date,
   DATETIME(al.created_at,'Asia/Dubai')                                  AS scan_at,
   LEAD(DATETIME(al.created_at,'Asia/Dubai')) OVER (
     PARTITION BY al.id_user, w.partner_warehouse_code,
                  DATE(DATETIME(al.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)
     ORDER BY al.created_at
   )                                                                     AS next_scan
 FROM noondwh.scgoms_goms.audit_log al
```

```
 JOIN noondwh.scgoms_goms.input_container_line icl
     ON icl.id_input_container_line = al.action_key
 JOIN noondwh.scgoms_goms.input_container ic
     ON ic.id_input_container = icl.id_input_container
 JOIN noondwh.scgoms_goms.warehouse w ON w.id_warehouse = ic.id_warehouse
 JOIN wh_scope ws ON w.partner_warehouse_code = ws.wh_code
 WHERE al.id_audit_action = 21
   AND JSON_EXTRACT_SCALAR(al.misc,'$.to') = 'pending_putaway'
   AND al.created_at >= biz_date_start AND al.created_at < biz_date_end
   AND al.id_user IS NOT NULL
),

sort AS (
 SELECT
   wh_code,
   id_user,
   biz_date,
   'sort'                                                                  AS fn,
   COUNT(*)                                                                AS units,
   -- capped gap sum = effective sort time without idle inflation
   SUM(LEAST(COALESCE(DATETIME_DIFF(next_scan, scan_at, SECOND), 300), 300))
                                                                           AS
effective_sec,
   MIN(scan_at)                                                            AS
day_first_event,
   MAX(scan_at)                                                            AS
day_last_event
 FROM sort_raw
 GROUP BY 1,2,3,4
),

/*
═══════════════════════════════════════════════════════════════
FUNCTION 5 — STOCKTAKE
Source : noondwh.mxdcss_dcss.job + audit_log
Units  : distinct job lines (excluding excess-source lines)
```

```
Time    : job opened_at → pending_confirmation_at
          (falls back to closed_at if no pending_confirmation exists)
Exclusion: force_closed jobs entirely
REF     : mp_productivity.sql (stocktake_job_times, inv_stocktake CTEs)
================================================================
*/
st_process_type AS (
-- Pre-join process_type per job to avoid subquery in JOIN (BigQuery restriction)
SELECT j.id_job, pt.code AS pt_code
FROM `noondwh.mxdcss_dcss.job` j
JOIN `noondwh.mxdcss_dcss.process` p       ON p.id_process      = j.id_process
JOIN `noondwh.mxdcss_dcss.process_type` pt ON pt.id_process_type = p.id_process_type
WHERE j.id_job_subtype = 10                -- stock_take jobs only
  AND j.created_at >= biz_date_start AND j.created_at < biz_date_end
),

st_job_times AS (
SELECT
  j.id_job, j.id_user, j.id_warehouse,
  MIN(CASE WHEN JSON_EXTRACT_SCALAR(al.misc,'$.to') = 'opened'
           THEN al.created_at END)                                    AS
opened_at,
  MIN(CASE WHEN JSON_EXTRACT_SCALAR(al.misc,'$.to') = 'pending_confirmation'
           THEN al.created_at END)                                    AS
pending_at,
  MIN(CASE WHEN JSON_EXTRACT_SCALAR(al.misc,'$.to') = 'closed'
           THEN al.created_at END)                                    AS
closed_at
FROM `noondwh.mxdcss_dcss.job` j
JOIN `noondwh.mxdcss_dcss.status` s  ON s.id_status = j.id_status
JOIN st_process_type jpt             ON jpt.id_job   = j.id_job
JOIN `noondwh.mxdcss_dcss.audit_log` al
    ON al.id_audit_action = 1
   AND al.action_key = j.id_job
   AND al.created_at >= biz_date_start AND al.created_at < biz_date_end
WHERE s.code = 'closed'                    -- force_closed excluded entirely
```

```sql
  AND j.created_at >= biz_date_start AND j.created_at < biz_date_end
  AND j.id_user IS NOT NULL
GROUP BY 1,2,3
),

stocktake AS (
SELECT
  w.partner_warehouse_code                                                AS wh_code,
  jt.id_user,
  DATE(DATETIME(jt.opened_at,'Asia/Dubai') - INTERVAL 6 HOUR)             AS biz_date,
  'stocktake'                                                             AS fn,
  COUNT(CASE
    WHEN jl.id_job_line IS NULL
      OR JSON_EXTRACT_SCALAR(jl.misc,'$.source') = 'excess' THEN NULL
    ELSE jl.id_job_line
  END)                                                                    AS units,
  SUM(TIMESTAMP_DIFF(
    COALESCE(jt.pending_at, jt.closed_at),
    jt.opened_at, SECOND
  ))                                                                      AS
effective_sec,
  MIN(DATETIME(jt.opened_at,'Asia/Dubai'))                                AS
day_first_event,
  MAX(DATETIME(COALESCE(jt.pending_at, jt.closed_at),'Asia/Dubai'))       AS
day_last_event
FROM st_job_times jt
LEFT JOIN `noondwh.mxdcss_dcss.job_line` jl ON jl.id_job       = jt.id_job
LEFT JOIN `noondwh.mxdcss_dcss.warehouse` w  ON w.id_warehouse   = jt.id_warehouse
JOIN wh_scope ws                            ON w.partner_warehouse_code = ws.wh_code
WHERE jt.opened_at IS NOT NULL
  AND COALESCE(jt.pending_at, jt.closed_at) IS NOT NULL
GROUP BY 1,2,3,4
),


/*
```

```
 ═══════════════════════════════════════════════════════════════
  FUNCTION 6 — INTERNAL MOVEMENT
  Source : noondwh.mxdcss_dcss.transaction_line + job
  Units  : SUM(tl.qty) — volume moved (can be > 1 per row)
  Time   : MIN→MAX of tl.created_at per user/wh/day (day-level,
           not per-job — understates time if multiple jobs per day)
  Filter : deep locations only (misc.is_deep = '1') to exclude
           forward-pick-face micro-moves
  NOTE   : id_user sourced from job table (not transaction_line)
  REF    : mp_productivity.sql (inv_movement CTE)
 ═══════════════════════════════════════════════════════════════
*/
movement AS (
  SELECT
    w.partner_warehouse_code                                          AS wh_code,
    j.id_user,
    DATE(DATETIME(tl.created_at,'Asia/Dubai') - INTERVAL 6 HOUR)      AS biz_date,
    'movement'                                                        AS fn,
    SUM(tl.qty)                                                       AS units,
    DATETIME_DIFF(
      MAX(DATETIME(tl.created_at,'Asia/Dubai')),
      MIN(DATETIME(tl.created_at,'Asia/Dubai')),
      SECOND
    )                                                                 AS
effective_sec,
    MIN(DATETIME(tl.created_at,'Asia/Dubai'))                         AS
day_first_event,
    MAX(DATETIME(tl.created_at,'Asia/Dubai'))                         AS
day_last_event
  FROM noondwh.mxdcss_dcss.transaction_line tl
  -- source location: only deep storage (is_deep = '1')
  JOIN noondwh.mxdcss_dcss.location l
      ON ('location', l.id_location) = (tl.src_key, CAST(tl.src_value AS INT64))
  LEFT JOIN noondwh.mxdcss_dcss.warehouse w    ON w.id_warehouse = l.id_warehouse
  JOIN noondwh.mxdcss_dcss.job j               ON j.id_job = tl.id_job
  JOIN noondwh.mxdcss_dcss.process p           ON p.id_process = j.id_process
```

```
 JOIN noondwh.mxdcss_dcss.process_type pt     ON pt.id_process_type =
p.id_process_type
 JOIN wh_scope ws ON w.partner_warehouse_code = ws.wh_code
 WHERE pt.code IN ('internal_movement','migration')
   AND tl.created_at >= biz_date_start AND tl.created_at < biz_date_end
   AND JSON_EXTRACT_SCALAR(l.misc,'$.is_deep') = '1'
   AND j.id_user IS NOT NULL
 GROUP BY 1,2,3,4
),

/*
═══════════════════════════════════════════════════════════════
 UNION ALL 6 FUNCTIONS
 Common schema: wh_code, id_user, biz_date, fn,
                units, effective_sec,
                day_first_event, day_last_event
═══════════════════════════════════════════════════════════════
*/
all_fns AS (
 SELECT * FROM pick
 UNION ALL SELECT * FROM put
 UNION ALL SELECT * FROM recv
 UNION ALL SELECT * FROM sort
 UNION ALL SELECT * FROM stocktake
 UNION ALL SELECT * FROM movement
),

/*
═══════════════════════════════════════════════════════════════
 UTILIZATION WINDOW — per user × wh × day ACROSS ALL FUNCTIONS
 first_event_all = earliest event across all 6 functions
 last_event_all  = latest  event across all 6 functions
 raw_shift_sec   = last_event_all - first_event_all (seconds)

 Cap at 43200s (12h):
   • If raw_shift_sec > 43200: set utilization_sec = 43200,
```

```
     flag over_12h = TRUE for ops review
   • Otherwise: utilization_sec = raw_shift_sec

 Why cap?  A picker could open a job at 06:01 and close the
 last job at 21:59 — 16h raw, but the actual shift is 12h.
 Values > 12h almost always indicate a forgotten open job or
 a double shift that should be reviewed.
═══════════════════════════════════════════════════════════════
*/
utilization_window AS (
 SELECT
   wh_code,
   id_user,
   biz_date,
   MIN(day_first_event)                                          AS
first_event_all,
   MAX(day_last_event)                                           AS
last_event_all,
   DATETIME_DIFF(
     MAX(day_last_event), MIN(day_first_event), SECOND
   )                                                             AS
raw_shift_sec,
   -- cap at 12h (43200s)
   LEAST(
     DATETIME_DIFF(MAX(day_last_event), MIN(day_first_event), SECOND),
     43200
   )                                                             AS
utilization_sec,
   -- flag: this user's shift window exceeded 12h — review recommended
   CASE
     WHEN DATETIME_DIFF(MAX(day_last_event), MIN(day_first_event), SECOND) > 43200
     THEN TRUE ELSE FALSE
   END                                                           AS
over_12h_flag
 FROM all_fns
 GROUP BY 1,2,3
```

```
),

/*
═══════════════════════════════════════════════════════════════
 USER × FUNCTION LEVEL — detail layer
 Joins utilization_window for the utilization denominator.
 IPH_effective   = units / (effective_sec / 3600)
 IPH_utilization = units / (utilization_sec / 3600)
                   (same denominator for all functions of a user)
═══════════════════════════════════════════════════════════════
*/
usernames AS (
 -- Prefer ops.idp.noon.team accounts (local WH logins) over noon.com SSO
 -- Falls back to raw id_user if not found in either table
 SELECT id_user,
   COALESCE(
     MIN(CASE WHEN username LIKE '%ops.idp.noon.team%'
              THEN SPLIT(username,'@')[OFFSET(0)] END),
     MIN(SPLIT(username,'@')[OFFSET(0)])
   ) AS username
 FROM (
   SELECT id_user, username FROM noondwh.scgoms_cwpicking.user
   UNION ALL
   SELECT id_user, username FROM noondwh.scgoms_goms.user
 )
 GROUP BY id_user
),

wh_group AS (
 -- Warehouse group label from wh_code prefix — mirrors mp_productivity logic
 SELECT wh_code,
   CASE
     WHEN wh_code LIKE 'AUHFKZ%'                              THEN 'FZ Kezad'
     WHEN wh_code LIKE 'AUHID%'                               THEN 'Kezad'
     WHEN wh_code LIKE 'DXBID%'                               THEN 'TP'
     WHEN wh_code LIKE 'DXSID%' OR wh_code LIKE 'DXSLQ%'   THEN 'Dubai South'
```

```
      WHEN wh_code = 'DXBFNV01'                                THEN 'F&V'
      ELSE 'Other'
    END AS warehouse_group
  FROM wh_scope
),

detail AS (
  SELECT
    f.biz_date,
    f.wh_code,
    wg.warehouse_group,
    COALESCE(u.username, CAST(f.id_user AS STRING))                AS username,
    f.id_user,
    f.fn                                                          AS
function_name,
    f.units,
    f.effective_sec,

    -- ── IPH WITHOUT UTILIZATION (job/scan time only) ──────────
    -- Denominator = active working time, idle stripped per function logic
    ROUND(SAFE_DIVIDE(f.units * 3600.0, f.effective_sec), 1)      AS
iph_effective,

    -- ── UTILIZATION WINDOW ────────────────────────────────────
    uw.utilization_sec,
    uw.raw_shift_sec,
    uw.over_12h_flag,      -- TRUE = shift window > 12h; ops should verify
    ROUND(uw.utilization_sec / 3600.0, 2)                         AS
utilization_hrs,

    -- ── IPH WITH UTILIZATION (full shift window) ──────────────
    -- Denominator = total shift time (first event → last event, max 12h)
    -- This is always ≤ iph_effective because the denominator is larger
    ROUND(SAFE_DIVIDE(f.units * 3600.0, uw.utilization_sec), 1)   AS
iph_utilization,
```

```
    -- ── UTILIZATION RATE ──────────────────────────────────────
    -- What fraction of shift window was active working time?
    ROUND(SAFE_DIVIDE(f.effective_sec, uw.utilization_sec) * 100, 1)  AS
utilization_pct,

    uw.first_event_all,
    uw.last_event_all
  FROM all_fns f
  JOIN utilization_window uw USING (wh_code, id_user, biz_date)
  LEFT JOIN usernames u  ON u.id_user = f.id_user
  LEFT JOIN wh_group wg  ON wg.wh_code = f.wh_code
),

/*
═══════════════════════════════════════════════════════════════
  SUMMARY — warehouse × function level
  Aggregates detail rows. IPH at summary level computed from
  SUM(units) / SUM(time) — not AVG(iph) per user — to avoid
  giving equal weight to low-volume users.

  iph_effective_summary   = total_units / (total_effective_sec / 3600)
  iph_utilization_summary = total_units / (total_utilization_sec / 3600)

  total_utilization_sec: sum of EACH USER's capped utilization_sec
  (not a single window across all users)

  over_12h_users: count of users flagged for shift > 12h — ops alert
═══════════════════════════════════════════════════════════════
*/
summary AS (
  SELECT
    biz_date,
    wh_code,
    warehouse_group,
    function_name,
    COUNT(DISTINCT id_user)                                          AS headcount,
```

```sql
    SUM(units)                                                          AS total_units,
    SUM(effective_sec)                                                  AS
total_effective_sec,
    -- Sum each user's capped utilization (not one big window across all users)
    SUM(utilization_sec)                                                AS
total_utilization_sec,
    ROUND(SAFE_DIVIDE(SUM(units) * 3600.0, SUM(effective_sec)), 1)      AS
iph_effective,
    ROUND(SAFE_DIVIDE(SUM(units) * 3600.0, SUM(utilization_sec)), 1)    AS
iph_utilization,
    ROUND(SAFE_DIVIDE(SUM(effective_sec), SUM(utilization_sec)) * 100, 1)
                                                                        AS
avg_utilization_pct,
    -- Users whose raw shift window exceeded 12h — review these
    COUNTIF(over_12h_flag = TRUE)                                       AS
over_12h_users,
    -- Average shift hours (capped) per user
    ROUND(SUM(utilization_sec) / COUNT(DISTINCT id_user) / 3600.0, 2) AS avg_shift_hrs
 FROM detail
 GROUP BY 1,2,3,4
)

/*
═══════════════════════════════════════════════════════════════════
 FINAL OUTPUTS — pick the block you need
═══════════════════════════════════════════════════════════════════
*/

-- ── OUTPUT A: SUMMARY — warehouse × function (default output) ─────
-- One row per warehouse × function for the biz_date window.
-- iph_effective   = active job time only (no idle)
-- iph_utilization = total shift window (first→last, max 12h)
-- over_12h_users  = flag count for ops review
SELECT
 biz_date,
 warehouse_group,
```

```
 wh_code,
 function_name,
 headcount,
 total_units,
 ROUND(total_effective_sec / 3600.0, 2)                                AS
total_effective_hrs,
 ROUND(total_utilization_sec / 3600.0, 2)                              AS
total_utilization_hrs,
 iph_effective,
 iph_utilization,
 avg_utilization_pct,
 avg_shift_hrs,
 over_12h_users
FROM summary
ORDER BY biz_date, warehouse_group, wh_code, function_name;


-- ── OUTPUT B: USER DETAIL — user × function (uncomment to use) ──────
-- One row per user × function.
-- Shows both IPH flavours + over_12h flag per user.
/*
SELECT
 biz_date,
 warehouse_group,
 wh_code,
 function_name,
 username,
 id_user,
 units,
 ROUND(effective_sec / 3600.0, 2)    AS effective_hrs,
 iph_effective,
 ROUND(utilization_hrs, 2)           AS shift_hrs_capped,
 iph_utilization,
 utilization_pct,
 over_12h_flag,                       -- TRUE = needs ops review
 raw_shift_sec,                       -- actual window before 12h cap
```

```
 first_event_all,
 last_event_all
FROM detail
ORDER BY biz_date, warehouse_group, wh_code, function_name, username;
*/


-- ── OUTPUT C: OVER-12H CASES (uncomment to use) ──────────────────
-- Shows only users where shift window > 12h.
-- Use this as the ops review alert list.
/*
SELECT DISTINCT
 biz_date,
 warehouse_group,
 wh_code,
 username,
 id_user,
 ROUND(raw_shift_sec / 3600.0, 2)         AS raw_shift_hrs,
 ROUND(utilization_hrs, 2)                AS capped_shift_hrs,
 first_event_all,
 last_event_all,
 '⚠️ shift window > 12h — verify if double shift or open job'
                                          AS review_note
FROM detail
WHERE over_12h_flag = TRUE
ORDER BY raw_shift_sec DESC;
*/
```

```
/*
═══════════════════════════════════════════════════════════════════════════
═══════
```

## WH PICKER METRICS PACE TABLE — User-level CW Picking daily metrics

```
-- Destination: noonbinimops.fulfillment.wh_picker_metrics_pace
-- Partitioned by date for efficient downstream queries.
-- Covers UAE (id_country=1), KSA (2,5), Egypt (3).
--  Lookback = 15 days.
═══════════════════════════════════════════════════════════════════════════
═══════
*/

#
═══════════════════════════════════════════════════════════════════════════
═══════
# FILE   : CWH_PACE/wh_picker_metrics_pace.toml
# PURPOSE: Builds and exposes the wh_picker_metrics_pace BigQuery
#          table — the canonical user-level CW Picking metrics table.
#          Contains two tasks:
#            1. wh_picker_metrics_pace  — CREATE OR REPLACE TABLE
#               partitioned by date with per-user picking metrics
#               (effective time, IPH, MS/BD/BS breakdown, tenure)
#            2. cwh_pace_picker_metrics — reads from that table and
#               pushes a warehouse-day summary to Google Sheets
# DAG    : Warehouse_Metrics_uae  (runs daily at 05:40 UTC)
# SOURCE : noondwh.scgoms_cwpicking  (picking_job, picklist_line,
#                                     picking_container, warehouse, user)
#          noonbinimops.fulfillment.wh_code_wh_category_mapping
# DEST   : noonbinimops.fulfillment.wh_picker_metrics_pace (BQ table)
#          Google Sheets picker_metrics tab
# AUTHOR : noon Minutes Ops
#
═══════════════════════════════════════════════════════════════════════════
═══════
```

```
#
# VARIABLE DEFINITIONS
# ──────────────────────────
# dag_id      : Parent DAG linking both tasks.
# task_id     : Unique name within the DAG.
# project_id  : GCP billing project for BigQuery.
# sql         : BigQuery DDL — CREATE OR REPLACE TABLE statement.
# query       : BigQuery SQL for the Sheets push task.
# depends     : List of task_ids that must complete before this task.
# url         : Destination Google Spreadsheet URL.
# worksheet   : Target tab name.
#
# KEY SQL TERMS — wh_picker_metrics_pace
# ──────────────────────────────────────────────────────────
# pjst.code = 'closed'          : only closed CW picking jobs
# INTERVAL 6 HOUR               : business day cutoff offset
# effective_time_sec            : adjusted picking time (excludes idle)
# Warehouse_group               : city-level grouping from wh_code prefix
#   AUH → Kezad, DXB → TP, RUH → Riyadh, JED → Jeddah, etc.
# wh_codes                      : STRING_AGG of all sub-warehouse categories
# user_ageing                   : days since user account created
# MS_* / BD_* / BS_*            : per-job-type (Multi Store, Batch Dynamic,
#                                 Batch Static) metrics
#
# KEY SQL TERMS — cwh_pace_picker_metrics
# ──────────────────────────────────────────────────────────
# working_hrs                   : effective picking hours per user
# active_hrs                    : first-to-last scan duration per user
# utilization                   : working_hrs / active_hrs ratio
# iph                           : items per effective working hour
# avg_working_hrs               : working_hrs / pickers
# avg_active_hrs                : active_hrs / pickers
#
═══════════════════════════════════════════════════════════════════════════════
═══════


WITH base AS (
-- ── CW Picking job-level grain ──────────────────────────────────────
SELECT
  src_w.partner_warehouse_code AS warehouse_code,
  pj.job_type,
  pj.id_user,
  pj.job_nr,
```

```
-- Business date (country aware, 6 AM cutoff)
DATE(
  DATETIME(
    pj.created_at,
    CASE
      WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
      WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
      WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
    END
  ) - INTERVAL 6 HOUR
) AS date,

-- Job start in local timezone
DATETIME(
  pj.created_at,
  CASE
    WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
    WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
    WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
  END
) AS job_accepted_at,

-- Job end in local timezone
DATETIME(
  pj.finsihed_picking_at,
  CASE
    WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
    WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
    WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
  END
) AS job_closed_at,

-- Raw UTC delta in seconds
DATETIME_DIFF(
  pj.finsihed_picking_at,
  pj.created_at,
  SECOND
) AS delta,

-- Distinct picked lines
COUNT(DISTINCT pll.id_picklist_line) AS picked_qty,

-- Unique containers per job
```

```
COUNT(
  DISTINCT COALESCE(pcr.container_barcode, pcr.tote_code)
) AS containers_used,

-- First scan timestamp
MIN(
  DATETIME(
    pcl.created_at,
    CASE
      WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
      WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
      WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
    END
  )
) AS min_picked_at

FROM noondwh.scgoms_cwpicking.picking_job pj
JOIN noondwh.scgoms_cwpicking.status pjst
  ON pj.id_status = pjst.id_status
JOIN noondwh.scgoms_cwpicking.picklist_line pll
  ON pll.id_picking_job = pj.id_picking_job
JOIN noondwh.scgoms_cwpicking.warehouse src_w
  ON pll.id_warehouse = src_w.id_warehouse
JOIN noondwh.scgoms_cwpicking.picking_container_line pcl
  ON pcl.id_picklist_line = pll.id_picklist_line
JOIN noondwh.scgoms_cwpicking.picking_container pcr
  ON pcr.id_picking_container = pcl.id_picking_container

WHERE
  pjst.code = 'closed'
  AND pcl.id_picking_container_line IS NOT NULL
  AND DATE(
        DATETIME(
          pj.created_at,
          CASE
            WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
            WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
            WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
          END
        ) - INTERVAL 6 HOUR
      ) >= CURRENT_DATE - 15   -- 15-day lookback for the table

GROUP BY ALL
),
```

```
base_jobs AS (
-- ── Effective time per job ──────────────────────────────────────────────
SELECT *,
  (CASE
    WHEN DATETIME_DIFF(min_picked_at, job_accepted_at, SECOND) > 600
    THEN DATETIME_DIFF(job_closed_at, min_picked_at, SECOND)
    ELSE delta
  END) AS effective_time_sec
FROM base
),

user_level AS (
-- ── User-day aggregation with per-job-type breakdown ────────────────────
SELECT
  date,
  -- City-level warehouse grouping from warehouse_code prefix
  CASE
    WHEN b.warehouse_code LIKE '%AUHFKZ%' THEN 'FZ Kezad'
    WHEN b.warehouse_code LIKE '%AUH%'    THEN 'Kezad'
    WHEN b.warehouse_code LIKE '%DXB%'    THEN 'TP'
    WHEN b.warehouse_code LIKE '%RUH%'    THEN 'Riyadh'
    WHEN b.warehouse_code LIKE '%JED%'    THEN 'Jeddah'
    WHEN b.warehouse_code LIKE '%BAH%'    THEN 'Bahrain'
    WHEN b.warehouse_code LIKE '%CAI%'    THEN 'Cairo'
    ELSE 'Other'
  END AS Warehouse_group,
  -- Aggregated list of detailed sub-warehouse categories for this user
  STRING_AGG(DISTINCT COALESCE(c.warehouse_catrgory, 'Other'), ', ' ORDER BY
COALESCE(c.warehouse_catrgory, 'Other')) AS wh_codes,
  id_user,
  SUM(picked_qty)          AS picked_qty,
  SUM(effective_time_sec)  AS effective_time_sec,
  SUM(containers_used)     AS containers_used,
  -- First scan: use min_picked_at if idle >10min before it, else job_accepted_at
  MIN(CASE WHEN DATETIME_DIFF(min_picked_at, job_accepted_at, SECOND) > 600
       THEN min_picked_at ELSE job_accepted_at END) AS first_scan_time,
  MAX(job_closed_at) AS last_scan_time,

  -- Multi Store Picking
  SUM(CASE WHEN job_type = 'multi_store_picking' THEN picked_qty END)        AS
MS_PickedQty,
  SUM(CASE WHEN job_type = 'multi_store_picking' THEN containers_used END)   AS
MS_UniqueContainers,
```

```
  SUM(CASE WHEN job_type = 'multi_store_picking' THEN effective_time_sec END) AS
MS_TimeSec,
  SAFE_DIVIDE(
    3600 * SUM(CASE WHEN job_type = 'multi_store_picking' THEN picked_qty END),
    SUM(CASE WHEN job_type = 'multi_store_picking' THEN effective_time_sec END)
  ) AS MS_IPH,

  -- Batch Dynamic
  SUM(CASE WHEN job_type = 'batch_dynamic' THEN picked_qty END)        AS
BD_PickedQty,
  SUM(CASE WHEN job_type = 'batch_dynamic' THEN containers_used END)   AS
BD_UniqueContainers,
  SUM(CASE WHEN job_type = 'batch_dynamic' THEN effective_time_sec END) AS
BD_TimeSec,
  SAFE_DIVIDE(
    3600 * SUM(CASE WHEN job_type = 'batch_dynamic' THEN picked_qty END),
    SUM(CASE WHEN job_type = 'batch_dynamic' THEN effective_time_sec END)
  ) AS BD_IPH,

  -- Batch Static
  SUM(CASE WHEN job_type = 'batch_static' THEN picked_qty END)        AS BS_PickedQty,
  SUM(CASE WHEN job_type = 'batch_static' THEN containers_used END)   AS
BS_UniqueContainers,
  SUM(CASE WHEN job_type = 'batch_static' THEN effective_time_sec END) AS BS_TimeSec,
  SAFE_DIVIDE(
    3600 * SUM(CASE WHEN job_type = 'batch_static' THEN picked_qty END),
    SUM(CASE WHEN job_type = 'batch_static' THEN effective_time_sec END)
  ) AS BS_IPH

FROM base_jobs b
LEFT JOIN noonbinimops.fulfillment.wh_code_wh_category_mapping c
  ON b.warehouse_code = c.warehouse_code
GROUP BY ALL
)

-- ── Final: add user tenure info ──────────────────────────────────────────
SELECT
  a.*,
  u.username,
  DATE(u.created_at) AS user_created_at,
  DATETIME_DIFF(date, DATE(u.created_at), DAY) AS user_ageing  -- days since account
creation
FROM user_level a
LEFT JOIN noondwh.scgoms_cwpicking.user u ON a.id_user = u.id_user
```

```
/*
================================================================================
========
```

## TRANSFER TRACKER (SUPPLY CHAIN) — ITEM-LEVEL STATUS DASHBOARD

```
================================================================================
========
  PURPOSE  : Tracks all inter-warehouse (hub transfer) items per day, broken
             down by source warehouse, destination darkstore, and country.
             Reports each item's stage in the transfer pipeline, from pending
             pick through putaway / cancellation.

  SOURCE   : noondwh.scgoms_goms (core OMS), noondwh.scgoms_cwpicking (WMS)
  GRAIN    : One row per (creation_date, source_warehouse, destination_darkstore)
  FILTERS  : Last 7 days | Marketplace id_mp = 9 (Noon Instant)

  AUTHOR   : —
  LAST     : (auto-refresh via refresh_time column)
================================================================================
========
*/

/*
--------------------------------------------------------------------------------
--------------------
   VARIABLE DEFINITIONS
   BigQuery doesn't support SQL DECLARE in SELECT context, so logical
   constants are documented here as comments and embedded inline below.

   DATE_WINDOW         : CURRENT_DATE() - 7   (rolling 7-day lookback)
```

```
  MARKETPLACE_ID      : 9                (Noon Instant marketplace)
  MP_STATUS_ACTIVE    : 30                 (item is live / not cancelled)
  MP_STATUS_CANCELLED : 31                      (item has been cancelled)
  STATUS_PACKED       : 17               (item is packed)
  STATUS_SORTING      : 36                (item is in sorting/staging area)
  STATUS_RECEIVED     : 69                 (item received at destination DS)
  STATUS_PENDING_PICK : 44 OR 1                (item awaiting picker assignment)
  STATUS_DELETED      : 32                (item record soft-deleted)
  HT_STATUS_STAGED    : 1                  (hub transfer line = staged)
  HT_STATUS_RELEASED  : 57                    (hub transfer line = released from staging)
  HT_STATUS_DISPATCHED: 36                     (hub transfer line = dispatched)
  HT_HEADER_EN_ROUTE  : 26                     (hub transfer header = in transit)
  HT_HEADER_RECEIVING : 13                    (hub transfer header = at receiving dock)

  CANCEL REASON CODES (scgoms_cwpicking.reason, type = 'item_cancellation'):
    ops_marked_missing : JLM
    oos_stock          : Stock_Unavailable_OOS | cutoff_passed_with_no_stock
    sla_breach         : Picking_Not_Completed_Within_SLA | cutoff_passed
──────────────────────────────────────────────────────────────────────────────
──────────────────────────── */

SELECT
  /* ── DIMENSION: Creation date, localised per country timezone ──────────── */
  CASE
    WHEN src_w.id_country = 1          THEN DATE(i.created_at, 'Asia/Dubai')   -- UAE
    WHEN src_w.id_country IN (2, 6)     THEN DATE(i.created_at, 'Asia/Riyadh')  -- KSA / BH
    WHEN src_w.id_country = 3          THEN DATE(i.created_at, 'Africa/Cairo') -- EGY
  END AS creation_date,

  /* ── DIMENSION: Source warehouse (hub / DC)
──────────────────────────────────────────── */
  src_w.partner_warehouse_code AS source_warehouse,

  /* ── DIMENSION: Warehouse category (DC, Hub, etc.) ──────────────────────── */
  COALESCE(c.warehouse_catrgory, 'Other') AS warehouse_category,

  /* ── DIMENSION: ISO country code
──────────────────────────────────────────────────── */
  CASE
    WHEN src_w.id_country = 1 THEN 'AE'
    WHEN src_w.id_country = 2 THEN 'SA'
    WHEN src_w.id_country = 6 THEN 'BH'
    WHEN src_w.id_country = 3 THEN 'EG'
  END AS country,
```

```
  /* ── DIMENSION: Destination darkstore
  ──────────────────────────────────────────────── */
  dst_w.partner_warehouse_code AS destination_darkstore,
  dst_w.name                   AS destination_name,

  /*
  ═══════════════════════════════════════════════════════════════════════════
  ═══════════
     METRIC BLOCK 1 — VOLUME OVERVIEW

  ═══════════════════════════════════════════════════════════════════════════
  ═══════════ */

  /* All distinct items created in the date window for this transfer leg */
  COUNT(DISTINCT i.id_item) AS total_qty,

  /* Items whose marketplace status = 31 (cancelled) */
  COUNT(DISTINCT CASE
    WHEN i.id_mp_status = 31 THEN i.id_item
  END) AS cancelled_qty,

  /*
  ═══════════════════════════════════════════════════════════════════════════
  ═══════════
     METRIC BLOCK 2 — PIPELINE STAGE COUNTERS
     Each bucket is mutually exclusive in intent, though an item may
     technically satisfy more than one condition if data is inconsistent.

  ═══════════════════════════════════════════════════════════════════════════
  ═══════════ */

  /* ── Stage: Pending Picking
  ──────────────────────────────────────────────────────── */
     Item is active (mp_status=30) and awaiting picker assignment.
     id_status 44 = picker not yet assigned; 1 = newly injected item.     */
  COUNT(DISTINCT CASE
    WHEN (i.id_status = 44 OR i.id_status = 1)
     AND i.id_mp_status = 30
    THEN i.id_item
  END) AS pending_picking,

  /* ── Stage: Pending Staging
  ──────────────────────────────────────────────────────── */
```

```
   Item has been picked (status 36=sorting or 17=packed),
   is active (mp_status=30), and its hub-transfer line is staged (htl=1). */
 COUNT(DISTINCT CASE
  WHEN i.id_status IN (36, 17)
   AND i.id_mp_status = 30
   AND htl.id_status = 1
  THEN i.id_item
 END) AS pending_staging,

 /* ── Stage: Pending Release from Staging ──────────────────────────────────
   Item is packed (17), active (30), and HTL shows released (57).
   Awaiting consolidation / load-out authorisation.                    */
 COUNT(DISTINCT CASE
  WHEN i.id_status = 17
   AND i.id_mp_status = 30
   AND htl.id_status = 57
  THEN i.id_item
 END) AS pending_release_staging,

 /* ── Stage: Pending Dispatch
 ──────────────────────────────────────────────────────────────────────

   HTL is dispatched (36) but hub-transfer header is still en-route (26).
   Items are loaded on vehicle but trip not yet confirmed at destination.  */
 COUNT(DISTINCT CASE
  WHEN i.id_mp_status = 30
   AND htl.id_status = 36
   AND ht.id_status  = 26
  THEN i.id_item
 END) AS pending_dispatch,

 /* ── Stage: Pending Receiving
 ──────────────────────────────────────────────────────────────────────

   HTL dispatched (36), hub-transfer header at receiving dock (13).
   Vehicle has arrived; GRN not yet confirmed.                         */
 COUNT(DISTINCT CASE
  WHEN i.id_mp_status = 30
   AND htl.id_status = 36
   AND ht.id_status  = 13
  THEN i.id_item
 END) AS pending_receiving,

 /* ── Stage: In Transit (catch-all) ────────────────────────────────────────
   Not cancelled, not deleted, and putaway has NOT been recorded.
   Covers items somewhere in the transfer pipeline with no end-state yet. */
```

```
  COUNT(DISTINCT CASE
    WHEN i.id_mp_status != 31
     AND i.id_status    != 32  -- exclude soft-deleted records
     AND JSON_EXTRACT_SCALAR(i.misc, '$.putaway_status') IS NULL
    THEN i.id_item
  END) AS transit,

  /* ── Stage: Received at Darkstore
  ──────────────────────────────────────────────────
     Item is packed (17), active (30), HTL shows received at DS (69).
     GRN completed; item is physically at destination but not yet put away.  */
  COUNT(DISTINCT CASE
    WHEN i.id_status    = 17
     AND i.id_mp_status = 30
     AND htl.id_status  = 69
    THEN i.id_item
  END) AS received_at_darkstore,

  /*
  ═══════════════════════════════════════════════════════════════════════════
  ═══════════
     METRIC BLOCK 3 — PUTAWAY OUTCOMES

  ═══════════════════════════════════════════════════════════════════════════
  ═══════════ */

  /* Putaway confirmed complete (misc.putaway_status = 'completed') */
  COUNT(DISTINCT CASE
    WHEN i.id_mp_status = 30
     AND JSON_EXTRACT_SCALAR(i.misc, '$.putaway_status') = 'completed'
    THEN i.id_item
  END) AS putaway_completed,

  /* Item could not be located during putaway (misc.putaway_status = 'missing') */
  COUNT(DISTINCT CASE
    WHEN i.id_mp_status = 30
     AND JSON_EXTRACT_SCALAR(i.misc, '$.putaway_status') = 'missing'
    THEN i.id_item
  END) AS missing,

  /*
  ═══════════════════════════════════════════════════════════════════════════
  ═══════════
     METRIC BLOCK 4 — CANCELLATION REASON BREAKDOWN
```

```
    All three counters require:
      - mp_status = 31      (cancelled)
      - picklist cancelled  (pllst.code = 'cancelled')
    Then split by cancellation reason code (rr.code).

================================================================================
================ */

  /* Ops-marked missing — picker confirmed item not found on shelf (reason: JLM) */
  COUNT(DISTINCT CASE
    WHEN i.id_mp_status  = 31
     AND pllst.code      = 'cancelled'
     AND rr.code         = 'JLM'
    THEN i.id_item
  END) AS ops_marked_missing,

  /* Out-of-stock cancellation — no sellable stock available at pick time */
  COUNT(DISTINCT CASE
    WHEN i.id_mp_status  = 31
     AND pllst.code      = 'cancelled'
     AND rr.code         IN ('Stock_Unavailable_OOS', 'cutoff_passed_with_no_stock')
    THEN i.id_item
  END) AS oos_stock,

  /* SLA breach cancellation — pick not completed within the promised window */
  COUNT(DISTINCT CASE
    WHEN i.id_mp_status  = 31
     AND pllst.code      = 'cancelled'
     AND rr.code         IN ('Picking_Not_Completed_Within_SLA', 'cutoff_passed')
    THEN i.id_item
  END) AS sla_breach,

  /* ── Metadata: last refresh timestamp (per-country timezone) ──────────── */
  CASE
    WHEN src_w.id_country = 1         THEN DATETIME(CURRENT_TIMESTAMP(),
'Asia/Dubai')
    WHEN src_w.id_country IN (2, 6)   THEN DATETIME(CURRENT_TIMESTAMP(),
'Asia/Riyadh')
    WHEN src_w.id_country = 3         THEN DATETIME(CURRENT_TIMESTAMP(),
'Africa/Cairo')
  END AS refresh_time
```

```
/*
-------------------------------------------------------------------------------
-------------------------
  FROM / JOIN BLOCK
  Join order:
    1. item (fact table)
    2. src_w / dst_w  — warehouse master for source and destination
    3. c              — warehouse-category mapping (noonbinimops, external)
    4. sl / sh        — shipment line → shipment header (links item to HT)
    5. htl / htg / ht — hub-transfer line → group → header (physical trip)
    6. tr / t / pll   — WMS: transfer request → line → picklist line
    7. rr / pllst     — WMS: cancellation reason & picklist status
-------------------------------------------------------------------------------
------------------------- */
FROM
  noondwh.scgoms_goms.item i

/* Source warehouse: resolves country, timezone, and warehouse code */
LEFT JOIN noondwh.scgoms_goms.warehouse src_w
  ON i.id_warehouse = src_w.id_warehouse

/* Destination darkstore: stored as JSON in item.misc → cast to INT64 */
LEFT JOIN noondwh.scgoms_goms.warehouse dst_w
  ON CAST(JSON_EXTRACT_SCALAR(i.misc, '$.id_dst_warehouse') AS INT64) =
dst_w.id_warehouse

/* Shipment line (item → shipment mapping) */
LEFT JOIN noondwh.scgoms_goms.shipment_line sl
  USING (id_item)

/* Shipment header (needed to link to hub_transfer_line) */
LEFT JOIN noondwh.scgoms_goms.shipment sh
  USING (id_shipment)

/* Hub transfer LINE: links the shipment to a physical transfer trip
   Only 'forward_nim' type transfers (DC → darkstore direction)          */
LEFT JOIN noondwh.scgoms_goms.hub_transfer_line htl
  ON  (htl.type, htl.line_type, htl.line_id)
    = ('forward_nim', 'shipment', sh.id_shipment)

/* Hub transfer GROUP (intermediate grouping layer, joined for completeness) */
LEFT JOIN noondwh.scgoms_goms.hub_transfer_group htg
  ON htg.id_hub_transfer_group = htl.id_hub_transfer_group
```

```
/* Hub transfer HEADER (vehicle-level trip record; carries overall trip status) */
LEFT JOIN noondwh.scgoms_goms.hub_transfer ht
  USING (id_hub_transfer)

/* Warehouse category mapping (external reference, noonbinimops project) */
LEFT JOIN noonbinimops.fulfillment.wh_code_wh_category_mapping c
  ON src_w.partner_warehouse_code = c.warehouse_code

/* ── WMS (cwpicking) joins — filtered to last 7 days to limit scan cost ── */

/* Transfer REQUEST header in the WMS (matches by transfer_nr on the item) */
LEFT JOIN noondwh.scgoms_cwpicking.transfer_request tr
  ON  tr.transfer_nr = i.transfer_nr
  AND DATE(tr.created_at) >= CURRENT_DATE() - 7

/* Transfer REQUEST LINE (SKU-level; joined by request + SKU) */
LEFT JOIN noondwh.scgoms_cwpicking.transfer_request_line t
  ON  tr.id_transfer_request = t.id_transfer_request
  AND i.catalog_sku          = t.sku
  AND DATE(t.created_at)     >= CURRENT_DATE() - 7

/* Picklist LINE (actual pick task; 1-to-1 with transfer request line) */
LEFT JOIN noondwh.scgoms_cwpicking.picklist_line pll
  ON  t.id_transfer_request_line = pll.id_transfer_request_line
  AND DATE(pll.created_at)       >= CURRENT_DATE() - 7

/* Cancellation REASON master (only item-level cancellation reasons) */
LEFT JOIN noondwh.scgoms_cwpicking.reason rr
  ON  rr.id_reason = pll.id_cancellation_reason
  AND rr.type      = 'item_cancellation'

/* Picklist STATUS master (e.g. 'cancelled', 'completed') */
LEFT JOIN noondwh.scgoms_cwpicking.status pllst
  ON pll.id_status = pllst.id_status

/*
──────────────────────────────────────────────────────────────────────────────
  FILTER:
    - Rolling 7-day window on item creation date (aligns with WMS join filter)
    - Only Noon Instant marketplace (id_mp = 9)
──────────────────────────────────────────────────────────────────────────────
──────────────────────────── */
WHERE
```

```
  DATE(i.created_at) >= CURRENT_DATE() - 7
  AND i.id_mp = 9

/* GROUP BY ALL: collapses all non-aggregated columns automatically (BigQuery) */
GROUP BY
  ALL




/*
================================================================================
========
```

## DRIVER TASK SUMMARY  (MIDDLE MILE) — TRIP-LEVEL OPERATIONAL DASHBOARD

```
================================================================================
========
  PURPOSE  : Tracks all outbound driver tasks over the last 15 days for UAE
             central warehouses. Reports trip count, pallet count, and item
             volume per warehouse group per day.

  SOURCE   : noondwh.scgoms_goms
  GRAIN    : One row per (warehouse_group, driver_task_created_date)
  FILTERS  : Last 15 days | UAE only (id_country = 1)

  AUTHOR   : —
  LAST     : (auto-refresh via CURRENT_DATE)
================================================================================
========
*/

/*
--------------------------------------------------------------------------------
--------------------------
   VARIABLE DEFINITIONS
   BigQuery doesn't support SQL DECLARE in SELECT context, so logical
   constants are documented here and embedded inline below.

   DATE_WINDOW            : CURRENT_DATE() - 15       (rolling 15-day lookback)
```

```
  UAE_COUNTRY_ID          : 1                    (id_country for UAE)
  ENTITY_TYPE_PALLET      : 'pallet'               (driver_task_line.entity_type filter)
  HUB_TRANSFER_LINE_TYPE  : 'shipment'               (hub_transfer_line.line_type filter)

  AUDIT ACTION IDs (scgoms_goms — do NOT reuse across datasets):
    AUDIT_TASK_COMPLETED  : 18  (driver_task  → completed)
    AUDIT_ROUTE_ARRIVED   : 16  (driver_task_route → arrived)
    AUDIT_TRANSFER_CLOSED : 11  (hub_transfer → closed)

  WAREHOUSE GROUP MAPPING (source_warehouse → business label):
    DXBID01, DXBID02 → TP_ambient
    DXBID03, DXBID04 → TP_chiller
    *DXB*            → TP_fnv
    *FKZ*            → Frozen
    AUHID01, AUHID02 → kezad_ambient
    AUHID03, AUHID04 → kezad_chiller
    *AUH*            → Kezad

  STATUS CODE MAPPING (driver_task_line.id_status → business label):
    pending    → In Transit
    handed_over → Completed
    skipped    → Returned to WH

  NOTE: Source warehouse is resolved from shipment.id_warehouse,
        NOT from hub_transfer polymorphic IDs (source_id / destination_id).
──────────────────────────────────────────────────────────────────────────────
──────────────────────────── */


-- ============================================================
-- CTE 1: base
-- Grain: one row per task × route × pallet × shipment × item
-- Resolves warehouse, audit timestamps, shipment & item counts
-- ============================================================
WITH base AS (
  SELECT
    dt.id_driver_task,
    dt.task_nr,
    u.email,
    u.username,
    ht.canter_code                    AS pallet_code,
    CASE
      WHEN s.code = 'pending'     THEN 'In Transit'
      WHEN s.code = 'handed_over' THEN 'Completed'
```

```
    WHEN s.code = 'skipped'     THEN 'Returned to WH'
    ELSE s.code
  END                           AS line_status,
  r.code                        AS hold_reason,
  dt.created_at                 AS task_created_at,
  MAX(al.created_at)            AS task_completed_at,
  dst_w.partner_warehouse_code  AS destination_dark_store,
  src_w.partner_warehouse_code  AS source_warehouse,
  IF(dtl.id_hold_reason IS NOT NULL, alu.username, NULL) AS
person_who_terminated_the_task,
  MIN(al1.created_at)           AS mark_arrived_timestamp,   -- first 'arrived' on route
  MAX(al2.created_at)           AS received_at_darkstore,    -- last hub_transfer 'closed'
  COUNT(DISTINCT sh.id_shipment)  AS qty_shipments,
  COUNT(DISTINCT it.id_item)      AS qty_items

FROM `noondwh.scgoms_goms.driver_task_line` dtl
LEFT JOIN `noondwh.scgoms_goms.driver_task_route` dtr
  USING (id_driver_task_route)
LEFT JOIN `noondwh.scgoms_goms.driver_task` dt
  USING (id_driver_task)
LEFT JOIN `noondwh.scgoms_goms.warehouse` dst_w
  ON dst_w.id_warehouse = dtr.id_destination
LEFT JOIN `noondwh.scgoms_goms.status` s
  ON s.id_status = dtl.id_status
LEFT JOIN `noondwh.scgoms_goms.reason` r
  ON r.id_reason = dtl.id_hold_reason
-- entity_type/entity_id is polymorphic; scoped to 'pallet' rows only
LEFT JOIN `noondwh.scgoms_goms.hub_transfer` ht
  ON (dtl.entity_type, dtl.entity_id) = ('pallet', ht.id_hub_transfer)
LEFT JOIN `noondwh.scgoms_goms.hub_transfer_line` htl
  USING (id_hub_transfer)
LEFT JOIN `noondwh.scgoms_goms.hub_transfer_group` htg
  USING (id_hub_transfer_group)
LEFT JOIN `noondwh.scgoms_goms.shipment` sh
  ON (htl.line_id, htl.line_type) = (sh.id_shipment, 'shipment')
LEFT JOIN `noondwh.scgoms_goms.shipment_line` shl
  ON shl.id_shipment = sh.id_shipment
-- Source warehouse from shipment — hub_transfer IDs are polymorphic
LEFT JOIN `noondwh.scgoms_goms.warehouse` src_w
  ON src_w.id_warehouse = CAST(sh.id_warehouse AS INT64)
LEFT JOIN `noondwh.scgoms_goms.item` it
  USING (id_item)
LEFT JOIN `noondwh.scgoms_goms.user` u
  USING (id_user)
```

```
  -- Audit: task completed timestamp (action 18)
  LEFT JOIN `noondwh.scgoms_goms.audit_log` al
    ON  al.action_key      = dt.id_driver_task
    AND al.id_audit_action = 18
    AND JSON_EXTRACT_SCALAR(al.misc, '$.to') = 'completed'
  LEFT JOIN `noondwh.scgoms_goms.user` alu
    ON alu.id_user = al.id_user
  -- Audit: route arrived timestamp (action 16)
  LEFT JOIN `noondwh.scgoms_goms.audit_log` al1
    ON  al1.action_key      = dtr.id_driver_task_route
    AND al1.id_audit_action = 16
    AND JSON_EXTRACT_SCALAR(al1.misc, '$.to') = 'arrived'
  -- Audit: hub_transfer closed = received at darkstore (action 11)
  LEFT JOIN `noondwh.scgoms_goms.audit_log` al2
    ON  al2.action_key      = ht.id_hub_transfer
    AND al2.id_audit_action = 11
    AND JSON_EXTRACT_SCALAR(al2.misc, '$.to') = 'closed'

  WHERE DATE(dtl.created_at, 'Asia/Dubai') >= DATE_SUB(CURRENT_DATE('Asia/Dubai'),
INTERVAL 15 DAY)
    AND src_w.id_country = 1   -- UAE only

  GROUP BY ALL
),

-- ============================================================
-- CTE 2: final_df
-- Grain: one row per task × destination darkstore
-- Maps source warehouses to business group labels
-- ============================================================
final_df AS (
  SELECT
    CASE
      WHEN source_warehouse IN ('DXBID01', 'DXBID02') THEN 'TP_ambient'
      WHEN source_warehouse IN ('DXBID03', 'DXBID04') THEN 'TP_chiller'
      WHEN source_warehouse LIKE '%DXB%'              THEN 'TP_fnv'
      WHEN source_warehouse LIKE '%FKZ%'              THEN 'Frozen'
      WHEN source_warehouse IN ('AUHID01', 'AUHID02') THEN 'kezad_ambient'
      WHEN source_warehouse IN ('AUHID03', 'AUHID04') THEN 'kezad_chiller'
      WHEN source_warehouse LIKE '%AUH%'              THEN 'Kezad'
    END                                     AS warehouse,
    id_driver_task,
    task_nr,
    DATE(task_created_at, 'Asia/Dubai')             AS driver_task_created_time,
```

```
    DATETIME(mark_arrived_timestamp, 'Asia/Dubai')   AS mark_arrived_timestamp,
    destination_dark_store                           AS dst_warehouse,
    COUNT(DISTINCT pallet_code)                      AS qty_pallets,
    SUM(qty_shipments)                               AS qty_shipments,
    SUM(qty_items)                                   AS qty_items

  FROM base
  GROUP BY ALL
)

-- ============================================================
-- FINAL OUTPUT
-- Grain: one row per (warehouse_group, driver_task_created_date)
-- ============================================================
SELECT
  warehouse,
  driver_task_created_time,
  COUNT(DISTINCT task_nr)  AS trips,
  SUM(qty_pallets)         AS pallets,
  SUM(qty_items)           AS items
FROM final_df
WHERE warehouse IS NOT NULL
  AND warehouse <> ''
GROUP BY ALL;




/*
================================================================================
========
```

## ULTRAFRESH PIPELINE STATUS

```
================================================================================
========
  PURPOSE  : Tracks all outbound transfer items for the last ~24 hours across
             Ultrafresh warehouses (DXBID07, AUHID07), broken down by slot,
             type, source warehouse, and destination darkstore.
             Reports item counts at each pipeline stage from pending pick
             through putaway / cancellation.
```

```
  SOURCE   : noondwh.scgoms_goms
  GRAIN    : One row per (created_at, slot, type, source_warehouse,
             destination_darkstore)
  FILTERS  : Rolling ~24h window | Marketplace id_mp = 9 (Noon Instant)
             | UAE only (id_country = 1) | Ultrafresh warehouses only

  AUTHOR   : —
  LAST     : (auto-refresh via CURRENT_TIMESTAMP)
================================================================================
========
*/

/*
──────────────────────────────────────────────────────────────────────────────
──────────────────────────

   VARIABLE DEFINITIONS
   BigQuery doesn't support SQL DECLARE in SELECT context, so logical
   constants are documented here and embedded inline below.

   DATE_WINDOW_NON_EGG  : CURRENT_TIMESTAMP() - 1 DAY + 13 HOUR   (non-egg
lookback start)
   DATE_WINDOW_EGG      : CURRENT_TIMESTAMP() - 1 DAY + 5 HOUR    (egg lookback
start — earlier window)
   MARKETPLACE_ID       : 9                                  (Noon Instant)
   MP_STATUS_ACTIVE     : 30                                   (item is live / not cancelled)
   MP_STATUS_CANCELLED  : 31                                       (item has been cancelled)
   STATUS_PACKED        : 17                                 (item is packed)
   STATUS_SORTING       : 36                                  (item is in sorting/staging area)
   STATUS_PENDING_PICK  : 44                                      (item awaiting picker assignment)
   STATUS_DELETED       : 32                                  (item record soft-deleted)
   HT_STATUS_STAGED     : 1                                    (hub_transfer_line = staged)
   HT_STATUS_RELEASED   : 57                                      (hub_transfer_line = released from
staging)
   HT_STATUS_DISPATCHED : 36                                       (hub_transfer_line = dispatched)
   HT_STATUS_RECEIVED   : 69                                     (hub_transfer_line = received at
darkstore)
   HT_HEADER_EN_ROUTE   : 26                                       (hub_transfer header = in transit)
   HT_HEADER_RECEIVING  : 13                                       (hub_transfer header = at receiving
dock)
   UAE_COUNTRY_ID       : 1                                  (id_country for UAE)

   SLOT LOGIC (non-egg, DXB/AUH/SHJ warehouses):
     Slot_1 : item.created_at in [T-1 13:00, T-1 21:00)
```

```
    Slot_2 : item.created_at in [T-1 21:00, T 13:00)
    Non DXB/AUH/SHJ destinations always → Slot_2

  WAREHOUSE CATEGORIES:
    AUHFKZ01/02/03  →  Frozen_TP
    DXBID01/02      →  Ambient_TP
    DXBID03/04      →  Chiller_TP
    DXBID07         →  Ultrafresh_TP
    DXBFNV01        →  FnV
    AUHID01/02      →  Ambient_AUH
    AUHID03/04      →  Chiller_AUH
    AUHID07         →  Ultrafresh_AUH

  TYPE LOGIC:
    crossdoc job + no transfer_nr  →  CROSS_DOC
    all others                     →  PICKING

  NOTE: egg SKUs use a wider time window (5h vs 13h offset) to capture
        earlier morning slots. The egg_base CTE currently mirrors non_egg_base
        output — extend egg-specific filters here if needed.
------------------------------------------------------------------------------
------------------------------ */



-- ============================================================
-- CTE 1: item_base_non_egg
-- Raw item rows for non-egg SKUs — last ~24h (13h offset)
-- Excludes a hardcoded list of egg/excluded catalog_skus
-- ============================================================
WITH item_base_non_egg AS (
  SELECT
    created_at,
    id_mp_status,
    id_status,
    misc,
    id_warehouse,
    id_item,
    id_mp,
    transfer_nr
  FROM `noondwh.scgoms_goms.item`
  WHERE created_at >= (CURRENT_TIMESTAMP() - INTERVAL 1 DAY + INTERVAL 13
HOUR)
    AND catalog_sku NOT IN (
```

'ZDBD96B2A31B90DA3535BZ-1','ZF70A41840211D44119A1Z-1','Z68E570CD7B584F2CEFFDZ-1',

'Z57C9E4E200DA0C9957ACZ-1','ZFE1F92583C9094B4E78DZ-1','ZFE40744FFD25831970A5Z-1',

'Z1902BB576BE48EE08D00Z-1','Z2BB15284D64530C1C05BZ-1','ZC9A51E5FA6608F752F70Z-1',

'Z4523F9F7005238D843D5Z-1','Z3789EBEB6595CE35DCCEZ-1','Z237C5055361C50A7A71FZ-1',

'Z6698D2D2BDBA41D58CB3Z-1','Z6FDBF76C2C322B067228Z-1','ZF26DC7CC594C8894A817Z-1',

'ZB35F71329465C90AEFFAZ-1','Z4BDD0FC2EEF96B2C5187Z-1','Z47FC5A39CB05156F8136Z-1',

'ZE65EEC6C1D8053F9E0D2Z-1','ZEEE3AB1C9858EB118FD2Z-1','Z397668FD6BFB5340E488Z-1',

'Z570B4BC1E994BBCAF2ABZ-1','Z6399810E6C33F913DA30Z-1','ZC695BAB888341A6915CEZ-1',

'ZF8BA5447E2C0C0B9574DZ-1','Z12C1214A9D7BB114F1C8Z-1','ZC14E5682390F8D7CC77AZ-1',

'Z59E57306C5391D31E795Z-1','ZD7C9862E3EBF56D7FD61Z-1','Z8679040FE648972CF10DZ-1',

'Z8979B9749AECF808C551Z-1','ZD30C97A6B25E02B50C1CZ-1','ZC04BA8EB8027D85D8CC4Z-1',

'ZD871DE485C3BF2CA33FDZ-1','Z756CF10F309513A05413Z-1','ZD53F0499F4CE32BB4E2CZ-1',

'Z3CCB27EA61F1AC63D820Z-1','Z814D23C0A6E15FD13E80Z-1','Z4D24B5134F244B218C2BZ-1',

'Z8523783B8F4D6BDF8DD2Z-1','Z55E99103039232BC1CACZ-1','ZEBBAF3BC29FDDB4E6501Z-1',

'Z3AF090856C0AA17C2D37Z-1','ZD66D8E379074BBC7099DZ-1','ZAD994290A1EB7059436CZ-1',

'Z24B7645810B458876CC9Z-1','Z5FEFC8C8E20639E6085CZ-1','ZFE681C5586B02F65C655Z-1',

'ZC6FAD4EDE55A6FF5BF82Z-1','Z438DC9F97D4FFEBF1BFFZ-1','Z2908E1774A24BFC597F8Z-1',

'ZBF92153759AE09FFE5FDZ-1','Z6F7937F3835EFBCAF5BEZ-1','ZD2198350DCBB144AEE1CZ-1',

'Z761ED2FC15A72F9D2285Z-1','ZD40F5EBAAD1869822DE8Z-1','Z7A6056149E2650D5A499Z-1',

'ZA9E905C93F2CD7735DD5Z-1','Z2752C87A1FB276102FA2Z-1','Z0243C9A53B31BA60665EZ-1',

'Z8D1C3044DA02EE1B1504Z-1','ZE3FC97EAF83207323499Z-1','Z80B4A702166E486F7244Z-1',

'Z75C6462D07D5B46E0DD2Z-1','Z7339D47BC259B160C535Z-1','Z51D8B79D3639A4F2A954Z-1',

'ZB434379F40CCA7D6C128Z-1','ZC110A4A8D5344F0982FAZ-1','Z20E34F716D800FD1FDE5Z-1',

'Z583AAE1E3EC9099B4B8EZ-1','Z5D0883A3BD2E4C658F12Z-1','Z782823BDF3B01C2DBEACZ-1',

'ZCC844D9C96729D6F905CZ-1','Z7B50EEDD4EFF9DB9E14CZ-1','Z69EC54C9B6830134C1DCZ-1',

'Z394AABB73EF98DA53702Z-1','ZEB1D66B02FD45C7F5235Z-1','ZBC0FA136F91D649B9FD6Z-1',

'Z9C159F269EA9F8384DC6Z-1','ZD9F490E66DB1F80AFCC1Z-1','Z6322C97105AD82323AA3Z-1',

'ZFBD594063B7289164CF5Z-1','Z961C8C56BE447410EB72Z-1','ZADD27C4506C3814DBC3EZ-1',

'Z26C1063E13C954FB1CDEZ-1','Z3CAC17258A9C7D4887C2Z-1','ZD8391FC74A2FC5EF033EZ-1',

'Z83FE44DC84B7E21BE121Z-1','ZFF3DCF1E224ECE397268Z-1','ZA86B12E6CBCDF73511D5Z-1',

'ZBB7797DD410A05E7F449Z-1','ZBBB3E724F92070B22A5AZ-1','ZFD0FA25041E79BCAE3E1Z-1',

'ZEF728A0BE09E273B418DZ-1','ZE531AF6D2A1ADBCBF2A7Z-1','ZE16B3BBA9BE4AAA16223Z-1',

'ZE94FDDD8E6BF88E7E25EZ-1','Z02932FC383C4D64C0531Z-1','Z3CBD70E2D219737FC600Z-1',

'Z0D54A1885494FD2FA756Z-1','ZAAED4556AB260734B90DZ-1','Z29D9CCA7D6121F24DF2DZ-1',

'ZAB807779B07579349A58Z-1','Z1F68261FC47CAC3C4988Z-1','Z88D4A83B064B9D777B8CZ-1',

'ZBA02034FAE84A5BCF4EDZ-1','ZF2A257FD0D4939D096DFZ-1','Z2ADE110495895FDC8952Z-1',

'Z6CC0F17FC7A014E14873Z-1','Z827EE80F7E6737FF1FB6Z-1','ZD3D939B457D73445F431Z-1',

'Z2F5326AE08471E123834Z-1','Z2998C777C770F2FFADD9Z-1','Z0C8912E4EDBDE2DEF2E4Z-1',

'Z906F2D48887E03E1EC6CZ-1','Z122EDA23764DF26A9882Z-1','ZA1C9E5092E061A0F714DZ-1',

'Z31B9D833F20568BBB8BBZ-1','Z2E15E0C2E491F6186756Z-1','Z50750AAEBCCBFCDF6A0CZ-1',

'ZCC79DB1122F11ACB520BZ-1','Z90B82F94E100FDF43BF1Z-1','ZA96C26651697804B8DA6Z-1',

'ZBEA211F51E5FC5D76FB5Z-1','Z9795A566BC1BF4DAE756Z-1','Z42C838C5664BD41B2599Z-1',

'ZD14379EF865941E963BFZ-1','ZFF16BC15B87424A622D0Z-1','ZFBD3786077B8CFE018C3Z-1',

'Z28DBC8B68211854F9F5DZ-1','Z848C3A0663A1FE22FC77Z-1','Z1F79D9EA2C90B4DCB984Z-1',

'Z49D1CC688D229525B3B8Z-1','Z836DCE0BC1AA21E7EC5CZ-1','Z548C9370373ED56555B2Z-1',

'ZBDFDD6B799D7487AE79FZ-1','ZB3A85AEBB693F271A525Z-1','Z599FF318F96E61F2A2A4Z-1',

'ZC2FF5C33D7B3725ACF6EZ-1','ZEC02FC92EB09067209E0Z-1','ZABCA9BC2DBDDAB3E89DCZ-1',

'ZC4D68917D25557E52ECCZ-1','Z17514F8D437CA0B9D870Z-1','ZB9EA17F3A70F41208CFEZ-1',

'ZD0D6360602ED8C4BF318Z-1','Z6F41B3BB4C3E33113BECZ-1','ZAEB476E1841334F2DD27Z-1',

'ZCFE4A08A5ED7D286F2CCZ-1','ZD6E64CF9425F4EE8403DZ-1','ZA2128D6F7E375C80BEDEZ-1',

'ZE542921BEDC3CEA7470EZ-1','Z5119FD7E7DECE6670AC8Z-1','Z491CAB3905C67E8EF4E9Z-1',

'Z020B7055C55464C03933Z-1','ZA482BA3EC0E9406ABCCDZ-1','Z8CC831ACDCB82BDE8749Z-1',

'ZEB1C9A42148B8D6284B4Z-1','Z9619AAFC5B57740355D1Z-1','Z9BAA09751DA3FC054F36Z-1',

'Z05AE3CA274AAA735585EZ-1','ZA4E0ACB7A951F7809C3CZ-1','Z92F971F0103C15513D9AZ-1',

'ZCB8179FF0B5858AE5349Z-1','ZBE3CE67F2194E5C15CDCZ-1','ZEFB5204E09F0FA3826A0Z-1',

'ZCE3239884FBA49EABE55Z-1','Z79515F74AEE2E05F4BA0Z-1','ZDA2B54DE409ECCD90501Z-1',

'Z82AB8FB6F81C2C0B947CZ-1','ZAF18384B777E5B559449Z-1','Z9BC60FBCBC579C9196B2Z-1',

'Z63B9364C8C0432162830Z-1','Z38AB4AFF569D5D3F92E8Z-1','ZDD0752A9E6A0CFBA7CB0Z-1',

'ZEDD6855A2A31EDE9A905Z-1','Z526A4BD7449FF65AAD69Z-1','ZB8B5DDAFE020A482EB18Z-1',

'Z81430141E96169903B57Z-1','Z0FFC2AC57BD5D3229B59Z-1','Z9FE2DF895B2D9EC58616Z-1',

'ZDB7442E2720FEDA7467FZ-1','Z85D15006D7CFEFB06646Z-1','Z8C9257D07648347AA3C9Z-1',

'ZD7A567D470D2C3018214Z-1','Z9674EAB005A7DB637F32Z-1','ZBC31CB49AEEECC1815F9Z-1',

'Z5C0714DAF6F5E0B294E9Z-1','ZB29D84A7371394199C0EZ-1','ZDACEC85D401F5CB0CD9AZ-1',

'Z17EED84357FB5111CBDBZ-1','Z6DE62DADCAD08F3F56C1Z-1','ZEC6629475BB9AB9EB4C7Z-1',

'Z247B07CE27213A4AA0A5Z-1','Z6EFE41E6BF3532783A85Z-1','Z6297EC0B698A25BB17F3Z-1',

'Z75730E0F0E5BFCF2FFECZ-1','ZBC979EABF4C8C011861AZ-1','Z58E9D67E1BDE14D02E01Z-1',

'Z5E6C719B9712863F87D4Z-1','ZAE5F7BD2E62D05C5CA1EZ-1','Z9D615F88ED8443E54E1AZ-1',

'ZE37E628CC17FDCF29122Z-1','Z9B2AEF365383186BC0B3Z-1','ZB3B4E1EB032CDFC1047CZ-1',

'ZC257374C53F1D7613031Z-1','Z8137477DAD2F18B99695Z-1','Z29FDC9FE318558CE94F4Z-1',

'ZB6A4F7CBDB43FF595714Z-1','Z524F68E8C1381739DFD8Z-1','Z5CDC39896AB38025C31EZ-1',

'ZABF30800324F4D29FDDCZ-1','Z990ED19303C5F828F6CFZ-1','Z9F2FABF0F2B3F9465182Z-1',

'Z3C85173387FA67ABBDFBZ-1','Z86457C794211AF4355A6Z-1','ZAE7F612710ED97DB4754Z-1',

'Z6D56CD0FE45876A4D5CEZ-1','Z0156CB1269D808EF4C23Z-1','ZAD4339C2C6C5B393658AZ-1',

'Z223D420C2D5CC499620BZ-1','ZAC26B8BCD3BE5968328FZ-1','Z482D1AB5EBA17EC9D7D9Z-1',

'ZF76498DF4E628EB6A3AAZ-1','Z2159D049A442310D80C8Z-1','ZE34620A7C51ABE6A0690Z-1',

'Z4A9AAA7951914712FC76Z-1','ZB93F34C79901B9F40F68Z-1','ZFD88C8CACBA0EBE8A6BBZ-1',

'ZAC3A656FF13B449BA002Z-1','Z78F8109C8C959DE4AB16Z-1','Z1C941FB22F5106B99295Z-1',

'Z350CF77EE5BCA729FD39Z-1','Z81E1FECC3001406D6488Z-1','Z43DA18A0D8C641DE8882Z-1',

'Z420BD54F147E553E14E1Z-1','Z3C0BB415FB587A848B91Z-1','Z9CE653B504BB78C8C04EZ-1',

'ZAC636EBB9D49E8669592Z-1','ZB4525A5A9C09D42519F4Z-1','Z4876422384CFF6BE146BZ-1',

'ZA2DD46C0103195FA4374Z-1','Z3B6C0C6541F54B93F966Z-1','Z8B997BB8943EB388B583Z-1',

'ZC6093570EB319E70C953Z-1','Z7061EBD116B90208F677Z-1','Z54CFE43B802BCD2AA4CBZ-1',

'ZA6B01376E813A457C18CZ-1','ZC15FB81271EC0B657483Z-1','ZC3AED3DD2490BF0EF60FZ-1',

'Z78626810767DCA2C51E0Z-1','ZADB8B6CE5CC2EE87BE23Z-1','Z09820ABE31797AC9F249Z-1',

'ZF437B46858154307F124Z-1','Z7E2B400D194CF82085A4Z-1','ZEA8F234CFC1EFFEC7B84Z-1',

'Z59EC067F6AD3BB3F1D86Z-1','Z50D025E7F5BC39C447E6Z-1','ZCA224ACD45CA6A2CCA94Z-1',

'Z313A55BAB6F259ECB775Z-1','Z341C5E2F2AF526800481Z-1','ZAFDFB3D6E9AC593D4D8DZ-1',

'ZEBBC44A1147CFC58E408Z-1','ZEDFD918C90BD58DAF525Z-1','ZE53FCEB22E3ACA2919AAZ-1',

'ZE39E40CE2D3451046FFCZ-1','Z48C58176B4BF5DB958B3Z-1','ZC5BA05FD1E3466857127Z-1',

'Z89F7EA3CA30E4C9AD23FZ-1','Z6470E36CE799E0D6BA98Z-1','Z3C4787E69EC873F73180Z-1',

'ZAA5CBF002E3EF80B1519Z-1','Z6879D1FE2EC4FFE56B4BZ-1','Z1F0BE45931A398399A3EZ-1',

'Z502D5863B452BBC4D17BZ-1','Z0353C8CED0C1D3B833EFZ-1','Z6345F8048DC7F3379F58Z-1',

'Z6174222AA908DBBEB684Z-1','Z7A2997D200EDFCE198B6Z-1','Z3DEE26B58B6D0FC1739FZ-1',

'ZB187C98CAC2E32C4E4EFZ-1','Z54D95600E1C086FBA6CAZ-1','Z9778E2D5EE755E16E2FBZ-1',

'Z324602388C44300407DFZ-1','Z932209052FDD71474FB9Z-1','Z77096D0127B38B666908Z-1',

'Z727E8A00D8CCA115FD20Z-1','Z0D759C09A44311ACF9A2Z-1','ZD1E2D3684EE7A1CF0B88Z-1',

'Z5D83F8790297E1917A62Z-1','Z7C5FDF73BBCB418875AFZ-1','Z2D965F328D6B463E7438Z-1',

'Z4D4FD79761874F5F8024Z-1','Z142D1F142A1AA7749B42Z-1','Z3B9943380C1B1EB89292Z-1',

'Z617516EF91E73F58A2BAZ-1','ZF7450164D4DB1B99C709Z-1','Z99687DEB4AFB175D3CB1Z-1',

'Z735EF8E1E31BD70D487FZ-1','ZEC4A4F284FA18BF4A894Z-1','Z7202F3E516C5FB132F8AZ-1',

'Z605FA2228DA84F304E63Z-1','ZFB42C683F513E64C2869Z-1','Z912E9F43664FC5B45983Z-1',

'ZAC35AB997C0B6E6B72DEZ-1','ZFEB80F096CA7DB11FA4FZ-1','Z5E4DEC7D4461BCAB77DDZ-1',

'ZD1AB44C857E688F984EFZ-1','ZBE15844CC4EE1858EE3FZ-1','Z5172747D966943FD5398Z-1',

'Z1ECDE11BDFC6E4356141Z-1','ZB2517DD5D9649EB4ABC3Z-1','Z75F8BA619A2D3AF5B583Z-1',

'Z8580289CB54678E80A82Z-1','Z07472F0161DE1F93DFE3Z-1','Z209EADFE14BCAF8A2587Z-1',

'ZC8026006F52A0F5097FEZ-1','ZCBB5566B7DB08CFDEF80Z-1','Z74918247D20179C00FB3Z-1',

'Z02E03D06366B51C46438Z-1','Z414D7D18049502BCB398Z-1','Z20520042A30D1ED94909Z-1',

'ZF42613F9D0171331B961Z-1','Z4B23C2FC367794ACF722Z-1','Z00355FACD4C48EC3825AZ-1',

'ZF27720D2D8F3AE188B08Z-1','ZBFF064C122B75F100FE6Z-1','Z4B0F9B95B22CEFED0443Z-1',

'ZA92126E3B45840DECFA1Z-1','Z3AC6AFABCF959AC91E7EZ-1','Z9E183B3425CEAB7B806Z-1',

'Z6BC14C9AA59C79E4C09EZ-1','Z3EDEF2412AD15CC9D0C5Z-1','ZE647CFBE0A34EEEBF93CZ-1',

'ZD1A3D5542C31118956B8Z-1','Z3124F3F26C1D31C22757Z-1','Z23539CD57210B9E53941Z-1'
  )
 GROUP BY 1,2,3,4,5,6,7,8

```
),


-- ============================================================
-- CTE 2: non_egg_base
-- Aggregates non-egg items into pipeline stage counts
-- Scoped to Ultrafresh warehouses: DXBID07, AUHID07
-- ============================================================
non_egg_base AS (
  SELECT
    -- Bucket into today or tomorrow based on cutoff (13h offset = midnight Dubai)
    CASE
      WHEN i.created_at < (CURRENT_TIMESTAMP() + INTERVAL 13 HOUR)
      THEN DATE(CURRENT_TIMESTAMP() + INTERVAL 13 HOUR)
      ELSE DATE(CURRENT_TIMESTAMP() + INTERVAL 13 HOUR + INTERVAL 1 DAY)
    END AS created_at,

    -- Slot logic: non DXB/AUH/SHJ destinations always Slot_2
    CASE
      WHEN (dst_w.partner_warehouse_code NOT LIKE 'DXB%'
        AND dst_w.partner_warehouse_code NOT LIKE 'AUH%'
        AND dst_w.partner_warehouse_code NOT LIKE 'SHJ%') THEN 'Slot_2'
      WHEN i.created_at >= (CURRENT_TIMESTAMP() - INTERVAL 1 DAY + INTERVAL 13
HOUR)
          AND i.created_at <  (CURRENT_TIMESTAMP() - INTERVAL 1 DAY + INTERVAL 21
HOUR)
      THEN 'Slot_1'
      WHEN i.created_at >= (CURRENT_TIMESTAMP() - INTERVAL 1 DAY + INTERVAL 21
HOUR)
          AND i.created_at <  (CURRENT_TIMESTAMP() + INTERVAL 13 HOUR)
      THEN 'Slot_2'
      ELSE 'Slot_1'
    END AS slot,

    -- crossdoc job with no transfer_nr = CROSS_DOC; everything else = PICKING
    CASE
      WHEN sj.job_type = 'crossdoc' AND i.transfer_nr IS NULL THEN 'CROSS_DOC'
      ELSE 'PICKING'
    END AS type,

    src_w.partner_warehouse_code  AS source_warehouse,
    dst_w.partner_warehouse_code  AS destination_darkstore,
    dst_w.name                    AS destination_name,
```

```
   CASE
     WHEN src_w.partner_warehouse_code IN ('AUHFKZ01','AUHFKZ02','AUHFKZ03') THEN
'Frozen_TP'
     WHEN src_w.partner_warehouse_code IN ('DXBID01','DXBID02')            THEN
'Ambient_TP'
     WHEN src_w.partner_warehouse_code IN ('DXBID03','DXBID04')            THEN
'Chiller_TP'
     WHEN src_w.partner_warehouse_code IN ('DXBID07')                    THEN 'Ultrafresh_TP'
     WHEN src_w.partner_warehouse_code IN ('DXBFNV01')                     THEN 'FnV'
     WHEN src_w.partner_warehouse_code IN ('AUHID01','AUHID02')            THEN
'Ambient_AUH'
     WHEN src_w.partner_warehouse_code IN ('AUHID03','AUHID04')            THEN
'Chiller_AUH'
     WHEN src_w.partner_warehouse_code IN ('AUHID07')                    THEN
'Ultrafresh_AUH'
   END AS wh_category,

   COUNT(DISTINCT i.id_item)                                  AS total_qty,
   COUNT(DISTINCT IF(i.id_mp_status = 31, i.id_item, NULL))          AS cancelled_qty,

   -- Awaiting picker assignment (status 44, active mp_status 30)
   COUNT(DISTINCT CASE
     WHEN i.id_status = 44 AND i.id_mp_status = 30 THEN i.id_item
   END)                                                  AS pending_picking,

   -- Packed/sorting but not yet in a hub_transfer_line, or htl staged (1)
   COUNT(DISTINCT IF(
     i.id_status IN (36, 17)
     AND i.id_mp_status = 30
     AND (htl.id_status = 1 OR htl.id_hub_transfer_line IS NULL),
     i.id_item, NULL))                                   AS pending_staging,

   -- Packed, htl released from staging (57)
   COUNT(DISTINCT IF(
     i.id_status = 17 AND i.id_mp_status = 30 AND htl.id_status = 57,
     i.id_item, NULL))                                   AS pending_release_staging,

   -- htl dispatched (36) + hub_transfer en route (26)
   COUNT(DISTINCT IF(
     i.id_mp_status = 30 AND htl.id_status = 36 AND ht.id_status = 26,
     i.id_item, NULL))                                   AS pending_dispatch,

   -- htl dispatched (36) + hub_transfer at receiving dock (13)
   COUNT(DISTINCT IF(
```

```
    i.id_mp_status = 30 AND htl.id_status = 36 AND ht.id_status = 13,
    i.id_item, NULL))                                   AS pending_receiving,

  -- Active, not deleted, putaway not started
  COUNT(DISTINCT IF(
    i.id_mp_status != 31
    AND i.id_status != 32
    AND JSON_EXTRACT_SCALAR(i.misc, '$.putaway_status') IS NULL,
    i.id_item, NULL))                                   AS transit,

  -- Packed, htl received at darkstore (69)
  COUNT(DISTINCT IF(
    i.id_status = 17 AND i.id_mp_status = 30 AND htl.id_status = 69,
    i.id_item, NULL))                                   AS received_at_darkstore,

  COUNT(DISTINCT IF(
    i.id_mp_status = 30
    AND JSON_EXTRACT_SCALAR(i.misc, '$.putaway_status') = 'completed',
    i.id_item, NULL))                                   AS putaway_completed,

  COUNT(DISTINCT IF(
    i.id_mp_status = 30
    AND JSON_EXTRACT_SCALAR(i.misc, '$.putaway_status') = 'missing',
    i.id_item, NULL))                                   AS missing,

  -- Cancelled items that were recreated via NSV3
  COUNT(DISTINCT IF(
    i.id_mp_status = 31
    AND JSON_EXTRACT_SCALAR(i.misc, '$.unfulfillable_reason') LIKE '%recreate_nsv3%',
    i.id_item, NULL))                                   AS recreate_nsv3

FROM item_base_non_egg i
LEFT JOIN `noondwh.scgoms_goms.warehouse` src_w
  ON i.id_warehouse = src_w.id_warehouse
LEFT JOIN `noondwh.scgoms_goms.warehouse` dst_w
  ON JSON_EXTRACT_SCALAR(i.misc, '$.id_dst_warehouse') = CAST(dst_w.id_warehouse
AS STRING)
LEFT JOIN `noondwh.scgoms_goms.shipment_line` sl USING (id_item)
LEFT JOIN `noondwh.scgoms_goms.shipment` sh USING (id_shipment)
-- hub_transfer_line: scoped to forward_nim shipment lines only
LEFT JOIN `noondwh.scgoms_goms.hub_transfer_line` htl
  ON (htl.type, htl.line_type, htl.line_id) = ('forward_nim', 'shipment', sh.id_shipment)
LEFT JOIN `noondwh.scgoms_goms.hub_transfer_group` htg
  ON htg.id_hub_transfer_group = htl.id_hub_transfer_group
```

```
  LEFT JOIN `noondwh.scgoms_goms.hub_transfer` ht USING (id_hub_transfer)
  LEFT JOIN `noondwh.scgoms_goms.input_container_line` icl ON icl.id_item = i.id_item
  LEFT JOIN `noondwh.scgoms_goms.input_container` ic ON icl.id_input_container =
ic.id_input_container
  LEFT JOIN `noondwh.scgoms_goms.sortation_job` sj ON ic.id_sortation_job =
sj.id_sortation_job

  WHERE i.id_mp = 9                                          -- Noon Instant only
    AND src_w.id_country = 1                                   -- UAE only
    AND src_w.partner_warehouse_code IN ('AUHID07', 'DXBID07')       -- Ultrafresh only

  GROUP BY 1,2,3,4,5,6,7
),


-- ============================================================
-- CTE 3: item_base_egg
-- Raw item rows for egg SKUs — wider window (5h offset)
-- No catalog_sku exclusion applied
-- ============================================================
item_base_egg AS (
  SELECT
    created_at,
    id_mp_status,
    id_status,
    misc,
    id_warehouse,
    id_item,
    id_mp,
    transfer_nr
  FROM `noondwh.scgoms_goms.item`
  WHERE created_at >= (CURRENT_TIMESTAMP() - INTERVAL 1 DAY + INTERVAL 5 HOUR)
  GROUP BY 1,2,3,4,5,6,7,8
),


-- ============================================================
-- CTE 4: egg_base
-- Placeholder — currently mirrors non_egg_base output
-- Extend here when egg-specific pipeline logic is needed
-- ============================================================
egg_base AS (
  SELECT * FROM non_egg_base
)
```

```
-- ============================================================
-- FINAL OUTPUT
-- Union egg and non-egg pipelines into a single result set
-- ============================================================
SELECT * FROM egg_base
UNION ALL
SELECT * FROM non_egg_base;
```

```
/*
================================================================================
========
```

## KEZAD (AUHID01/02) INVENTORY & PICKING HEALTH DASHBOARD

```
================================================================================
========
  PURPOSE  : Comprehensive inventory and picking health analysis for Kezad
             warehouses. Covers:
               1. SKU-level inventory with DOC, DRR, location type
               2. Average SKUs per bin broken down by zone
               3. Location catalogue (deep vs forward)
               4. Picking performance — picked vs skipped, deep vs forward,
                  top vs BAU SKUs
               5. Daily cancellations by reason
               6. Forward-to-deep migration suggestions (excess DOC > 14 days)
               7. Batch issue detection (deep expiry older than forward)

  SOURCE   : noondwh.mxdcss_dcss, noondwh.scgoms_cwpicking,
             noondwh.scgoms_goms, noonbinimops.fulfillment,
             noonbiinstnt.minutes
  GRAIN    : Varies per output block (see individual SELECT comments)
  FILTERS  : UAE (country_code = 'ae') | AUHID01 / AUHID02

  AUTHOR   : —
  LAST     : (auto-refresh via CURRENT_DATE / CURRENT_TIMESTAMP)
```

**Structure — outputs are commented out blocks so you run only what you need without re-computing the shared base CTEs**

==========
*/

/*
──────────────────────────────────────────────────────────────────────────────────────────────────────────────

  VARIABLE DEFINITIONS

  WAREHOUSE_FILTER       : AUHID01, AUHID02         (Kezad warehouses)
  COUNTRY_CODE           : 'ae'                     (UAE)
  UAE_COUNTRY_ID         : 1                        (id_country for UAE)
  SA_COUNTRY_ID          : 2 / 5                    (Saudi — used in date shift)
  TOP_SKU_FLAG           : is_top = 1               (top 2500 SKUs)
  DRR_WINDOW             : last 7 days (CURRENT_DATE-7 to CURRENT_DATE-1)
  PICKING_WINDOW         : last 8 business days     (pj.created_at - 6h30m)
  TRANSFER_JOIN_WINDOW   : last 10 days             (transfer_request.created_at)
  BUSINESS_DAY_CUTOFF    : 06:00 local (6h offset for AE, 6h for SA)
  AE_CUTOFF_OFFSET       : INTERVAL 6 HOUR + 30 MIN  (Dubai business day)
  SA_CUTOFF_OFFSET       : INTERVAL 6 HOUR          (Riyadh business day)

  DOC_BUCKETS (Days of Cover):
    Zero    : DOC = 0
    00-03   : DOC ≤ 3
    04-07   : DOC ≤ 7
    07-14   : DOC ≤ 14
    14++    : DOC > 14   ← candidates for forward→deep migration

  SKU TYPE:
    Slow_Moving : DRR < 1
    BAU         : DRR ≥ 1

  LOCATION TYPE:
    Deep_location    : misc.is_deep = '1'
    Forward_location : misc.is_deep != '1'

  SKIP REASONS COUNTED AS REAL MISS: 'missing', 'damaged', 'expired'
  MP_STATUS_CANCELLED  : 31   (item cancelled in goms)
  PICKING_JOB_CLOSED   : 'closed'
──────────────────────────────────────────────────────────────────────────────────────────────────────────────
*/

```
-- ============================================================
-- SHARED CTE 1: top_2500
-- Top SKUs for UAE (is_top = 1 flag from instock snapshot)
-- ============================================================
WITH top_2500 AS (
  SELECT sku
  FROM noonbiinstnt.minutes.instock_views_current_snapshot
  WHERE country_code = 'ae'
    AND is_top = 1
  GROUP BY ALL
),

-- ============================================================
-- SHARED CTE 2: deep_loc
-- All deep locations (misc.is_deep = '1') in AUHID01/02
-- Used across picking, migration, and batch issue analyses
-- ============================================================
deep_loc AS (
  SELECT
    w.partner_warehouse_code  AS wh_code,
    z.zone_code,
    l.location_code,
    JSON_EXTRACT_SCALAR(l.misc, '$.is_deep') AS is_deep
  FROM noondwh.mxdcss_dcss.country c
  LEFT JOIN noondwh.mxdcss_dcss.warehouse w USING (id_country)
  LEFT JOIN noondwh.mxdcss_dcss.tenant t USING (id_warehouse)
  LEFT JOIN noondwh.mxdcss_dcss.zone z USING (id_tenant)
  LEFT JOIN noondwh.mxdcss_dcss.location l USING (id_zone)
  LEFT JOIN noondwh.mxdcss_dcss.location_item li USING (id_location)
  JOIN noondwh.mxdcss_dcss.item i USING (id_item)
  WHERE w.partner_warehouse_code IN ('AUHID01', 'AUHID02')
    AND z.is_saleable = 1
    AND JSON_EXTRACT_SCALAR(l.misc, '$.is_deep') = '1'
  GROUP BY ALL
),

-- ============================================================
-- SHARED CTE 3: deep_zones
-- Distinct deep zone codes per warehouse (derived from deep_loc)
-- ============================================================
deep_zones AS (
  SELECT wh_code, zone_code
```

```
  FROM deep_loc
  GROUP BY ALL
),

-- ============================================================
-- SHARED CTE 4: inv_base
-- Saleable inventory snapshot for AUHID01, with top SKU flag
-- Source: fulfillment materialized stock table
-- ============================================================
inv_base AS (
  SELECT
    warehouse                                           AS partner_warehouse_code,
    zone_code,
    location_code,
    zsku,
    category,
    product_title,
    CASE WHEN zsku IN (SELECT sku FROM top_2500)
         THEN 'Top_2500' ELSE 'Non_Top_2500' END              AS sku_flag,
    SUM(stock_gross)                                    AS gross_qty,
    SUM(stock_gross) - SUM(stock_reserved)                   AS net_qty_at_wh
  FROM noonbinimops.fulfillment.minutes_dcss_stock_AE
  WHERE is_saleable = '1'
    AND warehouse = 'AUHID01'
  GROUP BY ALL
),

-- ============================================================
-- SHARED CTE 5: drr
-- Daily run rate per SKU — last 7 days delivered orders
-- dr = total_sales / 7  (used for DOC calculation)
-- Source: goms.item (more accurate than sales_order for this use case)
-- ============================================================
drr AS (
  SELECT
    catalog_sku                                         AS sku,
    COUNT(DISTINCT i.id_item)                               AS total_sales,
    COUNT(DISTINCT i.id_item)
      / COUNT(DISTINCT DATE(i.created_at, 'Asia/Dubai'))        AS dr
  FROM noondwh.scgoms_goms.item i
  LEFT JOIN noondwh.scgoms_goms.warehouse src_w
    ON i.id_warehouse = src_w.id_warehouse
  WHERE DATE(i.created_at) BETWEEN CURRENT_DATE() - 7 AND CURRENT_DATE() - 1
    AND i.id_mp = 9                                     -- Noon Instant only
```

```
    AND src_w.id_country = 1
    AND src_w.partner_warehouse_code = 'AUHID01'
  GROUP BY ALL
),

-- ============================================================
-- SHARED CTE 6: inv_with_doc
-- Inventory joined with DRR and location type — central enriched base
-- for all inventory-facing analyses (DOC, migration, zone breakdown)
-- ============================================================
inv_with_doc AS (
  SELECT
    a.partner_warehouse_code,
    a.zone_code,
    a.location_code,
    a.zsku,
    a.sku_flag,
    a.category,
    a.product_title,
    CASE WHEN d.dr < 1.0 THEN 'Slow_Moving' ELSE 'BAU' END            AS sku_type,
    d.dr                                                              AS drr,
    CASE WHEN b.is_deep = '1' THEN 'Deep_location'
         ELSE 'Forward_location' END                                  AS location_type,
    SUM(a.gross_qty)                                                  AS gross_qty,
    SUM(a.gross_qty) / GREATEST(d.dr, 0.1)                            AS doc_kept,
    COUNT(DISTINCT a.location_code)                                   AS locations_kept,
    COUNT(DISTINCT a.zone_code)                                       AS zones_kept,
    CASE
      WHEN SUM(a.gross_qty) / GREATEST(d.dr, 0.1) = 0   THEN 'Zero'
      WHEN SUM(a.gross_qty) / GREATEST(d.dr, 0.1) <= 3  THEN '00-03'
      WHEN SUM(a.gross_qty) / GREATEST(d.dr, 0.1) <= 7  THEN '04-07'
      WHEN SUM(a.gross_qty) / GREATEST(d.dr, 0.1) <= 14 THEN '07-14'
      ELSE '14++'
    END                                                               AS doc_bucket
  FROM inv_base a
  LEFT JOIN deep_loc b
    ON a.partner_warehouse_code = b.wh_code
   AND a.zone_code = b.zone_code
   AND a.location_code = b.location_code
  LEFT JOIN drr d ON d.sku = a.zsku
  GROUP BY ALL
),

-- ============================================================
```

```
-- SHARED CTE 7: picking_base
-- All closed picking jobs in last 8 business days for AUHID01/02
-- Enriched with skip reason, location type, top SKU flag
-- Used by: picking performance, JLM, daily cancellations
-- ============================================================
picking_base AS (
  SELECT
    src_w.partner_warehouse_code                                    AS wh_code,
    wz.code                                                         AS zone_code,
    pll.sku,
    pll.location_code,
    pll.id_picklist_line,
    CASE WHEN tt.sku IS NOT NULL THEN 'Top' ELSE 'BAU' END          AS sku_type,
    CASE
      WHEN src_w.id_country = 1
        THEN DATE(DATETIME(pj.created_at, 'Asia/Dubai')
             - INTERVAL 6 HOUR - INTERVAL 30 MINUTE)
      WHEN src_w.id_country = 2
        THEN DATE(DATETIME(pj.created_at, 'Asia/Riyadh')
             - INTERVAL 6 HOUR)
    END                                                             AS date,
    plc.id_picking_container_line,
    r.code                                                          AS skip_reason,
    dl.location_code                                                AS deep_location_code,
    i.id_item,
    i.id_mp_status
  FROM noondwh.scgoms_cwpicking.picking_job pj
  LEFT JOIN noondwh.scgoms_cwpicking.status pjst
    ON pj.id_status = pjst.id_status
  LEFT JOIN noondwh.scgoms_cwpicking.picklist_line pll
    USING (id_picking_job)
  LEFT JOIN noondwh.scgoms_cwpicking.status pllst
    ON pll.id_status = pllst.id_status
  LEFT JOIN noondwh.scgoms_cwpicking.warehouse src_w
    ON pll.id_warehouse = src_w.id_warehouse
  LEFT JOIN noondwh.scgoms_cwpicking.reason r
    ON r.id_reason = pll.id_skip_reason
  LEFT JOIN noondwh.scgoms_cwpicking.warehouse_zone wz
    ON pll.id_warehouse_zone = wz.id_warehouse_zone
  LEFT JOIN noondwh.scgoms_cwpicking.picking_container_line plc
    USING (id_picklist_line)
  LEFT JOIN deep_loc dl
    ON dl.location_code = pll.location_code
   AND dl.wh_code = src_w.partner_warehouse_code
```

```
  LEFT JOIN top_2500 tt ON tt.sku = pll.sku
  -- Transfer join scoped to last 10 days to limit scan cost
  LEFT JOIN noondwh.scgoms_cwpicking.transfer_request_line t
    ON t.id_transfer_request_line = pll.id_transfer_request_line
   AND DATE(t.created_at, 'Asia/Dubai') >= CURRENT_DATE() - 10
  LEFT JOIN noondwh.scgoms_cwpicking.transfer_request tr
    ON tr.id_transfer_request = t.id_transfer_request
   AND DATE(tr.created_at, 'Asia/Dubai') >= CURRENT_DATE() - 10
  LEFT JOIN noondwh.scgoms_goms.item i
    ON tr.transfer_nr = i.transfer_nr
   AND i.catalog_sku = t.sku
   AND DATE(i.created_at, 'Asia/Dubai') >= CURRENT_DATE() - 10
  WHERE pjst.code = 'closed'
    AND src_w.partner_warehouse_code IN ('AUHID01', 'AUHID02')
    AND DATE(
          DATETIME(pj.created_at, 'Asia/Dubai')
          - INTERVAL 6 HOUR - INTERVAL 30 MINUTE
        ) >= CURRENT_DATE() - 8
),

-- ============================================================
-- SHARED CTE 8: picking_agg
-- Aggregates picking_base to (wh, zone, date) grain
-- Computes picked/skipped counts split by top/deep dimensions
-- ============================================================
picking_agg AS (
  SELECT
    wh_code,
    zone_code,
    date,

    -- Overall
    COUNT(DISTINCT CASE
      WHEN skip_reason IN ('missing','damaged','expired')
       AND id_picking_container_line IS NULL THEN id_picklist_line END)  AS skipped_items,
    COUNT(DISTINCT CASE
      WHEN id_picking_container_line IS NOT NULL THEN id_picklist_line END) AS
picked_items,

    -- Unique SKU×location combos
    COUNT(DISTINCT CASE
      WHEN id_picking_container_line IS NOT NULL
        THEN CONCAT(sku, location_code) END)                    AS picked_sku_loc,
    COUNT(DISTINCT CASE
```

```
      WHEN skip_reason IN ('missing','damaged','expired')
       AND id_picking_container_line IS NULL
        THEN CONCAT(sku, location_code) END)                    AS skipped_sku_loc,

    -- Top SKU split
    COUNT(DISTINCT CASE
      WHEN sku_type = 'Top'
       AND skip_reason IN ('missing','damaged','expired')
       AND id_picking_container_line IS NULL THEN id_picklist_line END)  AS
skipped_items_top,
    COUNT(DISTINCT CASE
      WHEN sku_type = 'Top'
       AND id_picking_container_line IS NOT NULL THEN id_picklist_line END) AS
picked_items_top,

    -- Deep location split
    COUNT(DISTINCT CASE
      WHEN deep_location_code IS NOT NULL
       AND skip_reason IN ('missing','damaged','expired')
       AND id_picking_container_line IS NULL THEN id_picklist_line END)  AS
skipped_items_deep,
    COUNT(DISTINCT CASE
      WHEN deep_location_code IS NOT NULL
       AND id_picking_container_line IS NOT NULL THEN id_picklist_line END) AS
picked_items_deep,

    -- Top × Deep
    COUNT(DISTINCT CASE
      WHEN sku_type = 'Top' AND deep_location_code IS NOT NULL
       AND skip_reason IN ('missing','damaged','expired')
       AND id_picking_container_line IS NULL THEN id_picklist_line END)  AS
skipped_items_deep_top,
    COUNT(DISTINCT CASE
      WHEN sku_type = 'Top' AND deep_location_code IS NOT NULL
       AND id_picking_container_line IS NOT NULL THEN id_picklist_line END) AS
picked_items_deep_top,

    -- Top SKU×location combos
    COUNT(DISTINCT CASE
      WHEN sku_type = 'Top'
       AND id_picking_container_line IS NOT NULL
        THEN CONCAT(sku, location_code) END)                    AS picked_sku_loc_top,
    COUNT(DISTINCT CASE
      WHEN sku_type = 'Top'
```

```
      AND skip_reason IN ('missing','damaged','expired')
      AND id_picking_container_line IS NULL
       THEN CONCAT(sku, location_code) END)                    AS skipped_sku_loc_top,

   -- Cancellations (id_mp_status = 31 in goms)
   COUNT(DISTINCT CASE
     WHEN skip_reason IN ('missing','damaged','expired')
      AND id_mp_status = 31 THEN id_item END)                  AS
cancelled_due_to_missing,
   COUNT(DISTINCT CASE
     WHEN sku_type = 'Top'
      AND skip_reason IN ('missing','damaged','expired')
      AND id_mp_status = 31 THEN id_item END)                  AS
cancelled_due_to_missing_top,
   COUNT(DISTINCT CASE
     WHEN deep_location_code IS NOT NULL
      AND skip_reason IN ('missing','damaged','expired')
      AND id_mp_status = 31 THEN id_item END)                  AS
cancelled_due_to_missing_deep

  FROM picking_base
  GROUP BY ALL
),

-- ============================================================
-- SHARED CTE 9: sku_qty_raw
-- Location-level expiry and quantity detail for AUHID01
-- Used by batch issue detection (deep expiry < forward expiry)
-- ============================================================
sku_qty_raw AS (
  SELECT
    w.partner_warehouse_code                                  AS wh_code,
    li.barcode,
    i.id_partner,
    l.id_location,
    l.location_code,
    JSON_EXTRACT_SCALAR(l.misc, '$.is_deep')                  AS is_deep,
    li.qty,
    i.expiry_date,
    CASE WHEN JSON_EXTRACT_SCALAR(l.misc, '$.is_deep') = '1'
        THEN li.qty ELSE 0 END                                AS deep_qty,
    CASE WHEN JSON_EXTRACT_SCALAR(l.misc, '$.is_deep') != '1'
        THEN i.expiry_date END                                AS forward_expiry_date,
    CASE WHEN JSON_EXTRACT_SCALAR(l.misc, '$.is_deep') = '1'
```

```
        THEN i.expiry_date END                                  AS deep_expiry_date
  FROM noondwh.mxdcss_dcss.country c
  LEFT JOIN noondwh.mxdcss_dcss.warehouse w USING (id_country)
  LEFT JOIN noondwh.mxdcss_dcss.tenant t USING (id_warehouse)
  LEFT JOIN noondwh.mxdcss_dcss.zone z USING (id_tenant)
  LEFT JOIN noondwh.mxdcss_dcss.location l USING (id_zone)
  LEFT JOIN noondwh.mxdcss_dcss.location_item li USING (id_location)
  JOIN noondwh.mxdcss_dcss.item i USING (id_item)
  WHERE w.partner_warehouse_code = 'AUHID01'
    AND z.is_saleable = 1
    AND i.expiry_date IS NOT NULL
),

-- Earliest forward expiry per barcode+partner — batch issue anchor
forward_min AS (
  SELECT wh_code, barcode, id_partner,
    MIN(forward_expiry_date) AS min_forward_expiry
  FROM sku_qty_raw
  WHERE forward_expiry_date IS NOT NULL
  GROUP BY ALL
),

-- Total deep qty with older expiry than forward anchor (true batch issue qty)
batch_issue_calc AS (
  SELECT
    s.wh_code, s.barcode, s.id_partner,
    SUM(
      CASE WHEN s.deep_expiry_date < f.min_forward_expiry
           THEN s.deep_qty ELSE 0 END
    ) AS true_batch_issue_qty
  FROM sku_qty_raw s
  LEFT JOIN forward_min f
    ON s.wh_code = f.wh_code AND s.barcode = f.barcode AND s.id_partner = f.id_partner
  GROUP BY ALL
),

-- SKU → barcode mapping from fulfillment materialized table
sku_map AS (
  SELECT warehouse AS partner_warehouse_code, pbarcode_canonical, id_partner, zsku
  FROM noonbinimuae.minutes.minutes_dcss_stock_AE
  WHERE warehouse IN ('AUHID01', 'AUHID02')
  GROUP BY ALL
),
```

```
-- ============================================================
-- MIGRATION: excess forward inventory candidates (DOC > 14 days)
-- SKUs with forward qty exceeding 14 days of cover
-- ============================================================
migration_candidates AS (
  SELECT
    *,
    gross_qty - GREATEST(COALESCE(drr, 0.1) * 14, 2) AS qty_to_be_moved
  FROM inv_with_doc
  WHERE doc_bucket = '14++'
    AND location_type = 'Forward_location'
    AND gross_qty - GREATEST(COALESCE(drr, 0.1) * 14, 2) > 0
)


/* ============================================================
   OUTPUT 1: SKU inventory detail — DOC, DRR, location type
   Grain: one row per (warehouse, sku, zone, location)
   ============================================================ */
SELECT
  partner_warehouse_code,
  zsku,
  zone_code,
  location_code,                    -- use for forward→deep planning
  sku_flag,
  sku_type,
  drr,
  category,
  product_title,
  location_type,
  gross_qty,
  doc_kept,
  locations_kept,
  zones_kept
FROM inv_with_doc

--
————————————————————————————————————————————————————————————
——
-- Uncomment the block you need. Only one SELECT runs at a time.
--
————————————————————————————————————————————————————————————
——
```

```
/* ============================================================
   OUTPUT 2: Average SKUs per bin broken down by zone
   Grain: one row per (warehouse, zone)
   ============================================================
SELECT
  partner_warehouse_code,
  zone_code,
  COUNT(DISTINCT location_code)                    AS locations,
  COUNT(DISTINCT zsku)                             AS skus,
  COUNT(DISTINCT zsku) / COUNT(DISTINCT location_code)   AS sku_per_location
FROM inv_with_doc
GROUP BY ALL;
*/

/* ============================================================
   OUTPUT 3: Location catalogue — deep vs forward flag
   Grain: one row per (warehouse, zone, location)
   ============================================================
SELECT DISTINCT
  wh_code,
  zone_code,
  location_code,
  CASE WHEN is_deep = '1' THEN 'Deep_location' ELSE 'Forward_location' END AS
location_type
FROM deep_loc
ORDER BY 1, 2, 3;
*/

/* ============================================================
   OUTPUT 4: Picking performance — picked/skipped by zone/day
   Grain: one row per (warehouse, zone, date)
   ============================================================
SELECT * FROM picking_agg;
*/

/* ============================================================
   OUTPUT 5: Daily cancellations (only SKUs with cancellations)
   Adds SKU and sku_type dimension to picking_agg
   Grain: one row per (warehouse, zone, sku, date)
   ============================================================
SELECT
  pb.wh_code,
  pb.zone_code,
  pb.sku,
```

```
  pb.sku_type,
  pb.date,
  COUNT(DISTINCT CASE
    WHEN pb.skip_reason IN ('missing','damaged','expired')
     AND pb.id_picking_container_line IS NULL
    THEN pb.id_picklist_line END)                    AS skipped_items,
  COUNT(DISTINCT CASE
    WHEN pb.id_picking_container_line IS NOT NULL
    THEN pb.id_picklist_line END)                    AS picked_items,
  COUNT(DISTINCT CASE
    WHEN pb.skip_reason IN ('missing','damaged','expired')
     AND pb.id_mp_status = 31
    THEN pb.id_item END)                             AS cancelled_due_to_missing
FROM picking_base pb
GROUP BY ALL
HAVING cancelled_due_to_missing > 0;
*/

/* ============================================================
   OUTPUT 6a: Forward→Deep migration — deep zones only
   Grain: one row per (warehouse, sku, location) to move
   ============================================================
SELECT
  a.*,
  SUM(stock_gross - stock_reserved)
    OVER (PARTITION BY a.warehouse, a.zsku
          ORDER BY stock_gross - stock_reserved DESC, a.location_code) AS cum_qty,
  b.qty_to_be_moved
FROM noonbinimops.fulfillment.minutes_dcss_stock_AE a
JOIN migration_candidates b
  ON a.warehouse = b.partner_warehouse_code AND a.zsku = b.zsku
LEFT JOIN deep_loc d
  ON a.warehouse = d.wh_code AND a.zone_code = d.zone_code AND a.location_code =
d.location_code
JOIN deep_zones dz ON dz.zone_code = a.zone_code AND dz.wh_code = a.warehouse
WHERE d.location_code IS NULL
  AND is_saleable = '1'
  AND a.warehouse = 'AUHID01'
QUALIFY cum_qty <= qty_to_be_moved + 20
ORDER BY cum_qty DESC;
*/

/* ============================================================
   OUTPUT 6b: Forward→Deep migration — all zones
```

```
   Same as 6a but without the deep_zones JOIN constraint
   ============================================================
SELECT
  a.*,
  SUM(stock_gross - stock_reserved)
    OVER (PARTITION BY a.warehouse, a.zsku
          ORDER BY stock_gross - stock_reserved DESC, a.location_code) AS cum_qty,
  b.qty_to_be_moved
FROM noonbinimops.fulfillment.minutes_dcss_stock_AE a
JOIN migration_candidates b
  ON a.warehouse = b.partner_warehouse_code AND a.zsku = b.zsku
LEFT JOIN deep_loc d
  ON a.warehouse = d.wh_code AND a.zone_code = d.zone_code AND a.location_code =
d.location_code
WHERE d.location_code IS NULL
  AND is_saleable = '1'
  AND a.warehouse = 'AUHID01'
QUALIFY cum_qty <= qty_to_be_moved + 20
ORDER BY cum_qty DESC;
*/

/* ============================================================
   OUTPUT 7: Batch issue — deep locations with older expiry
             than the earliest forward stock for same barcode
   Grain: one row per (sku, barcode, location)
   ============================================================
SELECT
  sm.partner_warehouse_code,
  sm.zsku,
  CASE WHEN t2k.sku IS NOT NULL THEN 'Top_2500' ELSE 'bau' END AS sku_flag,
  r.location_code,
  r.id_location,
  r.barcode,
  r.id_partner,
  r.expiry_date          AS location_expiry_date,
  f.min_forward_expiry,
  r.qty                  AS location_qty,
  r.deep_qty,
  bi.true_batch_issue_qty
FROM sku_map sm
JOIN sku_qty_raw r
  ON sm.partner_warehouse_code = r.wh_code
 AND sm.pbarcode_canonical = r.barcode
 AND sm.id_partner = r.id_partner
```

```
LEFT JOIN forward_min f
  ON f.wh_code = r.wh_code AND f.barcode = r.barcode AND f.id_partner = r.id_partner
LEFT JOIN batch_issue_calc bi
  ON bi.wh_code = r.wh_code AND bi.barcode = r.barcode AND bi.id_partner = r.id_partner
LEFT JOIN top_2500 t2k ON t2k.sku = sm.zsku
WHERE r.is_deep = '1'
  AND r.expiry_date < f.min_forward_expiry
  AND bi.true_batch_issue_qty > 0
ORDER BY sm.zsku, r.barcode, r.expiry_date;
*/



/*
═══════════════════════════════════════════════════════════════════════
```

## Zone × Aisle health dashboard — AUHID01

```
 GRAIN     : One row per warehouse × zone × aisle
             Deep / Forward metrics are COLUMNS, not rows
 DATE SCOPE: Stock take >= 2026-04-14 (operational go-live date)
             Skip metrics = rolling last 7 days
 KEY FIXES :
   1. Date filter = >= '2026-04-14' (not MTD — Apr 1 had bulk cancellations)
   2. location_dim for job matching: is_active removed — inactive locations
      still receive stock-take tasks and must be counted
   3. location_dim for location COUNTS: is_active = 1 kept (display only)
═══════════════════════════════════════════════════════════════════════
*/

-- ─────────────────────────────────────────────────────────────────────
-- CTE 1a : Location dim for JOB MATCHING
--          No is_active filter — inactive locs still get ST tasks
-- ─────────────────────────────────────────────────────────────────────
WITH location_dim_jobs AS (
 SELECT DISTINCT
   w.partner_warehouse_code                          AS warehouse_code,
   z.zone_code,
   l.location_code,
```

```
    SPLIT(l.location_code, '-')[SAFE_OFFSET(1)]     AS aisle,
    JSON_EXTRACT_SCALAR(l.misc, '$.is_deep')        AS is_deep
  FROM `noondwh.mxdcss_dcss.warehouse`  w
  JOIN `noondwh.mxdcss_dcss.zone`       z  ON z.id_warehouse = w.id_warehouse
  JOIN `noondwh.mxdcss_dcss.location`   l  ON l.id_zone      = z.id_zone
  WHERE w.partner_warehouse_code = 'AUHID01'
    AND z.is_saleable = 1
    AND l.is_virtual  = 0
    and is_active = 1
    -- is_active intentionally excluded: inactive locs still receive tasks
),

-- ─────────────────────────────────────────────────────────────────
-- CTE 1b : Location dim for COUNTS (display: active locations only)
-- ─────────────────────────────────────────────────────────────────
location_dim_counts AS (
  SELECT DISTINCT
    w.partner_warehouse_code                        AS warehouse_code,
    z.zone_code,
    l.id_location,
    l.location_code,
    SPLIT(l.location_code, '-')[SAFE_OFFSET(1)]     AS aisle,
    JSON_EXTRACT_SCALAR(l.misc, '$.is_deep')        AS is_deep
  FROM `noondwh.mxdcss_dcss.warehouse`  w
  JOIN `noondwh.mxdcss_dcss.zone`       z  ON z.id_warehouse = w.id_warehouse
  JOIN `noondwh.mxdcss_dcss.location`   l  ON l.id_zone      = z.id_zone
  WHERE w.partner_warehouse_code = 'AUHID01'
    AND z.is_saleable = 1
    AND l.is_active   = 1
    AND l.is_virtual  = 0
),

-- ─────────────────────────────────────────────────────────────────
-- CTE 2 : Location counts — overall / deep / forward (active only)
-- ─────────────────────────────────────────────────────────────────
location_counts AS (
```

```sql
 SELECT
   warehouse_code,
   zone_code,
   aisle,
   COUNT(DISTINCT id_location)                                    AS
total_locations,
   COUNT(DISTINCT CASE WHEN is_deep = '1' THEN id_location END)     AS deep_locations,
   COUNT(DISTINCT CASE WHEN is_deep = '0' THEN id_location END)     AS
forward_locations
 FROM location_dim_counts
 GROUP BY 1, 2, 3
),

-- ─────────────────────────────────────────────────────────────────
-- CTE 3 : Stock-take jobs from 2026-04-14 onwards
--         Excludes force_closed & cancelled
--         Uses location_dim_jobs (no is_active filter)
-- ─────────────────────────────────────────────────────────────────
stock_take_base AS (
 SELECT
   w.partner_warehouse_code                    AS warehouse_code,
   ld.zone_code,
   ld.aisle,
   ld.is_deep,
   ld.location_code                            AS job_location_code,
   j.job_nr,
   s.code                                      AS job_status
 FROM `noondwh.mxdcss_dcss.job`          j
 JOIN `noondwh.mxdcss_dcss.status`       s   USING (id_status)
 JOIN `noondwh.mxdcss_dcss.warehouse`    w   USING (id_warehouse)
 JOIN `noondwh.mxdcss_dcss.process`      p   USING (id_process)
 JOIN `noondwh.mxdcss_dcss.process_type` pt  USING (id_process_type)
 JOIN location_dim_jobs                  ld
   ON  ld.warehouse_code = w.partner_warehouse_code
  AND  ld.location_code  = JSON_EXTRACT_SCALAR(j.misc, '$.location_code')
 WHERE j.id_job_subtype           = 10
```

```
  AND pt.code                    = 'routine_stock_take'
  AND w.partner_warehouse_code   = 'AUHID01'
  AND s.code NOT IN ('force_closed', 'cancelled')
  AND DATE(DATETIME(j.created_at, 'Asia/Dubai')) >= DATE '2026-04-14'
),

stock_take_agg AS (
 SELECT
  warehouse_code,
  zone_code,
  aisle,
  COUNT(DISTINCT job_nr)                                                AS st_created,
  COUNT(DISTINCT CASE WHEN job_status = 'closed' THEN job_nr END)    AS st_closed,
  COUNT(DISTINCT CASE WHEN is_deep = '1' THEN job_nr END)            AS
st_created_deep,
  COUNT(DISTINCT CASE WHEN is_deep = '1' AND job_status = 'closed'
                      THEN job_nr END)                               AS
st_closed_deep,
  COUNT(DISTINCT CASE WHEN is_deep = '0' THEN job_nr END)            AS
st_created_fwd,
  COUNT(DISTINCT CASE WHEN is_deep = '0' AND job_status = 'closed'
                      THEN job_nr END)                               AS
st_closed_fwd,
  ROUND(SAFE_DIVIDE(
    COUNT(DISTINCT job_nr),
    COUNT(DISTINCT job_location_code)
  ), 2)                                                              AS w2w_overall,
  ROUND(SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN is_deep = '1' THEN job_nr END),
    COUNT(DISTINCT CASE WHEN is_deep = '1' THEN job_location_code END)
  ), 2)                                                              AS w2w_deep,
  ROUND(SAFE_DIVIDE(
    COUNT(DISTINCT CASE WHEN is_deep = '0' THEN job_nr END),
    COUNT(DISTINCT CASE WHEN is_deep = '0' THEN job_location_code END)
  ), 2)                                                              AS w2w_fwd
 FROM stock_take_base
```

```
 GROUP BY 1, 2, 3
),

-- ──────────────────────────────────────────────────────────────────
-- CTE 4 : Skip metrics — rolling 7 days (cwpicking)
-- ──────────────────────────────────────────────────────────────────
skip_base AS (
 SELECT
   src_w.partner_warehouse_code                    AS warehouse_code,
   DATE(
     DATETIME(pj.created_at, 'Asia/Dubai')
     - INTERVAL 6 HOUR - INTERVAL 30 MINUTE
   )                                               AS biz_day,
   ld.zone_code,
   ld.aisle,
   ld.is_deep,
   pll.id_picklist_line
 FROM `noondwh.scgoms_cwpicking.picking_job`          pj
 JOIN `noondwh.scgoms_cwpicking.status`               pjst
   ON pjst.id_status = pj.id_status AND pjst.code = 'closed'
 JOIN `noondwh.scgoms_cwpicking.picklist_line`        pll
   ON pll.id_picking_job = pj.id_picking_job
  AND pll.id_old_picklist_line IS NULL
 JOIN `noondwh.scgoms_cwpicking.status`               pllst
   ON pllst.id_status = pll.id_status AND pllst.code = 'skipped'
 JOIN `noondwh.scgoms_cwpicking.reason`               r
   ON r.id_reason = pll.id_skip_reason AND r.code IN ('missing','damaged','expired')
 JOIN `noondwh.scgoms_cwpicking.warehouse`            src_w
   ON src_w.id_warehouse = pll.id_warehouse
  AND src_w.partner_warehouse_code = 'AUHID01'
 LEFT JOIN `noondwh.scgoms_cwpicking.picking_container_line` plc
   ON plc.id_picklist_line = pll.id_picklist_line
 JOIN location_dim_jobs                               ld
   ON  ld.warehouse_code = src_w.partner_warehouse_code
  AND  ld.location_code  = pll.location_code
 WHERE plc.id_picking_container_line IS NULL
```

```
   AND DATE(
         DATETIME(pj.created_at,'Asia/Dubai')
         - INTERVAL 6 HOUR - INTERVAL 30 MINUTE
       ) between CURRENT_DATE('Asia/Dubai') - 7 and CURRENT_DATE('Asia/Dubai') - 1
),

skip_wh_totals AS (
 SELECT
   warehouse_code,
   COUNT(DISTINCT id_picklist_line)                                    AS wh_total_7d,
   COUNT(DISTINCT CASE WHEN biz_day = CURRENT_DATE('Asia/Dubai') - 1
                       THEN id_picklist_line END)                      AS wh_total_d1
 FROM skip_base
 GROUP BY 1
),

skip_agg AS (
 SELECT
   sb.warehouse_code, sb.zone_code, sb.aisle,
   COUNT(DISTINCT sb.id_picklist_line)                                 AS
skip_items_7d,
   ROUND(SAFE_DIVIDE(COUNT(DISTINCT sb.id_picklist_line)*100.0,
                     MAX(wt.wh_total_7d)),2)                           AS
skip_contrib_7d_pct,
   ROUND(SAFE_DIVIDE(COUNT(DISTINCT id_picklist_line), 7), 1) AS avg_skips_per_day_7d,
   COUNT(DISTINCT CASE WHEN sb.is_deep='1' THEN sb.id_picklist_line END) AS
skip_items_7d_deep,
   ROUND(SAFE_DIVIDE(COUNT(DISTINCT CASE WHEN sb.is_deep='1'
                           THEN sb.id_picklist_line END)*100.0,
                     MAX(wt.wh_total_7d)),2)                           AS
skip_contrib_7d_pct_deep,
   COUNT(DISTINCT CASE WHEN sb.is_deep='0' THEN sb.id_picklist_line END) AS
skip_items_7d_fwd,
   ROUND(SAFE_DIVIDE(COUNT(DISTINCT CASE WHEN sb.is_deep='0'
                           THEN sb.id_picklist_line END)*100.0,
```

```
                    MAX(wt.wh_total_7d)),2)                              AS
skip_contrib_7d_pct_fwd,
   COUNT(DISTINCT CASE WHEN sb.biz_day = CURRENT_DATE('Asia/Dubai')-1
                       THEN sb.id_picklist_line END)                     AS
skip_items_d1,
   ROUND(SAFE_DIVIDE(COUNT(DISTINCT CASE WHEN sb.biz_day =
CURRENT_DATE('Asia/Dubai')-1
                           THEN sb.id_picklist_line END)*100.0,
                    MAX(wt.wh_total_d1)),2)                              AS
skip_contrib_d1_pct,
   COUNT(DISTINCT CASE WHEN sb.biz_day = CURRENT_DATE('Asia/Dubai')-1
                         AND sb.is_deep='1' THEN sb.id_picklist_line END) AS
skip_items_d1_deep,
   COUNT(DISTINCT CASE WHEN sb.biz_day = CURRENT_DATE('Asia/Dubai')-1
                         AND sb.is_deep='0' THEN sb.id_picklist_line END) AS
skip_items_d1_fwd
 FROM skip_base sb
 JOIN skip_wh_totals wt USING (warehouse_code)
 GROUP BY 1, 2, 3
)

-- ─────────────────────────────────────────────────────────────────────
-- FINAL
-- ─────────────────────────────────────────────────────────────────────
SELECT
 lc.warehouse_code                                    AS warehouse,
 lc.zone_code,
 lc.aisle,
 lc.total_locations,
 lc.deep_locations,
 lc.forward_locations,
 COALESCE(st.st_created,       0)                     AS st_created,
 COALESCE(st.st_closed,        0)                     AS st_closed,
 ROUND(SAFE_DIVIDE(COALESCE(st.st_closed,0),
       NULLIF(COALESCE(st.st_created,0),0)),4)       AS st_adherence_pct,
 COALESCE(st.st_created_deep,  0)                     AS st_created_deep,
```

```
 COALESCE(st.st_closed_deep,    0)                    AS st_closed_deep,
 ROUND(SAFE_DIVIDE(COALESCE(st.st_closed_deep,0),
       NULLIF(COALESCE(st.st_created_deep,0),0)),4)  AS st_adherence_deep_pct,
 COALESCE(st.st_created_fwd,    0)                    AS st_created_fwd,
 COALESCE(st.st_closed_fwd,     0)                    AS st_closed_fwd,
 ROUND(SAFE_DIVIDE(COALESCE(st.st_closed_fwd,0),
       NULLIF(COALESCE(st.st_created_fwd,0),0)),4)   AS st_adherence_fwd_pct,
 COALESCE(st.w2w_overall, 0)                          AS w2w_overall,
 COALESCE(st.w2w_deep,    0)                          AS w2w_deep,
 COALESCE(st.w2w_fwd,     0)                          AS w2w_fwd,
 COALESCE(sk.skip_items_7d,            0)             AS skip_items_7d,
 COALESCE(sk.skip_contrib_7d_pct,      0)             AS skip_contrib_7d_pct,
 COALESCE(sk.avg_skips_per_day_7d,     0)             AS avg_skips_per_day_7d,
 COALESCE(sk.skip_items_7d_deep,       0)             AS skip_items_7d_deep,
 COALESCE(sk.skip_contrib_7d_pct_deep, 0)             AS skip_contrib_7d_pct_deep,
 COALESCE(sk.skip_items_7d_fwd,        0)             AS skip_items_7d_fwd,
 COALESCE(sk.skip_contrib_7d_pct_fwd,  0)             AS skip_contrib_7d_pct_fwd,
 COALESCE(sk.skip_items_d1,            0)             AS skip_items_d1,
 COALESCE(sk.skip_contrib_d1_pct,      0)             AS skip_contrib_d1_pct,
 COALESCE(sk.skip_items_d1_deep,       0)             AS skip_items_d1_deep,
 COALESCE(sk.skip_items_d1_fwd,        0)             AS skip_items_d1_fwd
FROM location_counts lc
LEFT JOIN stock_take_agg st USING (warehouse_code, zone_code, aisle)
LEFT JOIN skip_agg       sk USING (warehouse_code, zone_code, aisle)
ORDER BY lc.zone_code, lc.aisle
```

```
/*
═══════════════════════════════════════════════════════════════
```

# ACTUAL MANPOWER — DAY-ON-DAY BY FUNCTION & OVERALL

```
 Purpose : Count of employees actually present per date,
           split by warehouse × Function, with an OVERALL row.
 Source  : noonbinimksa.Stores.wh_Biometric_base
 Grain   : date × wh_code × Function  (function-level rows)
           date × wh_code × 'OVERALL' (total row per warehouse)

 DEFINITIONS
 ─────────────
 total_roster        : all employees scheduled (working_day = 1)
 present             : valid_in_time IS NOT NULL
 absent              : on roster but no valid in-punch
 half_punch          : valid_in_time exists, valid_out_time is NULL (overlap
documented clearly in comments)
 avg_worked_hrs      : avg(worked_hours) for present employees only
 dod_present_change  : present count today vs yesterday (LAG)
 dod_pct_change      : % change in present count day-on-day

 NOTE: working_day = 1 filters to scheduled working days only.
       Remove this filter if you want to see all calendar days
       including week-offs.
 ═══════════════════════════════════════════════════════════════
*/



WITH bio AS (
 SELECT
   created_date                                          AS biz_date,
   wh_code,
   wh_name,
```

```
    Function                                                AS function_name,
    employee_id,
    CASE WHEN valid_in_time  IS NOT NULL THEN 1 ELSE 0 END  AS is_present,
    -- NOTE: half_punch is a SUBSET of present (in but no out).
    --       present + absent = total_roster, but half_punch overlaps present.
    CASE WHEN valid_in_time  IS NOT NULL
          AND  valid_out_time IS NULL  THEN 1 ELSE 0 END  AS is_half_punch,
    worked_hours
  FROM `noonbinimksa.Stores.wh_Biometric_base`
  WHERE working_day = 1
    AND created_date >= CURRENT_DATE('Asia/Riyadh') - 30
),

-- Function-level aggregation
fn_agg AS (
  SELECT
    biz_date, wh_code, wh_name,
    function_name,
    COUNT(DISTINCT employee_id)                               AS total_roster,
    COUNT(DISTINCT CASE WHEN is_present  = 1 THEN employee_id END) AS present,
    COUNT(DISTINCT CASE WHEN is_present  = 0 THEN employee_id END) AS absent,
    COUNT(DISTINCT CASE WHEN is_half_punch = 1 THEN employee_id END) AS half_punch,
    ROUND(AVG(CASE WHEN is_present = 1 THEN worked_hours END), 2) AS avg_worked_hrs
  FROM bio
  GROUP BY biz_date, wh_code, wh_name, function_name
),

-- Overall-level aggregation (all functions combined)
overall_agg AS (
  SELECT
    biz_date, wh_code, wh_name,
    'OVERALL'                                                 AS function_name,
    COUNT(DISTINCT employee_id)                                 AS total_roster,
    COUNT(DISTINCT CASE WHEN is_present  = 1 THEN employee_id END) AS present,
    COUNT(DISTINCT CASE WHEN is_present  = 0 THEN employee_id END) AS absent,
    COUNT(DISTINCT CASE WHEN is_half_punch = 1 THEN employee_id END) AS half_punch,
```

```
   ROUND(AVG(CASE WHEN is_present = 1 THEN worked_hours END), 2) AS avg_worked_hrs
 FROM bio
 GROUP BY biz_date, wh_code, wh_name    -- literal 'OVERALL' excluded from GROUP BY
),

-- Union both layers
combined AS (
 SELECT * FROM fn_agg
 UNION ALL
 SELECT * FROM overall_agg
),

lagged AS (
 SELECT
   *,
   LAG(present) OVER (
     PARTITION BY wh_code, function_name
     ORDER BY biz_date
   )                                                    AS prev_day_present
 FROM combined
)

-- Final output
SELECT
 biz_date,
 wh_code,
 wh_name,
 function_name,
 total_roster,
 present,
 absent,
 half_punch,
 ROUND(SAFE_DIVIDE(present, total_roster) * 100, 1)     AS attendance_pct,
 avg_worked_hrs,
 prev_day_present,
 present - prev_day_present                             AS dod_change,
```

```
 ROUND(
   SAFE_DIVIDE(present - prev_day_present,
               NULLIF(prev_day_present, 0)) * 100, 1
 )                                                    AS dod_pct_change

FROM lagged
ORDER BY wh_code, function_name, biz_date
```

```
/*
============================================================
```

## Vehicle GPS Distance & Driving Time Analysis

```
  Purpose:
    Measures distance traveled and effective driving time
    per vehicle per day, linked back to which warehouse(s)
    the vehicle served, using GPS pings from scgoms_trakker.
  Grain:     One row per vehicle per task-day
 Sources:
   - scgoms_goms.driver_task_line  → task events, entity linkage
   - scgoms_goms.driver_task_route → route grouping
   - scgoms_goms.driver_task       → vehicle + user assignment
   - scgoms_goms.user              → driver name
   - scgoms_goms.hub_transfer      → pallet → hub_transfer link
   - scgoms_goms.hub_transfer_line → hub_transfer → shipment link
   - scgoms_goms.shipment          → source warehouse
   - scgoms_goms.hub_transfer_group→ grouping (unused here, kept for context)
   - scgoms_trakker.trip           → GPS trip sessions
   - scgoms_trakker.trip_detail    → GPS segment-level distance & time

 Key filters:
   - Last 8 days (rolling window)
   - hub_transfer.type = 'forward_nim'  (exclude RTO / reverse logistics)
```

```
   - distance_kilometers > 2 for driving time (noise threshold)
   - distance_kilometers > 2 for distance too (consistent with time filter)

 Known behavior:
   - Warehouse column uses STRING_AGG — a vehicle serving multiple
     warehouses in a day will show comma-separated values (e.g. 'DXB, FNV')
   - next_day_task is NULL for a vehicle's last working day in the window;
     those days use end-of-day as the GPS window ceiling instead of NULL
============================================================
*/

-- ============================================================
-- CTE 1: vehicle_base
-- One row per vehicle per task-day.
-- Determines which warehouse(s) a vehicle served and
-- the time window of its tasks.
-- ============================================================
WITH vehicle_base AS (
 SELECT
   DATE(dtl.created_at, 'Asia/Dubai')                        AS task_created_at,
   src_w.id_country,

   -- Warehouse label: derived from partner_warehouse_code
   -- STRING_AGG intentionally de-dupes; multi-warehouse vehicles get
   -- comma-separated values like 'DXB, FNV'
   STRING_AGG(
     DISTINCT CASE
       WHEN src_w.partner_warehouse_code LIKE '%FNV%' THEN 'FNV'
       WHEN src_w.partner_warehouse_code LIKE '%FKZ%' THEN 'FKZ'
       WHEN src_w.partner_warehouse_code LIKE '%DXB%' THEN 'DXB'
       WHEN src_w.partner_warehouse_code LIKE '%AUH%' THEN 'AUH'
       WHEN src_w.partner_warehouse_code LIKE '%RUH%' THEN 'RUH'
       WHEN src_w.partner_warehouse_code LIKE '%JED%' THEN 'JED'
       ELSE 'Others'
     END,
     ', '
```

```
  )                                                       AS warehouse,

  dt.id_vehicle,
  MIN(u.username)                                         AS driver_name,

  -- Window start: earliest task event on this day for GPS join
  MIN(DATETIME(dtl.created_at, 'Asia/Dubai'))             AS first_task_created_at,

  -- Window end: latest task event on this day (used as fallback ceiling)
  MAX(DATETIME(dtl.created_at, 'Asia/Dubai'))             AS last_task_created_at,

  COUNT(DISTINCT dtl.id_driver_task)                      AS task_num

FROM `noondwh.scgoms_goms.driver_task_line`    dtl
LEFT JOIN `noondwh.scgoms_goms.driver_task_route` dtr  USING (id_driver_task_route)
LEFT JOIN `noondwh.scgoms_goms.driver_task`       dt   USING (id_driver_task)
LEFT JOIN `noondwh.scgoms_goms.user`              u    USING (id_user)

-- Link task line entity to a pallet-level hub_transfer
LEFT JOIN `noondwh.scgoms_goms.hub_transfer`      ht
  ON  dtl.entity_type = 'pallet'
  AND dtl.entity_id   = ht.id_hub_transfer
  AND ht.type         = 'forward_nim'            -- exclude RTO / reverse logistics

LEFT JOIN `noondwh.scgoms_goms.hub_transfer_line` htl  USING (id_hub_transfer)
LEFT JOIN `noondwh.scgoms_goms.hub_transfer_group` htg USING (id_hub_transfer_group)

-- Resolve shipment from hub_transfer_line (only shipment-type lines)
LEFT JOIN `noondwh.scgoms_goms.shipment`          sh
  ON  htl.line_id   = sh.id_shipment
  AND htl.line_type = 'shipment'

-- Source warehouse from shipment (not from polymorphic hub_transfer fields)
LEFT JOIN `noondwh.scgoms_goms.warehouse`         src_w
  ON src_w.id_warehouse = CAST(sh.id_warehouse AS INT64)
```

```
 WHERE DATE(dtl.created_at, 'Asia/Dubai') >= CURRENT_DATE('Asia/Dubai') - 8

 GROUP BY
   DATE(dtl.created_at, 'Asia/Dubai'),
   src_w.id_country,
   dt.id_vehicle
),

-- ============================================================
-- CTE 2: final_vehicle
-- Adds the next working day's start time per vehicle using LEAD.
-- Used to bound the GPS trip join: trips that START between
-- this day's first task and the next day's first task belong
-- to this day's run.
-- For the last day in the window, next_day_task is NULL —
-- we fall back to last_task_created_at + 1 hour as the ceiling.
-- ============================================================
final_vehicle AS (
 SELECT
   *,
   LEAD(first_task_created_at)
     OVER (PARTITION BY id_vehicle ORDER BY first_task_created_at) AS next_day_task
 FROM vehicle_base
),

-- ============================================================
-- CTE 3: driver_gps_base
-- Joins GPS trip data to the vehicle's task window.
--
-- GPS join logic:
--   trip_start (DATE) must fall within [first_task_created_at, ceiling)
--   ceiling = next_day_task if available, else last_task_created_at + 1hr
--
-- Distance filter:
--   Segments < 2 km excluded from BOTH distance and driving time
--   to avoid counting idle GPS noise / parking lot pings.
```

```sql
-- ============================================================
driver_gps_base AS (
 SELECT
   v.task_created_at,
   v.id_country,
   v.warehouse,
   v.id_vehicle,
   v.driver_name,
   v.task_num,

   -- Total distance, excluding short idle segments (< 2 km)
   SUM(
     CASE WHEN d.distance_kilometers > 2 THEN d.distance_kilometers ELSE 0 END
   )                                                    AS distance_km,

   -- Effective driving time in hours, same 2 km noise threshold
   SUM(
     CASE WHEN d.distance_kilometers > 2 THEN d.driving_time ELSE 0 END
   ) / 3600.0                                           AS total_driving_time_hrs

 FROM final_vehicle v

 -- Join GPS trips that started within this vehicle's task window for the day
 LEFT JOIN `noondwh.scgoms_trakker.trip` t
   ON  v.id_vehicle = t.id_vehicle
   AND DATE(t.trip_start, 'Asia/Dubai') >= DATE(v.first_task_created_at)
   AND DATE(t.trip_start, 'Asia/Dubai') <  DATE(
         COALESCE(v.next_day_task, DATETIME_ADD(v.last_task_created_at, INTERVAL 1
HOUR))
       )

 JOIN `noondwh.scgoms_trakker.trip_detail` d
   ON t.id_trip = d.id_trip

 GROUP BY
   v.task_created_at,
```

```
    v.id_country,
    v.warehouse,
    v.id_vehicle,
    v.driver_name,
    v.task_num
)

-- ============================================================
-- Final output
-- Adds a driving time bucket for quick segmentation.
-- Sorted by: country → date → warehouse → vehicle
-- ============================================================
SELECT
 *,
 CASE
   WHEN total_driving_time_hrs < 5 THEN 'LT_5hrs'
   ELSE 'GT_5hrs'
 END AS time_bucket
FROM driver_gps_base
ORDER BY
 id_country,
 task_created_at,
 warehouse,
 Id_vehicle
```

```
/*
--
================================================================================
=====
```

## Putaway Pendency : Open for more than 2 days, with remaining items

```
--          not yet completed. Used to track aging inbound backlogs across warehouses.
--
================================================================================
=====
*/
--
================================================================================
=====
-- CTE 1: pending_asn
-- PURPOSE: Get distinct ASN (process) numbers for inbound jobs that are still
--          in 'pending' or 'opened' status, created within the last 60 days.
--          This acts as a filter — we only want ASNs that are still active/unfinished.
--
================================================================================
=====
WITH pending_asn AS (
    SELECT
        p.process_nr AS asn_nr      -- ASN number (e.g., ASN-000123); unique ID for each
inbound shipment
```

```
   FROM noondwh.mxdcss_dcss.job j                                    -- Core jobs table (each
scan/task at the DC)
   LEFT JOIN noondwh.mxdcss_dcss.warehouse w   ON j.id_warehouse = w.id_warehouse  --
Warehouse where job is happening
   LEFT JOIN noondwh.mxdcss_dcss.status s      ON j.id_status = s.id_status        -- Status
lookup (pending, opened, closed, etc.)
   LEFT JOIN noondwh.mxdcss_dcss.process p     ON j.id_process = p.id_process      -- The
parent process (ASN) this job belongs to
   LEFT JOIN noondwh.mxdcss_dcss.process_type pt ON p.id_process_type =
pt.id_process_type  -- Process type (inbound, outbound, etc.)

   WHERE
      j.id_job_subtype IN (4, 5, 6)                    -- Filter: only inbound-related job subtypes
(receive/putaway/label)
      AND DATE(j.created_at, 'Asia/Dubai') >= CURRENT_DATE() - INTERVAL 60 DAY  -- Only
jobs created in last 60 days (UAE timezone)
      AND s.code IN ('pending', 'opened')                -- Only ASNs that are still active (not
closed/cancelled)
      AND pt.code IN ('inbound')                       -- Only inbound process types (not returns,
transfers, etc.)
      AND w.id_country = 1                            -- UAE only (country ID = 1)
   GROUP BY 1                                        -- Deduplicate: one row per ASN number
),

--
================================================================================
-- CTE 2: base
-- PURPOSE: For each job within the pending ASNs, fetch job-level details including:
--          - Container barcode (the tote or carton being processed)
--          - User who worked on it
--          - Item barcode
--          - Count of remaining vs completed job lines
--          - Earliest job date for the ASN (used for aging calculation)
--
================================================================================
base AS (
   SELECT
      j.job_nr,                                  -- Unique job number (e.g., JOB-000456); one job =
one scan task
      w.partner_warehouse_code,                     -- External/partner code for the warehouse
(e.g., DXBID01)
      p.process_nr AS asn_nr,                    -- The ASN number this job belongs to
```

```
    COALESCE(jc.tote_code, jc.barcode) AS container_barcode,  -- Container identifier; prefer
tote_code, fall back to barcode
    u.username,                              -- The DC operator/user who worked on this job
    i.barcode AS pbarcode,                       -- Product/item barcode for catalog enrichment
later (used for product title join)

    -- Earliest job creation date across all jobs in this ASN (Dubai timezone)
    -- Used as the "ASN received date" for aging calculation
    MIN(DATE(j.created_at, 'Asia/Dubai')) OVER (PARTITION BY p.process_nr) AS
min_asn_job_date,

    -- Count of job lines still not completed (i.e., job is active and line isn't done)
    -- id_status = 3 is "completed"; all others are pending/in-progress
    COUNT(CASE WHEN s.code IN ('pending', 'opened') AND jl.id_status != 3 THEN 1 ELSE
NULL END) AS items_remaining,

    -- Count of job lines that have been fully processed/completed
    COUNT(CASE WHEN jl.id_status = 3 THEN 1 ELSE NULL END) AS items_completed

  FROM noondwh.mxdcss_dcss.job j
  LEFT JOIN noondwh.mxdcss_dcss.warehouse w       ON j.id_warehouse = w.id_warehouse
  LEFT JOIN noondwh.mxdcss_dcss.status s          ON j.id_status = s.id_status
  LEFT JOIN noondwh.mxdcss_dcss.user u            ON j.id_user = u.id_user              -- Who
performed the job
  LEFT JOIN noondwh.mxdcss_dcss.job_subtype js    ON j.id_job_subtype = js.id_job_subtype
-- Subtype details (not used in output but available)
  LEFT JOIN noondwh.mxdcss_dcss.process p         ON j.id_process = p.id_process
  LEFT JOIN noondwh.mxdcss_dcss.process_type pt   ON p.id_process_type =
pt.id_process_type
  LEFT JOIN noondwh.mxdcss_dcss.job_container jc  ON jc.id_job = j.id_job              --
Container (tote/carton) assigned to this job
  LEFT JOIN noondwh.mxdcss_dcss.job_line jl       ON jl.id_job = j.id_job              -- Individual
line items within the job (1 line = 1 unit/SKU task)
  LEFT JOIN noondwh.mxdcss_dcss.item i            USING (id_item)                      --
Item/product details (barcode)

  WHERE
    j.id_job_subtype IN (4, 5, 6)                          -- Inbound job subtypes only
    AND DATE(j.created_at, 'Asia/Dubai') >= CURRENT_DATE() - INTERVAL 60 DAY  -- Last
60 days (UAE timezone)
    AND w.id_country = 1                                   -- UAE only
    AND p.process_nr IN (SELECT * FROM pending_asn)                    -- Only jobs belonging
to still-pending ASNs
```

```
    GROUP BY
        1, 2, 3, 4, 5, 6, j.created_at  -- Group at job+container+user+item level
)

--
================================================================================
-- FINAL SELECT
-- PURPOSE: Enrich base data with:
--          1. WH_Category: human-readable warehouse grouping for reporting
--          2. product_title: product name from catalog join
--          3. ageing: how many days since the ASN first arrived at the DC
-- FILTERS: Only ASNs older than 2 days (min_asn_job_date <= today - 2)
--          AND that still have unprocessed items (items_remaining > 0)
--
================================================================================
SELECT
    b.*,                -- All columns from base CTE

    -- Warehouse category mapping: groups partner warehouse codes into business-friendly
labels
    -- Helps report by fulfilment zone (FZ Kezad, TP Chiller, FNV, etc.)
    CASE
        WHEN partner_warehouse_code IN ('AUHFKZ01', 'AUHFKZ02', 'AUHFKZ03') THEN 'FZ
Kezad'        -- Free Zone Kezad warehouses
        WHEN partner_warehouse_code IN ('AUHID01', 'AUHID02')              THEN
'Kezad_Ambient'    -- Kezad ambient-temp storage
        WHEN partner_warehouse_code IN ('AUHID03', 'AUHID04')              THEN
'Kezad_Chiller'    -- Kezad chiller/cold-chain storage
        WHEN partner_warehouse_code IN ('DXBID01', 'DXBID02')              THEN 'TP_Ambient'
-- Third Party Dubai ambient
        WHEN partner_warehouse_code IN ('DXBID03', 'DXBID04')              THEN 'TP_Chiller'
-- Third Party Dubai chiller
        WHEN partner_warehouse_code IN ('DXBID07')                         THEN 'TP_Ultrafresh'
-- Ultra-fresh (very short shelf-life) Dubai
        WHEN partner_warehouse_code IN ('DXBFNV01')                        THEN 'FNV'            --
Fruits & Vegetables dedicated warehouse
        ELSE 'Other'
    END AS WH_Category,

    -- Product title from the Noon catalog; joined via pbarcode (canonical product barcode)
    -- Source: noonbinimops.fulfillment.mmmcaa_darkstore — the merchandise master / catalog
table
```

```
    MAX(product_title) AS product_title,  -- MAX used to collapse duplicates from GROUP BY ALL

    -- Aging in days: how long this ASN has been sitting unprocessed
    -- Calculated from the earliest job date associated with this ASN to today
    DATE_DIFF(CURRENT_DATE, min_asn_job_date, DAY) AS ageing  -- ageing = number of days since first inbound job for this ASN

FROM base b
LEFT JOIN noonbinimops.fulfillment.mmmcaa_darkstore m
    ON b.pbarcode = m.pbarcode_canonical  -- Join on canonical product barcode to get product title

WHERE
    min_asn_job_date <= CURRENT_DATE - 2    -- Only ASNs that have been pending for more than 2 days (exclude very recent)
    AND items_remaining > 0                 -- Only ASNs that still have work to do (exclude fully completed ones)

GROUP BY ALL         -- BigQuery syntax: group by all non-aggregated columns automatically
ORDER BY
    partner_warehouse_code,   -- Primary sort: group by warehouse
    min_asn_job_date          -- Secondary sort: oldest ASNs first within each warehouse
```

```
/* ============================================================================
```

## WH - Picking, Sorting, and QPL metrics

```
  It includes:
  - Picking productivity (IPH, time, qty)
  - Sorting productivity (time, qty, IPH)
  - QPL (Quantity per line)
  - Breakdown by job types (MS, BD, BS, Batch)

  BUSINESS LOGIC:
  - Business day = local time - 6 hours (6 AM cutoff)
  - Timezones vary by country
  - Only closed jobs are considered
  - Last ~8–10 days of data included

============================================================================ */

WITH base AS (

/* ============================================================================
  BASE LAYER (Job-level aggregation)
  - Each row represents a picking job
```

```
  - Calculates time, picked qty, containers used
============================================================================== */
SELECT
 src_w.partner_warehouse_code AS warehouse_code,    -- Warehouse identifier
 pj.job_type,                                       -- Type of picking job
 pj.id_user,                                        -- Picker ID
 pj.job_nr,                                         -- Job number

 /* ----------------------------------------------------------------------------
    BUSINESS DATE LOGIC:
    Convert UTC → local timezone → subtract 6 hours
    This shifts business day to start at 6 AM
 ---------------------------------------------------------------------------- */
 DATE(
   DATETIME(
     pj.created_at,
     CASE
       WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
       WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
       WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
     END
   ) - INTERVAL 6 HOUR
 ) AS date,

 /* Job accepted timestamp (localized) */
 DATETIME(
   pj.created_at,
   CASE
     WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
     WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
     WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
   END
 ) AS job_accepted_at,

 /* Job closed timestamp (localized) */
 DATETIME(
```

```
  pj.finsihed_picking_at,
  CASE
    WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
    WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
    WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
  END
) AS job_closed_at,

/* ------------------------------------------------------------------------
   TOTAL JOB DURATION (in seconds)
   Raw difference between job start and end
------------------------------------------------------------------------ */
DATETIME_DIFF(
  pj.finsihed_picking_at,
  pj.created_at,
  SECOND
) AS delta,

/* Total picked items (distinct lines) */
COUNT(DISTINCT pll.id_picklist_line) AS picked_items,

/* Unique containers used per job */
COUNT(
  DISTINCT COALESCE(pcr.container_barcode, pcr.tote_code)
) AS containers_used,

/* First picking scan timestamp */
MIN(
  DATETIME(
    pcl.created_at,
    CASE
      WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
      WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
      WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
    END
  )
```

```
 ) AS min_picked_at

FROM noondwh.scgoms_cwpicking.picking_job pj

/* Join job status → filter only closed jobs */
JOIN noondwh.scgoms_cwpicking.status pjst
 ON pj.id_status = pjst.id_status

/* Picklist lines (items picked) */
JOIN noondwh.scgoms_cwpicking.picklist_line pll
 ON pll.id_picking_job = pj.id_picking_job

/* Warehouse details */
JOIN noondwh.scgoms_cwpicking.warehouse src_w
 ON pll.id_warehouse = src_w.id_warehouse

/* Container line (scan events) */
JOIN noondwh.scgoms_cwpicking.picking_container_line pcl
 ON pcl.id_picklist_line = pll.id_picklist_line

/* Container details */
JOIN noondwh.scgoms_cwpicking.picking_container pcr
 ON pcr.id_picking_container = pcl.id_picking_container

WHERE
 pjst.code = 'closed'  -- Only completed jobs
 AND pcl.id_picking_container_line IS NOT NULL

 /* Filter last 8 business days */
 AND DATE(
       DATETIME(
         pj.created_at,
         CASE
           WHEN src_w.id_country = 1 THEN 'Asia/Dubai'
           WHEN src_w.id_country IN (2,5) THEN 'Asia/Riyadh'
           WHEN src_w.id_country = 3 THEN 'Africa/Cairo'
```

```
        END
      ) - INTERVAL 6 HOUR
    ) >= CURRENT_DATE - 8

GROUP BY ALL
),

/* ============================================================================
  USER LEVEL AGGREGATION
  - Aggregates multiple jobs per user per day
============================================================================ */
user_level AS (
SELECT
 date,
 warehouse_code,
 job_type,
 id_user,

 /* Total picked quantity */
 SUM(picked_items) AS picked_qty,

 /* Total containers used */
 SUM(containers_used) AS containers_used,

 /* ----------------------------------------------------------------------
    EFFECTIVE TIME CALCULATION:
    If delay > 10 min before first scan → remove idle time
    Else use full job duration
 ---------------------------------------------------------------------- */
 SUM(
   CASE
     WHEN DATETIME_DIFF(min_picked_at, job_accepted_at, SECOND) > 600
     THEN DATETIME_DIFF(job_closed_at, min_picked_at, SECOND)
     ELSE delta
   END
 ) AS effective_time_sec
```

```sql
FROM base
GROUP BY ALL
),

/* ================================================================================
  PICKING METRICS (Warehouse Level)
================================================================================ */
pick_df AS (
SELECT
 date,
 warehouse_code,

 /* Overall totals */
 SUM(picked_qty) AS Total_Picked_Qty,
 SUM(effective_time_sec) AS total_time_sec,
 SUM(containers_used) AS pick_containers_used,

 /* ================= MULTI STORE PICKING ================= */
 SUM(CASE WHEN job_type = 'multi_store_picking' THEN picked_qty END) AS MS_PickedQty,
 SUM(CASE WHEN job_type = 'multi_store_picking' THEN containers_used END) AS
MS_UniqueContainers,
 SUM(CASE WHEN job_type = 'multi_store_picking' THEN effective_time_sec END) AS
MS_TimeSec,

 /* IPH = Items per hour */
 SAFE_DIVIDE(
   3600 * SUM(CASE WHEN job_type = 'multi_store_picking' THEN picked_qty END),
   SUM(CASE WHEN job_type = 'multi_store_picking' THEN effective_time_sec END)
 ) AS MS_IPH,

 /* ================= BATCH DYNAMIC ================= */
 SUM(CASE WHEN job_type = 'batch_dynamic' THEN picked_qty END) AS BD_PickedQty,
 SUM(CASE WHEN job_type = 'batch_dynamic' THEN containers_used END) AS
BD_UniqueContainers,
 SUM(CASE WHEN job_type = 'batch_dynamic' THEN effective_time_sec END) AS BD_TimeSec,
```

```
 SAFE_DIVIDE(
   3600 * SUM(CASE WHEN job_type = 'batch_dynamic' THEN picked_qty END),
   SUM(CASE WHEN job_type = 'batch_dynamic' THEN effective_time_sec END)
 ) AS BD_IPH,

 /* ================ BATCH STATIC ================ */
 SUM(CASE WHEN job_type = 'batch_static' THEN picked_qty END) AS BS_PickedQty,
 SUM(CASE WHEN job_type = 'batch_static' THEN containers_used END) AS
BS_UniqueContainers,
 SUM(CASE WHEN job_type = 'batch_static' THEN effective_time_sec END) AS BS_TimeSec,

 SAFE_DIVIDE(
   3600 * SUM(CASE WHEN job_type = 'batch_static' THEN picked_qty END),
   SUM(CASE WHEN job_type = 'batch_static' THEN effective_time_sec END)
 ) AS BS_IPH,

 /* ================ COMBINED BATCH ================ */
 SUM(CASE WHEN job_type IN ('batch_static','batch_dynamic') THEN picked_qty END) AS
Batch_PickedQty,
 SUM(CASE WHEN job_type IN ('batch_static','batch_dynamic') THEN containers_used END)
AS Batch_UniqueContainers,
 SUM(CASE WHEN job_type IN ('batch_static','batch_dynamic') THEN effective_time_sec
END) AS Batch_TimeSec,

 SAFE_DIVIDE(
   3600 * SUM(CASE WHEN job_type IN ('batch_static','batch_dynamic') THEN picked_qty
END),
   SUM(CASE WHEN job_type IN ('batch_static','batch_dynamic') THEN effective_time_sec
END)
 ) AS Batch_IPH,

 /* Number of unique pickers */
 COUNT(DISTINCT id_user) AS pickers

FROM user_level
```

```sql
GROUP BY date, warehouse_code
),

/* ================================================================================
  SORTING LOGIC
================================================================================ */
sort_base AS (
SELECT
 /* Business date logic (same as picking) */
 DATE(
   DATETIME(
     al3.created_at,
     CASE
       WHEN w.id_country = 1 THEN 'Asia/Dubai'
       WHEN w.id_country IN (2,5) THEN 'Asia/Riyadh'
       WHEN w.id_country = 3 THEN 'Africa/Cairo'
     END
   ) - INTERVAL 6 HOUR
 ) AS date,

 w.partner_warehouse_code AS warehouse_code,
 sj.id_user,
 pj.job_type AS picking_job_type,
 it.id_item,

 /* Putaway timestamp */
 DATETIME(
   al3.created_at,
   CASE
     WHEN w.id_country = 1 THEN 'Asia/Dubai'
     WHEN w.id_country IN (2,5) THEN 'Asia/Riyadh'
     WHEN w.id_country = 3 THEN 'Africa/Cairo'
   END
 ) AS putaway_time

FROM noondwh.scgoms_goms.input_container ic
```

```
JOIN noondwh.scgoms_goms.sortation_job sj
 ON ic.id_sortation_job = sj.id_sortation_job
JOIN noondwh.scgoms_goms.warehouse w
 ON ic.id_warehouse = w.id_warehouse
JOIN noondwh.scgoms_goms.input_container_line icl
 ON ic.id_input_container = icl.id_input_container
JOIN noondwh.scgoms_goms.item it
 ON icl.id_item = it.id_item
JOIN noondwh.scgoms_goms.audit_log al3
 ON al3.id_audit_action = 21
 AND al3.action_key = icl.id_input_container_line
 AND JSON_EXTRACT_SCALAR(al3.misc, '$.to') = 'putaway'
 AND DATE(al3.created_at) >= CURRENT_DATE - 9
JOIN noondwh.scgoms_cwpicking.picking_container pc
 ON pc.container_barcode = ic.container_barcode
JOIN noondwh.scgoms_cwpicking.picking_job pj
 ON pj.id_picking_job = pc.id_picking_job

WHERE
 sj.job_type = 'nonsort'
 AND it.transfer_nr IS NOT NULL
 AND sj.id_user IS NOT NULL

 /* Filter last 8 days */
 AND DATE(
       DATETIME(al3.created_at,
         CASE
           WHEN w.id_country = 1 THEN 'Asia/Dubai'
           WHEN w.id_country IN (2,5) THEN 'Asia/Riyadh'
           WHEN w.id_country = 3 THEN 'Africa/Cairo'
         END
       ) - INTERVAL 6 HOUR
     ) >= CURRENT_DATE - 8
),

/* Calculate time gap between scans */
```

```
sort_scan AS (
SELECT
 *,
 DATETIME_DIFF(
   putaway_time,
   LAG(putaway_time) OVER (PARTITION BY id_user, date ORDER BY putaway_time),
   SECOND
 ) AS item_time
FROM sort_base
),

/* Aggregate sorting metrics */
sort_df AS (
SELECT
 date,
 warehouse_code,

 /* Batch Dynamic sorting */
 COUNT(DISTINCT CASE WHEN picking_job_type = 'batch_dynamic' THEN id_item END) AS
BD_sorting_qty,
 SUM(
   CASE WHEN picking_job_type = 'batch_dynamic' THEN
     CASE
       WHEN item_time <= 300 THEN item_time
       WHEN item_time IS NULL THEN 300
       ELSE 0
     END
   END
 ) AS BD_sort_time_sec,

 /* Batch Static sorting */
 COUNT(DISTINCT CASE WHEN picking_job_type = 'batch_static' THEN id_item END) AS
BS_sorting_qty,
 SUM(
   CASE WHEN picking_job_type = 'batch_static' THEN
     CASE
```

```
      WHEN item_time <= 300 THEN item_time
      WHEN item_time IS NULL THEN 300
      ELSE 0
    END
  END
 ) AS BS_sort_time_sec,

 /* Overall sorting */
 COUNT(DISTINCT id_item) AS sorting_qty_overall,
 SUM(
  CASE
    WHEN item_time <= 300 THEN item_time
    WHEN item_time IS NULL THEN 300
    ELSE 0
  END
 ) AS sorting_time_overall

FROM sort_scan
GROUP BY date, warehouse_code
),

/* ================================================================================
  QPL (Quantity Per Line)
================================================================================ */
qpl_df AS (
SELECT
 DATE(
  DATETIME(
    tr.created_at,
    CASE
      WHEN w.id_country = 1 THEN 'Asia/Dubai'
      WHEN w.id_country IN (2,5) THEN 'Asia/Riyadh'
      WHEN w.id_country = 3 THEN 'Africa/Cairo'
    END
  )
 ) AS date,
```

```sql
 w.partner_warehouse_code AS warehouse_code,

 /* QPL = total qty / total lines */
 SAFE_DIVIDE(
   SUM(trl.quantity),
   COUNT(trl.id_transfer_request_line)
 ) AS Overall_QPL

FROM noondwh.scgoms_cwpicking.transfer_request tr
JOIN noondwh.scgoms_cwpicking.transfer_request_line trl
 USING (id_transfer_request)
JOIN noondwh.scgoms_cwpicking.warehouse w
 ON tr.id_src_warehouse = w.id_warehouse

WHERE
 DATE(
   DATETIME(
     tr.created_at,
     CASE
       WHEN w.id_country = 1 THEN 'Asia/Dubai'
       WHEN w.id_country IN (2,5) THEN 'Asia/Riyadh'
       WHEN w.id_country = 3 THEN 'Africa/Cairo'
     END
   )
 ) >= CURRENT_DATE - 10

GROUP BY date, warehouse_code
)

/* ============================================================================
  FINAL OUTPUT
============================================================================ */
SELECT
 p.date                                          AS Pick_Date,
 p.warehouse_code                                AS Warehouse_ID,
```

```
COALESCE(c.warehouse_catrgory, 'Other')           AS WH_Category,

q.Overall_QPL,

Total_Picked_Qty,
total_time_sec / 3600                             AS Picking_hrs,
Total_Picked_Qty * 3600 / total_time_sec          AS Picking_IPH,

/* Multi Store */
p.MS_PickedQty,
p.MS_UniqueContainers,
ROUND(p.MS_TimeSec / 3600, 2)                     AS MS_PickedTime,
p.MS_IPH,

/* Batch Dynamic */
p.BD_PickedQty,
p.BD_UniqueContainers,
ROUND(p.BD_TimeSec / 3600, 2)                     AS BD_PickedTime,
p.BD_IPH,

/* Batch Static */
p.BS_PickedQty,
p.BS_UniqueContainers,
ROUND(p.BS_TimeSec / 3600, 2)                     AS BS_PickedTime,
p.BS_IPH,

/* Sorting */
s.BD_sorting_qty,
ROUND(s.BD_sort_time_sec / 3600, 2)               AS BD_Sort_Time,
SAFE_DIVIDE(3600 * s.BD_sorting_qty, s.BD_sort_time_sec) AS BD_Sort_IPH,

s.BS_sorting_qty,
ROUND(s.BS_sort_time_sec / 3600, 2)               AS BS_Sort_Time,
SAFE_DIVIDE(3600 * s.BS_sorting_qty, s.BS_sort_time_sec) AS BS_Sort_IPH,

pick_containers_used,
```

```
 pickers,

 Batch_PickedQty,
 Batch_UniqueContainers,
 Batch_TimeSec,
 Batch_IPH,

 sorting_qty_overall,
 sorting_time_overall

FROM pick_df p
LEFT JOIN sort_df s
 ON p.date = s.date AND p.warehouse_code = s.warehouse_code
LEFT JOIN qpl_df q
 ON p.date = q.date AND p.warehouse_code = q.warehouse_code
LEFT JOIN noonbinimops.fulfillment.wh_code_wh_category_mapping c
 ON p.warehouse_code = c.warehouse_code;
```

```
/* ================================================================================
```

## Fake Stocktake Identification Analysis

```
  PURPOSE:
  Identify correlation between:
  - Stocktake audit locations (deep vs forward)
  - Skipped picks (missing/damaged/expired)
  - SKU classification (Top vs BAU)

  OUTPUT:
  - Locations audited
  - User who audited
  - Whether location is Deep/Forward
  - Whether SKU is Top or BAU
  - If pick skip happened within 24 hours after audit

================================================================================ */

WITH top_2500 AS (

/* ================================================================================
```

```
  TOP SKU LIST
  - Extract top SKUs (Top 2500 items) for UAE
  - Used to classify picks into:
       'top' → high priority SKUs
       'bau' → regular SKUs
============================================================================== */
SELECT
 sku
FROM noonbiinstnt.minutes.instock_views_current_snapshot
WHERE country_code = 'ae'
 AND is_top = 1
GROUP BY ALL
),

/* ============================================================================
  DEEP LOCATION IDENTIFICATION
  - Identifies whether a location is "Deep storage"
  - Deep = bulk storage (back area)
  - Forward = picking/front area
============================================================================== */
deep_loc AS (
SELECT
 w.partner_warehouse_code AS wh_code,    -- Warehouse ID
 l.location_code,                        -- Location identifier

 /* Extract "is_deep" flag from JSON metadata */
 JSON_EXTRACT_SCALAR(l.misc, '$.is_deep') AS is_deep

FROM noondwh.mxdcss_dcss.country c

LEFT JOIN noondwh.mxdcss_dcss.warehouse w USING (id_country)
LEFT JOIN noondwh.mxdcss_dcss.tenant t USING (id_warehouse)
LEFT JOIN noondwh.mxdcss_dcss.zone z USING (id_tenant)
LEFT JOIN noondwh.mxdcss_dcss.location l USING (id_zone)
LEFT JOIN noondwh.mxdcss_dcss.location_item li USING (id_location)
JOIN noondwh.mxdcss_dcss.item i USING (id_item)
```

```sql
WHERE
 w.partner_warehouse_code IN ('AUHID01','AUHID02')  -- Target warehouses
 AND z.is_saleable = 1                              -- Only saleable zones
 AND JSON_EXTRACT_SCALAR(l.misc, '$.is_deep') = '1' -- Only deep locations

GROUP BY ALL
),

/* ============================================================================
  AUDIT LOCATIONS (STOCKTAKE)
  - Captures stocktake jobs performed on locations
  - Determines whether location is Deep or Forward
============================================================================ */
audit_locs AS (
SELECT
   /* Business date (localized) */
   DATE(j.updated_at, 'Asia/Dubai') AS updated_date,

   /* Exact timestamp */
   DATETIME(j.updated_at, 'Asia/Dubai') AS updated_at,

   w.partner_warehouse_code AS wh_code,

   /* Extract location_code from job metadata */
   JSON_EXTRACT_SCALAR(j.misc, '$.location_code') AS location_code,

   /* Classify location type */
   CASE
     WHEN dl.location_code IS NOT NULL THEN 'Deep'
     ELSE 'Forward'
   END AS location_type,

   /* User details */
   u.username,
   u.email
```

```sql
FROM noondwh.mxdcss_dcss.job j

JOIN noondwh.mxdcss_dcss.status s USING (id_status)
JOIN noondwh.mxdcss_dcss.warehouse w USING (id_warehouse)
JOIN noondwh.mxdcss_dcss.process p USING (id_process)
JOIN noondwh.mxdcss_dcss.process_type pt USING (id_process_type)

/* Join with deep locations to classify */
LEFT JOIN deep_loc dl
 ON dl.location_code = JSON_EXTRACT_SCALAR(j.misc, '$.location_code')
 AND dl.wh_code = w.partner_warehouse_code

/* User info */
LEFT JOIN noondwh.mxdcss_dcss.user u
 ON u.id_user = j.id_user

WHERE
 DATE(j.updated_at, 'Asia/Dubai') >= CURRENT_DATE - 3   -- Last 3 days
 AND s.code IN ('closed')                               -- Completed jobs
 AND w.partner_warehouse_code IN ('AUHID01','AUHID02')
 AND j.id_job_subtype = 10                              -- Stocktake jobs

GROUP BY ALL
),

/* ============================================================================
  PICK SKIP DATA
  - Captures skipped picks due to:
       missing, damaged, expired
  - Identifies SKU type (Top vs BAU)
============================================================================ */
pick AS (
SELECT
 src_w.partner_warehouse_code AS wh_code,
```

```sql
 /* Pick date and timestamp */
 DATE(pll.updated_at, 'Asia/Dubai') AS created_date,
 DATETIME(pll.updated_at, 'Asia/Dubai') AS created_at,

 /* SKU classification */
 CASE
   WHEN tt.sku IS NOT NULL THEN 'top'
   ELSE 'bau'
 END AS sku_type,

 pll.location_code

FROM noondwh.scgoms_cwpicking.picking_job pj

LEFT JOIN noondwh.scgoms_cwpicking.status pjst
 ON pj.id_status = pjst.id_status

LEFT JOIN noondwh.scgoms_cwpicking.picklist_line pll
 USING (id_picking_job)

LEFT JOIN noondwh.scgoms_cwpicking.status pllst
 ON pll.id_status = pllst.id_status

LEFT JOIN noondwh.scgoms_cwpicking.warehouse src_w
 ON pll.id_warehouse = src_w.id_warehouse

LEFT JOIN noondwh.scgoms_cwpicking.reason r
 ON r.id_reason = pll.id_skip_reason

LEFT JOIN noondwh.scgoms_cwpicking.warehouse_zone wz
 ON pll.id_warehouse_zone = wz.id_warehouse_zone

LEFT JOIN noondwh.scgoms_cwpicking.warehouse dst_w
 ON pll.id_destination = dst_w.id_warehouse

LEFT JOIN noondwh.scgoms_cwpicking.picking_container_line plc
```

```sql
 USING (id_picklist_line)

LEFT JOIN noondwh.scgoms_cwpicking.picking_container pcr
 ON pcr.id_picking_container = plc.id_picking_container

LEFT JOIN noondwh.scgoms_cwpicking.cluster cl
 ON cl.id_cluster = pll.id_cluster

/* Join top SKU list */
LEFT JOIN top_2500 tt
 ON tt.sku = pll.sku

WHERE
 /* Business date logic (6.5 hour offset) */
 DATE(DATETIME(pj.created_at, 'Asia/Dubai')
      - INTERVAL '6' HOUR
      - INTERVAL '30' MINUTE) >= CURRENT_DATE - 4

 AND pjst.code = 'closed'                                -- Completed jobs
 AND src_w.partner_warehouse_code IN ('AUHID01','AUHID02')

 /* Only skipped picks */
 AND pllst.code = 'skipped'

 /* Only specific skip reasons */
 AND r.code IN ('missing', 'damaged', 'expired')

 /* Ensure no container assigned (true skip) */
 AND plc.id_picking_container_line IS NULL

GROUP BY ALL
)

/* ============================================================================
  FINAL OUTPUT
  - Joins audit events with skipped picks
```

```
  - Matches same location within 24 hours after audit
============================================================================== */
SELECT
 a.wh_code,

 /* Audit date */
 a.updated_date AS audited_date,

 a.location_code,

 /* Auditor info */
 a.username,
 a.email,

 /* Deep vs Forward */
 location_type,

 /* SKU classification */
 sku_type

FROM audit_locs a

/* Join pick events within 24h after audit */
JOIN pick p
 ON a.wh_code = p.wh_code
 AND a.location_code = p.location_code
 AND p.created_at BETWEEN a.updated_at
                      AND a.updated_at + INTERVAL '24' HOUR

GROUP BY ALL
```

```
/* ========================================================================
```

## Fake JLM Identification and Analysis

```
  PURPOSE:
  Identify "Fake JLM" behavior:
  - A picker skips an item (missing/damaged/expired)
  - Then picks the SAME SKU from SAME LOCATION within 4 hours

  This helps detect:
  → False skips
  → Incorrect inventory reporting
  → Potential process gaps or misuse

======================================================================== */

WITH overall AS (

/* ========================================================================
  BASE DATA (LINE-LEVEL PICK ACTIVITY)
  - Each row represents SKU activity at location level
```

```
  - Captures both:
      • skipped items
      • successfully picked items
================================================================================ */
SELECT
 src_w.partner_warehouse_code AS wh_code,    -- Warehouse ID

 /* ------------------------------------------------------------------------
    BUSINESS DATE LOGIC:
    - Convert to Dubai timezone
    - Shift by 6.5 hours (6h + 30m)
    - Defines business day cutoff
 ------------------------------------------------------------------------ */
 DATE(
   DATETIME(pj.created_at, 'Asia/Dubai')
   - INTERVAL '6' HOUR
   - INTERVAL '30' MINUTE
 ) AS date,

 pj.id_user,               -- Picker ID
 pll.location_code,        -- Location of pick
 pll.sku,                  -- SKU
 pll.updated_at,           -- Event timestamp

 /* ------------------------------------------------------------------------
    SKIPPED ITEMS:
    Conditions:
    - Status = skipped
    - Reason = missing / damaged / expired
    - No container assigned (true skip)
 ------------------------------------------------------------------------ */
 COUNT(DISTINCT CASE
   WHEN pllst.code = 'skipped'
     AND r.code IN ('missing', 'damaged', 'expired')
     AND plc.id_picking_container_line IS NULL
   THEN pll.id_picklist_line
```

```
 END) AS skipped_items,

 /* ------------------------------------------------------------------------
    PICKED ITEMS:
    Condition:
    - Container line exists → item successfully picked
 ------------------------------------------------------------------------ */
 COUNT(DISTINCT CASE
   WHEN plc.id_picking_container_line IS NOT NULL
   THEN pll.id_picklist_line
 END) AS picked_items

FROM noondwh.scgoms_cwpicking.picking_job pj

/* Job status */
LEFT JOIN noondwh.scgoms_cwpicking.status pjst
 ON pj.id_status = pjst.id_status

/* Picklist lines (SKU-level detail) */
LEFT JOIN noondwh.scgoms_cwpicking.picklist_line pll
 USING (id_picking_job)

/* Line status */
LEFT JOIN noondwh.scgoms_cwpicking.status pllst
 ON pll.id_status = pllst.id_status

/* Warehouse */
LEFT JOIN noondwh.scgoms_cwpicking.warehouse src_w
 ON pll.id_warehouse = src_w.id_warehouse

/* Skip reason */
LEFT JOIN noondwh.scgoms_cwpicking.reason r
 ON r.id_reason = pll.id_skip_reason

/* Additional joins (context, not directly used in logic) */
LEFT JOIN noondwh.scgoms_cwpicking.warehouse_zone wz
```

```
 ON pll.id_warehouse_zone = wz.id_warehouse_zone

LEFT JOIN noondwh.scgoms_cwpicking.warehouse dst_w
 ON pll.id_destination = dst_w.id_warehouse

LEFT JOIN noondwh.scgoms_cwpicking.picking_container_line plc
 USING (id_picklist_line)

LEFT JOIN noondwh.scgoms_cwpicking.picking_container pcr
 ON pcr.id_picking_container = plc.id_picking_container

LEFT JOIN noondwh.scgoms_cwpicking.cluster cl
 ON cl.id_cluster = pll.id_cluster

/* Transfer joins (enrichment layer) */
LEFT JOIN noondwh.scgoms_cwpicking.transfer_request_line t
 ON t.id_transfer_request_line = pll.id_transfer_request_line
 AND DATE(t.created_at, 'Asia/Dubai') >= CURRENT_DATE - 10

LEFT JOIN noondwh.scgoms_cwpicking.transfer_request tr
 ON tr.id_transfer_request = t.id_transfer_request
 AND DATE(tr.created_at, 'Asia/Dubai') >= CURRENT_DATE - 10

LEFT JOIN noondwh.scgoms_goms.item i
 ON tr.transfer_nr = i.transfer_nr
 AND i.catalog_sku = t.sku
 AND DATE(i.created_at, 'Asia/Dubai') >= CURRENT_DATE - 10

WHERE
 /* Last 7 business days */
 DATE(
   DATETIME(pj.created_at, 'Asia/Dubai')
   - INTERVAL '6' HOUR
   - INTERVAL '30' MINUTE
 ) >= CURRENT_DATE - 7
```

```sql
 AND pjst.code = 'closed'    -- Completed jobs only
 AND src_w.partner_warehouse_code IN ('AUHID01')

GROUP BY ALL
),

/* ================================================================================
  FAKE DETECTION LOGIC
  - Match skipped → later picked (same SKU + location)
  - Within 4 hours window
================================================================================ */
raw_data AS (
SELECT
 a.*,

 /* ------------------------------------------------------------------------
    FAKE FLAG:
    If same SKU & location gets picked after skip within 4 hours
 ------------------------------------------------------------------------ */
 MAX(COALESCE(b.picked_items, 0)) AS fake

FROM overall a

LEFT JOIN overall b
 ON a.wh_code = b.wh_code
 AND a.location_code = b.location_code
 AND a.sku = b.sku

 /* Must happen AFTER skip */
 AND b.updated_at > a.updated_at

 /* Within 4-hour window */
 AND b.updated_at < a.updated_at + INTERVAL '4' HOUR

 /* Same business date */
 AND a.date = b.date
```

```sql
 /* Ensure valid conditions */
 AND b.picked_items > 0     -- Later picked
 AND a.skipped_items > 0    -- Initially skipped

GROUP BY ALL
)

/* ================================================================================
  FINAL OUTPUT (USER LEVEL METRICS)
================================================================================ */
SELECT
 date,

 r.id_user,                  -- Picker ID
 u.username AS email,        -- User email

 /* ------------------------------------------------------------------------
    FAKE JLM COUNT:
    Number of timestamps where fake detected
 ------------------------------------------------------------------------ */
 COUNT(DISTINCT CASE
   WHEN fake > 0 THEN r.updated_at
 END) AS fake_jlm,

 /* Unique locations involved in fake cases */
 COUNT(DISTINCT CASE
   WHEN fake > 0 THEN location_code
 END) AS fake_jlm_locations,

 /* Total skipped items that became fake */
 SUM(CASE
   WHEN fake > 0 THEN skipped_items
 END) AS fake_jlm_skipped_items,

 /* Overall skipped items */
```

```sql
 SUM(skipped_items) AS total_skipped_items,

 /* Total picked items */
 SUM(picked_items) AS picked_items

FROM raw_data r

/* User info */
JOIN noondwh.scgoms_goms.user u
 ON u.id_user = r.id_user

GROUP BY ALL

/* Only show users with fake activity */
HAVING fake_jlm > 0
```

```
/* ================================================================================
```

## Dispatch and Receiving Adherence

```
  PURPOSE:
  Analyze driver task performance by comparing:
  - Actual dispatch & receiving timestamps
  - Planned dispatch & receiving slot timings

  Also calculates:
  → Dispatch delay (in minutes)
  → Receiving delay (in minutes)
  → Utilisation factor (pallet efficiency)

  Note - This not for all the warehouses, there are different tables for different
warehouses and different queries for the same

================================================================================ */

WITH
```

```sql
/* ============================================================================
  DRIVER TASK DATA
  - Contains actual execution data of trips
============================================================================ */
driver_task AS (
SELECT
 id_driver_task,                -- Unique driver task ID
 task_nr,                      -- Task number (business identifier)
 driver_task_created_time,     -- Actual dispatch timestamp
 dst_warehouse,                -- Destination warehouse
 qty_items,                    -- Total items dispatched
 qty_pallets,                  -- Total pallets dispatched
 mark_arrived_timestamp        -- Actual receiving timestamp

FROM noonbinimops.fulfillment.tp_driver_task

WHERE
 dst_warehouse IS NOT NULL              -- Must have warehouse assigned
 AND mark_arrived_timestamp IS NOT NULL -- Only completed deliveries
),

/* ============================================================================
  DISPATCH SLOT MASTER
  - Planned schedule for dispatch & receiving
============================================================================ */
dispatch_slots AS (
SELECT
 trip_key,             -- Unique trip identifier
 ds_code,              -- Warehouse code
 pallet_count,         -- Maximum pallet capacity
 dispatch_time,        -- Planned dispatch time (HH:MM:SS)
 receiving_time        -- Planned receiving time (HH:MM:SS)

FROM noonbinimops.fulfillment.tp_dispatch_slots
```

```sql
GROUP BY 1,2,3,4,5   -- Deduplicate
),

/* ============================================================================
  DISPATCH DELTA CALCULATION
  - Finds closest planned dispatch slot to actual dispatch time
  - Checks:
       same day
       previous day
       next day
  - Picks minimum difference
============================================================================ */
min_diff AS (
SELECT
 dt.task_nr,
 dt.dst_warehouse,

 ds.trip_key,

 ds.pallet_count AS max_pallet_cnt,     -- Planned capacity
 dt.qty_pallets AS pallets_dispatched,  -- Actual pallets

 dt.driver_task_created_time AS actual_dispatch_time,
 dt.mark_arrived_timestamp AS actual_receive_time,

 dt.qty_items AS items_dispatched,

 ds.dispatch_time,
 ds.receiving_time,

 /* ----------------------------------------------------------------------
    DISPATCH DELTA (in minutes)
    - Compare actual vs planned dispatch time
    - Try 3 possibilities:
        same date
        previous date
```

```
       next date
   - Take minimum difference
------------------------------------------------------------------------- */
LEAST(

  /* Same day match */
  ABS(TIMESTAMP_DIFF(
    TIMESTAMP(SAFE.PARSE_DATETIME(
      '%Y-%m-%d %H:%M:%S',
      FORMAT_TIMESTAMP('%Y-%m-%d', dt.driver_task_created_time)
       || ' ' || ds.dispatch_time)),
    TIMESTAMP(dt.driver_task_created_time),
    MINUTE
  )),

  /* Previous day match */
  ABS(TIMESTAMP_DIFF(
    TIMESTAMP(SAFE.PARSE_DATETIME(
      '%Y-%m-%d %H:%M:%S',
      FORMAT_TIMESTAMP('%Y-%m-%d', dt.driver_task_created_time - INTERVAL 1 DAY)
       || ' ' || ds.dispatch_time)),
    TIMESTAMP(dt.driver_task_created_time),
    MINUTE
  )),

  /* Next day match */
  ABS(TIMESTAMP_DIFF(
    TIMESTAMP(SAFE.PARSE_DATETIME(
      '%Y-%m-%d %H:%M:%S',
      FORMAT_TIMESTAMP('%Y-%m-%d', dt.driver_task_created_time + INTERVAL 1 DAY)
       || ' ' || ds.dispatch_time)),
    TIMESTAMP(dt.driver_task_created_time),
    MINUTE
  ))

) AS dispatch_delta
```

```
FROM driver_task dt

/* Match slots by warehouse */
JOIN dispatch_slots ds
 ON ds.ds_code = dt.dst_warehouse
),

/* ============================================================================
  FIND MINIMUM DISPATCH DELTA
  - For each task, pick the closest slot
============================================================================ */
min_time_diff AS (
SELECT
 task_nr,
 dst_warehouse,
 MIN(dispatch_delta) AS min_time_diff
FROM min_diff
GROUP BY 1,2
),

/* ============================================================================
  FILTER BEST MATCHED SLOT
============================================================================ */
t1 AS (
SELECT md.*
FROM min_diff md

JOIN min_time_diff mtd
 ON md.task_nr = mtd.task_nr
AND md.dst_warehouse = mtd.dst_warehouse
AND md.dispatch_delta = mtd.min_time_diff
),

/* ============================================================================
  RECEIVING DELTA CALCULATION
```

```
  - Same logic as dispatch but for receiving time
============================================================================== */
t2 AS (
SELECT
 t1.*,

 /* ----------------------------------------------------------------------------
    RECEIVING DELTA (in minutes)
 ---------------------------------------------------------------------------- */
 LEAST(

   /* Same day */
   ABS(TIMESTAMP_DIFF(
     TIMESTAMP(SAFE.PARSE_DATETIME(
       '%Y-%m-%d %H:%M:%S',
       FORMAT_TIMESTAMP('%Y-%m-%d', t1.actual_receive_time)
         || ' ' || t1.receiving_time)),
     TIMESTAMP(t1.actual_receive_time),
     MINUTE
   )),

   /* Previous day */
   ABS(TIMESTAMP_DIFF(
     TIMESTAMP(SAFE.PARSE_DATETIME(
       '%Y-%m-%d %H:%M:%S',
       FORMAT_TIMESTAMP('%Y-%m-%d', t1.actual_receive_time - INTERVAL 1 DAY)
         || ' ' || t1.receiving_time)),
     TIMESTAMP(t1.actual_receive_time),
     MINUTE
   )),

   /* Next day */
   ABS(TIMESTAMP_DIFF(
     TIMESTAMP(SAFE.PARSE_DATETIME(
       '%Y-%m-%d %H:%M:%S',
       FORMAT_TIMESTAMP('%Y-%m-%d', t1.actual_receive_time + INTERVAL 1 DAY)
```

```
        || ' ' || t1.receiving_time)),
    TIMESTAMP(t1.actual_receive_time),
    MINUTE
  ))

 ) AS receiving_delta

FROM t1
),

/* ============================================================================
  UTILISATION FACTOR
  - Measures pallet efficiency
  - Formula:
      (actual pallets / max capacity) * 100
============================================================================ */
utilisation_factor AS (
SELECT
 t2.*,

 ROUND(
   100 * SAFE_DIVIDE(
     CAST(t2.pallets_dispatched AS FLOAT64),
     CAST(t2.max_pallet_cnt AS FLOAT64)
   ),
   2
 ) AS utilisation_factor

FROM t2
)

/* ============================================================================
  FINAL OUTPUT
============================================================================ */
SELECT *
FROM utilisation_factor
```

```
-- Optional filter:
-- WHERE task_nr = 'DT3045469089467A';
```

```
/*
═══════════════════════════════════════════════════════════════════
```

## STAGING USER PRODUCTIVITY — warehouse × date × staged_user

```
(Validation of Logic Pending)
 ──────────────────────────────────────────────────────────────────
 SOURCE   : noondwh.scgoms_goms       (audit_log, htl, shipment)
            noondwh.scgoms_cwpicking  (picking_container, picklist_line)
 GRAIN    : one row per source_warehouse × biz_date × staged_user
 SCOPE    : UAE (1), KSA (2,5), Egypt (3) — central warehouses only
            multi_store_picking jobs only
 BIZ DATE : audit_log scan time localised per country then - 6h
              UAE  → Asia/Dubai   - 6h
              KSA  → Asia/Riyadh  - 6h
              Egypt→ Africa/Cairo - 6h
 TIME LOGIC (scan-only, no job proxy):
   - Each audit_log row (action=12, to='staged') = one tote scan by a user
   - LEAD() gets next scan for same user × warehouse × biz_date
   - Gap ≤ 600s  → working time  (productive scanning)
```

```
   - Gap > 600s  → idle          (excluded from working time)
   - active_time  = MAX(scan) - MIN(scan) per user × wh × day
   - working_time = SUM of sub-600s inter-scan gaps only
 WINDOW   : last 7 completed business days (today excluded)
═══════════════════════════════════════════════════════════════
*/

WITH

-- ── TIMEZONE MAP ──────────────────────────────────────────────────────
tz AS (
 SELECT id_country, tz_name FROM UNNEST([
   STRUCT(1 AS id_country, 'Asia/Dubai'   AS tz_name),
   STRUCT(2 AS id_country, 'Asia/Riyadh'  AS tz_name),
   STRUCT(5 AS id_country, 'Asia/Riyadh'  AS tz_name),
   STRUCT(3 AS id_country, 'Africa/Cairo' AS tz_name)
 ])
),

-- ── STEP 1: RAW STAGING SCAN EVENTS ──────────────────────────────────
-- One row per user × tote scan (audit_log staged event)
-- Items per tote aggregated here (cwpicking grain) to avoid fan-out downstream
-- Join chain: audit_log → htl → shipment → picking_container → picklist_line
raw_scans AS (
 SELECT
   u.username                                                    AS staged_user,
   al.id_user,
   src_w.partner_warehouse_code                                  AS
source_warehouse,
   src_w.id_country,
   c.tz_name,
   cwpc.container_barcode                                        AS awb_nr,
   -- Localise scan time to country tz
   DATETIME(al.created_at, c.tz_name)                           AS
scan_at_local,
   -- Items in this tote: only non-recreated picklist lines (skip-and-recreate guard)
```

```
    COUNT(DISTINCT CASE WHEN pll.id_old_picklist_line IS NULL
                        THEN pll.id_picklist_line END)                  AS
items_in_tote
  FROM `noondwh.scgoms_goms.audit_log` al
  JOIN `noondwh.scgoms_goms.hub_transfer_line` htl
    ON  htl.id_hub_transfer_line             = al.action_key
    AND htl.type                             = 'forward_nim'
    AND htl.line_type                        = 'shipment'
  JOIN `noondwh.scgoms_goms.shipment` sh
    ON  sh.id_shipment = htl.line_id
    AND sh.created_at >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 8 DAY)
  -- Bridge goms → cwpicking via awb_nr = container_barcode
  JOIN `noondwh.scgoms_cwpicking.picking_container` cwpc
    ON cwpc.container_barcode = sh.awb_nr
  JOIN `noondwh.scgoms_cwpicking.picking_job` cwpj
    ON  cwpj.id_picking_job = cwpc.id_picking_job
    AND cwpj.job_type       = 'multi_store_picking'
  JOIN `noondwh.scgoms_cwpicking.picking_container_line` pcl
    ON pcl.id_picking_container = cwpc.id_picking_container
  JOIN `noondwh.scgoms_cwpicking.picklist_line` pll
    ON pll.id_picklist_line = pcl.id_picklist_line
  JOIN `noondwh.scgoms_cwpicking.warehouse` src_w
    ON  src_w.id_warehouse = cwpj.id_warehouse
    AND src_w.type         = 'central'
  JOIN tz c
    ON c.id_country = src_w.id_country
  -- Resolve staging user from goms.user (staging is a goms operation)
  LEFT JOIN `noondwh.scgoms_goms.user` u
    ON u.id_user = al.id_user
  WHERE al.id_audit_action                   = 12
    AND JSON_EXTRACT_SCALAR(al.misc, '$.to') = 'staged'
    AND al.created_at >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
    AND DATE(DATETIME(al.created_at, c.tz_name) - INTERVAL 6 HOUR)
        < CURRENT_DATE('Asia/Dubai')
  GROUP BY 1,2,3,4,5,6,7
),
```

```sql
-- ── STEP 2: ADD BUSINESS DATE + LEAD() FOR NEXT SCAN ──────────────────────
-- LEAD() partitioned by user × warehouse × biz_date
-- ensures gaps don't bleed across warehouses or days
scan_timeline AS (
 SELECT
   staged_user,
   id_user,
   source_warehouse,
   id_country,
   tz_name,
   awb_nr,
   items_in_tote,
   scan_at_local,
   -- Business date: -6h from local time aligns shift to 06:00 start
   DATE(scan_at_local - INTERVAL 6 HOUR)                                AS biz_date,
   LEAD(scan_at_local) OVER (
     PARTITION BY
       id_user,
       source_warehouse,
       DATE(scan_at_local - INTERVAL 6 HOUR)
     ORDER BY scan_at_local
   )                                                                     AS
next_scan_local
 FROM raw_scans
),

-- ── STEP 3: CLASSIFY INTER-SCAN GAPS ──────────────────────────────────────
-- ≤ 600s (10 min) → working gap  (counted toward working time)
-- > 600s          → idle         (excluded; break, waiting, system delay)
-- NULL            → last scan of day, no gap to measure
scan_gaps AS (
 SELECT
   staged_user,
   id_user,
   source_warehouse,
```

```
    biz_date,
    awb_nr,
    items_in_tote,
    scan_at_local,
    CASE
      WHEN next_scan_local IS NULL THEN NULL
      WHEN TIMESTAMP_DIFF(next_scan_local, scan_at_local, SECOND) <= 600
        THEN TIMESTAMP_DIFF(next_scan_local, scan_at_local, SECOND)
      ELSE NULL
    END                                                              AS
working_gap_seconds
  FROM scan_timeline
)

-- ── STEP 4: AGGREGATE per warehouse × date × user ──────────────────────
SELECT
  source_warehouse,
  biz_date,
  COALESCE(staged_user, CAST(id_user AS STRING))                     AS staged_user,

  -- Volume
  COUNT(DISTINCT awb_nr)                                             AS
totes_staged,
  SUM(items_in_tote)                                                 AS qty_staged,
  ROUND(SUM(items_in_tote) / NULLIF(COUNT(DISTINCT awb_nr), 0), 1)   AS
avg_items_per_tote,

  -- Scan window timestamps
  MIN(scan_at_local)                                                 AS first_scan,
  MAX(scan_at_local)                                                 AS last_scan,

  -- Active time: first scan → last scan (includes idle gaps), in minutes
  ROUND(
    TIMESTAMP_DIFF(MAX(scan_at_local), MIN(scan_at_local), SECOND)
    / 60.0, 1
```

```
 )                                                                    AS
active_time_mins,

 -- Working time: sum of sub-10-min inter-scan gaps only, in minutes
 ROUND(SUM(COALESCE(working_gap_seconds, 0)) / 60.0, 1)              AS
working_time_mins,

 -- Utilisation: working / active (how much of session was productive)
 ROUND(
   100.0 * SUM(COALESCE(working_gap_seconds, 0))
   / NULLIF(
       TIMESTAMP_DIFF(MAX(scan_at_local), MIN(scan_at_local), SECOND)
     , 0)
 , 1)                                                                 AS
utilisation_pct,

 -- Throughput metrics
 ROUND(
   SUM(items_in_tote)
   / NULLIF(SUM(COALESCE(working_gap_seconds, 0)) / 3600.0, 0)
 , 1)                                                                 AS
items_per_working_hour,
 ROUND(
   COUNT(DISTINCT awb_nr)
   / NULLIF(SUM(COALESCE(working_gap_seconds, 0)) / 3600.0, 0)
 , 1)                                                                 AS
totes_per_working_hour

FROM scan_gaps
GROUP BY 1, 2, 3
ORDER BY biz_date DESC, source_warehouse, qty_staged DESC
```

# Barcode History Dcss

### Easy Guide

**Purpose:** Builds a barcode movement history so you can see where an item moved from, where it moved to, and who processed it.
**Key aliases / tables:** tl = transaction_line, j = job, p = process, pt = process_type, w = warehouse, i = item, src_l = location, src_z = zone
**Main fields / variables:** process_type, pbarcode_canonical, source, source_code, src_zone, destination, destination_code, dst_zone, created_at_ts, created_at_date

## Barcode History

```sql
SELECT DISTINCT
  p.process_nr,
  pt.code AS process_type,
  i.barcode AS pbarcode_canonical,
  i.id_partner,
  w.partner_warehouse_code,
  tl.src_key AS source,
  CASE
    WHEN tl.src_key = 'location' THEN src_l.location_code
    ELSE src_jc.barcode
  END AS source_code,
  src_z.zone_code AS src_zone,
  tl.dst_key AS destination,
  CASE
    WHEN tl.dst_key = 'location' THEN dst_l.location_code
    ELSE dst_jc.barcode
  END AS destination_code,
  dst_z.zone_code AS dst_zone,
  tl.qty,
  u.username,
  u.email,
  datetime(tl.created_at, 'Asia/Dubai') as created_at_ts,
  date(tl.created_at, 'Asia/Dubai') as created_at_date,
FROM
  noondwh.mxdcss_dcss.transaction_line tl
LEFT JOIN
  noondwh.mxdcss_dcss.job j
ON
  tl.id_job = j.id_job
LEFT JOIN
  noondwh.mxdcss_dcss.process p
ON
  j.id_process = p.id_process
```

```
LEFT JOIN
  noondwh.mxdcss_dcss.process_type pt
ON
  p.id_process_type = pt.id_process_type
LEFT JOIN
  noondwh.mxdcss_dcss.warehouse w
ON
  w.id_warehouse = j.id_warehouse
LEFT JOIN
  noondwh.mxdcss_dcss.item i
ON
  tl.id_item = i.id_item
LEFT JOIN
  noondwh.mxdcss_dcss.location src_l
ON
  tl.src_key = 'location' AND CAST(tl.src_value AS STRING) =
CAST(src_l.id_location AS STRING)
LEFT JOIN
  noondwh.mxdcss_dcss.zone src_z
ON
  src_l.id_zone = src_z.id_zone
LEFT JOIN
  noondwh.mxdcss_dcss.job_container src_jc
ON
  tl.src_key = 'job_container' AND CAST(tl.src_value AS STRING) =
CAST(src_jc.id_job_container AS STRING)
LEFT JOIN
  noondwh.mxdcss_dcss.location dst_l
ON
  tl.dst_key = 'location' AND CAST(tl.dst_value AS STRING) =
CAST(dst_l.id_location AS STRING)
LEFT JOIN
  noondwh.mxdcss_dcss.zone dst_z
ON
  dst_l.id_zone = dst_z.id_zone
LEFT JOIN
  noondwh.mxdcss_dcss.job_container dst_jc
ON
  tl.dst_key = 'job_container' AND CAST(tl.dst_value AS STRING) =
CAST(dst_jc.id_job_container AS STRING)
LEFT JOIN
  noondwh.mxdcss_dcss.user u
ON
  j.id_user = u.id_user
```

# PnL

### Easy Guide

**Purpose:** Summarizes PnL-related inventory movements by day, movement type, subtype, month, and week.
**Key aliases / tables:** txlogs
**Main fields / variables:** week_no
**Filter hints:** BETWEEN '2026-01-01' AND CURRENT_DATE() -1

## PnL UAE 2026

```SQL
       SELECT
            transaction_date,
            financial_movement_type,
            subtype,
            YEAR_MONTH,
            EXTRACT(
                WEEK
                FROM
                    transaction_date
            ) AS week_no,
            SUM(qty) qty,
            SUM(qty * allocated_unit_cost_lcy) value,
        FROM
            noonbinimops.fulfillment.txlogs
        WHERE
            financial_movement_type IN ("adjustments", "other", "transit",
"adjustment")
            AND id_partner = 9411
            AND transaction_date BETWEEN '2026-01-01' AND CURRENT_DATE()  -1
        GROUP BY
            1,
            2,
            3,
            4,
            5
```

# Adj DS

### Easy Guide

**Purpose:** Calculates dark-store inventory adjustments, enriches them with product details, and converts unit movements into value impact.
**CTEs:** parsed, t1, sku_info, p1, p2, t22
**Key aliases / tables:** stj = stock_take_job, tx = stock_tx, sl = stock_line, w = warehouse, p = product, pft = product_fulltype, f = family, pen = product_en
**Main fields / variables:** family, title_en, adj_type, warehouse, put_date, job_nr, put_user, price, month, wh_code
**Filter hints:** posted_date >= '2026-01-01'

```
WITH
  parsed AS (
SELECT sl.*,
stj.id_stock_take_job,
stj.id_user,


    f.name_fr AS family,
    pen.title AS title_en


  FROM noondwh.mxfulfillment_norms.stock_take_job stj
  JOIN noondwh.wms.stock_tx tx ON (tx.tx_nr = stj.stock_take_job_nr)
  JOIN noonbicedata.wms_v3.stock_line sl USING(id_stock_tx)
      left join noondwh.wms.warehouse w
    on w.code = sl.warehouse_pick

    LEFT JOIN
    noondwh.instant_spanner.product p
  ON
    (p.sku = sl.sku)
  LEFT JOIN
    noondwh.wecat_md.product_fulltype pft
  USING
    (id_product_fulltype)
  LEFT JOIN
    noondwh.wecat_md.family f
  USING
    (id_family)
  LEFT JOIN
    noondwh.instant_spanner.product_en pen
  ON
    (p.sku = pen.sku)
  where tx.tx_intent = 'inventory_adjustment'
  and tx.posted_date >= '2026-01-01'
     and w.id_country = 2
),
```

```
 t1 AS (
 SELECT
   CASE
     WHEN area_pick = 'INVADJ' AND area_put = 'STOCK' THEN 'Positive'
     WHEN area_put = 'INVADJ'
   AND area_pick = 'STOCK' THEN 'Negative'
 END
   AS adj_type,
   p.*,
   warehouse_pick as warehouse,
   -- DATE(TIMESTAMP_ADD(updated_at, INTERVAL 4 HOUR)) AS put_date,
   id_stock_take_job AS job_nr,
   id_user AS put_user,
   posted_date as put_date,
   -- stj.id_warehouse
 FROM
   parsed p  ),
 sku_info AS (
 SELECT
   pbarcode_canonical,
   id_partner,
   category,
   brand_code,
   Product_subtype,
   product_fulltype_code
 FROM
   noonbilogmis.reporting.DS_Stock_Detail
 GROUP BY
   ALL),
    p1 as (
 select pbarcode_canonical,
  id_partner,
  avg(price) as price
 from
  noonbinimops.fulfillment.master_price_v1
  group by all
)
 , p2 as (
 select pbarcode_canonical,
 --  id_partner,
  avg(price) as price
 from
  noonbinimops.fulfillment.master_price_v1
  group by all
```

```
)


,  t22 AS (
  SELECT
    adj_type,
    put_date,
    put_user,
    FORMAT_DATE('%B', put_date) AS month,
    b.id_partner,
    b.sku,
    title_en,
    family,
    si.category,
    si.brand_code,
    si.Product_subtype,
    si.product_fulltype_code,
    b.pbarcode_canonical,
    b.barcode,
    warehouse AS wh_code,
    area AS warehouse_name,
    job_nr,
    doc_nr,
    SUM(qty) AS units,
  FROM
    t1 b
  LEFT JOIN
    noondwh.mxfulfillment_norms.warehouse wh
  ON
    (wh.partner_warehouse_code = b.warehouse)




  LEFT JOIN
    sku_info si
  ON
    si.id_partner = b.id_partner
    AND si.pbarcode_canonical = b.pbarcode_canonical
     LEFT JOIN
     noonbiinstnt.darkstore.noon_catalog_with_details p
  ON
    p.id_partner = b.id_partner
    AND p.pbarcode_canonical = b.pbarcode_canonical
```

```
  GROUP BY
    ALL
     ),
     t2 as (
      select b.*,
          coalesce(p1.price, p2.price) AS value_per_item,
coalesce(p1.price, p2.price) * units AS total_variance_value
      from t22 b
      left join p1
on b.pbarcode_canonical = p1.pbarcode_canonical
and b.id_partner = p1.id_partner
left join p2
on b.pbarcode_canonical = p2.pbarcode_canonical
     )
  ,t3 AS (
  SELECT
    put_date,
    put_user,
    FORMAT_DATE('%B', put_date) AS month,
    id_partner,
    sku,
    title_en,
    family,
    category,
    brand_code,
    Product_subtype,
    product_fulltype_code,
    pbarcode_canonical,
    barcode,
    wh_code,
    warehouse_name,
    job_nr,
    doc_nr,
    SUM(CASE
        WHEN adj_type = 'Positive' THEN total_variance_value
    END
      ) AS positive_variance_value,
    SUM(CASE
        WHEN adj_type = 'Negative' THEN total_variance_value
    END
      ) AS negative_variance_value,
    SUM(CASE
        WHEN adj_type = 'Positive' THEN units
    END
```

```
    ) AS positive_variance_units,
   SUM(CASE
      WHEN adj_type = 'Negative' THEN units
   END
    ) AS negative_variance_units
  FROM
   t2
  GROUP BY
   ALL ),
  t4 AS (
  SELECT
   *,
   EXTRACT(month
   FROM
    put_date) AS month_no,
   IFNULL(positive_variance_value,0) - IFNULL(negative_variance_value,0) AS net_value,
   IFNULL(positive_variance_units,0) - IFNULL(negative_variance_units,0) AS net_units
  FROM
   t3 )
   , t5 as (
SELECT
  *,
  CASE
   WHEN net_units > 0 THEN 'positive_adj'
   WHEN net_units < 0 THEN 'negative_adj'
   WHEN net_units = 0 THEN 'zero'
END
  AS variance_category
FROM
  t4
WHERE
  wh_code NOT LIKE 'CAIDS%'
  and put_date >= '2026-01-01'
   )
   select
   put_date,
CAST(put_user AS STRING) AS put_user,
cast(month as string) as month,
id_partner,
sku,
title_en,
family,
category,
brand_code,
```

```
Product_subtype,
product_fulltype_code,
pbarcode_canonical,
barcode,
wh_code,
warehouse_name,
positive_variance_units,
negative_variance_units,
cast(month_no as int64) as month_no,
net_units,
variance_category,
job_nr,
doc_nr,
CAST(negative_variance_value AS NUMERIC) AS negative_variance_value,
CAST(positive_variance_value AS NUMERIC) AS positive_variance_value,
CAST(net_value AS NUMERIC) AS net_value
   from t5
```

# Non-Live Warehouse Inventory

### Easy Guide

**Purpose:** Builds a non-live inventory view with quantities and estimated values using fallback pricing.
**CTEs:** p1, p2, wms_base, wms_1, dcss_base, dcss_1
**Key aliases / tables:** s = stock, l = location, w = warehouse, z = zone, a = area, it = item, id = item_detail, b = wms_base
**Main fields / variables:** price, warehouse, wms_qty, id_partner, wms_value, pbarcode_canonical, dcss_qty, dcss_value, total_inventory, total_inventory_value

## UAE

```
WITH
 p1 AS (
   SELECT
     pbarcode_canonical,
     id_partner,
     avg(price) AS price
   FROM
     noonbinimops.fulfillment.master_price_v1
   GROUP BY ALL
 ),
 p2 AS (
   SELECT
     pbarcode_canonical,
     --  id_partner,
     avg(price) AS price
   FROM
     noonbinimops.fulfillment.master_price_v1
   GROUP BY ALL
 )

,  wms_base AS (
   SELECT
     w.code AS warehouse,
id.pbarcode_canonical,
```

```
id.id_partner,
     SUM(s.qty) AS wms_qty
   FROM noondwh.wms.stock s
   LEFT JOIN noondwh.wms.location l
     USING (id_location)
   LEFT JOIN noondwh.wms.warehouse w
     USING (id_warehouse)
   LEFT JOIN noondwh.wms.zone z
     USING (id_zone)
   LEFT JOIN noondwh.wms.area a
     USING (id_area)
   LEFT JOIN noondwh.wms.item it
     USING (id_item)
   LEFT JOIN noonbicedata.wms_v2.item_detail id
     USING (id_item)
   WHERE
     (w.code LIKE '%AUHID%' OR w.code LIKE '%DXBNIM%')
     AND a.code = 'STOCK'
     AND s.qty > 0
   GROUP BY all
 ),
wms_1 as(
 select warehouse,
 case when COALESCE(b.id_partner) in (9411,9311) then "94311" else "non_94311" end  as
id_partner,
 sum(wms_qty) as wms_qty,
 sum(coalesce(p1.price, p2.price) * wms_qty) as wms_value
 from wms_base b
 LEFT JOIN p1
 ON
   b.pbarcode_canonical = p1.pbarcode_canonical
   AND b.id_partner = p1.id_partner
LEFT JOIN p2
 ON b.pbarcode_canonical = p2.pbarcode_canonical
 group by all
)
```

```
,
 dcss_base AS (
   SELECT
     w.partner_warehouse_code AS warehouse,
     i.barcode as pbarcode_canonical,
i.id_partner,
     SUM(li.qty - li.qty_reserved) AS dcss_qty
   FROM noondwh.mxdcss_dcss.location_item li
   JOIN noondwh.mxdcss_dcss.item i
     USING (id_item)
   LEFT JOIN noondwh.mxdcss_dcss.location l
     USING (id_location)
   LEFT JOIN noondwh.mxdcss_dcss.zone z
     USING (id_zone)
   LEFT JOIN noondwh.mxdcss_dcss.warehouse w
     ON w.id_warehouse = z.id_warehouse
   WHERE
     id_country = 1
     AND is_saleable = 0
   GROUP BY all
 )
, dcss_1 as (
 select warehouse,
 case when COALESCE(b.id_partner) in (9411,9311) then "94311" else "non_94311" end  as
id_partner,
 sum(dcss_qty) as dcss_qty,
 sum(coalesce(p1.price, p2.price) * dcss_qty) as dcss_value
   from dcss_base b
 LEFT JOIN p1
 ON
   b.pbarcode_canonical = p1.pbarcode_canonical
   AND b.id_partner = p1.id_partner
LEFT JOIN p2
 ON b.pbarcode_canonical = p2.pbarcode_canonical
group by all
),
```

```
final as (SELECT distinct
 COALESCE(w.id_partner, d.id_partner) AS id_partner,
 COALESCE(w.warehouse, d.warehouse) AS warehouse,
 sum(IFNULL(d.dcss_qty, 0)) AS dcss_qty,
 sum(IFNULL(w.wms_qty, 0)) AS wms_qty,
 sum(IFNULL(d.dcss_qty, 0)) + sum(IFNULL(w.wms_qty, 0)) AS total_inventory,
 sum(IFNULL(d.dcss_value, 0)) AS dcss_value,
 sum(IFNULL(w.wms_value, 0)) AS wms_value,
 sum(IFNULL(d.dcss_value, 0)) + sum(IFNULL(w.wms_value, 0)) AS total_inventory_value
FROM wms_1 w
FULL OUTER JOIN dcss_1 d
 ON w.warehouse = d.warehouse and d.id_partner=w.id_partner
group by all
ORDER BY warehouse
)


select * from final
```

## KSA

```
WITH
 p1 AS (
   SELECT
     pbarcode_canonical,
     id_partner,
     avg(price) AS price
   FROM
      noonbinimops.fulfillment.master_price_ksa_v1
   GROUP BY ALL
 ),
 p2 AS (
   SELECT
```

```
    pbarcode_canonical,
    --  id_partner,
    avg(price) AS price
  FROM
     noonbinimops.fulfillment.master_price_ksa_v1
  GROUP BY ALL
)

,  wms_base AS (
  SELECT
    w.code AS warehouse,
id.pbarcode_canonical,
id.id_partner,
    SUM(s.qty) AS wms_qty
  FROM noondwh.wms.stock s
  LEFT JOIN noondwh.wms.location l
    USING (id_location)
  LEFT JOIN noondwh.wms.warehouse w
    USING (id_warehouse)
  LEFT JOIN noondwh.wms.zone z
    USING (id_zone)
  LEFT JOIN noondwh.wms.area a
    USING (id_area)
  LEFT JOIN noondwh.wms.item it
    USING (id_item)
  LEFT JOIN noonbicedata.wms_v2.item_detail id
    USING (id_item)
  WHERE
  w.code LIKE '%RUHNIM%'
    AND a.code = 'STOCK'
    AND s.qty > 0
  GROUP BY all
),
wms_1 as (
 select warehouse,
```

```
case when COALESCE(b.id_partner) in (9411,9311) then "94311" else "non_94311" end  as
id_partner,
 sum(wms_qty) as wms_qty,
 sum(coalesce(p1.price, p2.price) * wms_qty) as wms_value
 from wms_base b
 LEFT JOIN p1
 ON
   b.pbarcode_canonical = p1.pbarcode_canonical
   AND b.id_partner = p1.id_partner
LEFT JOIN p2
 ON b.pbarcode_canonical = p2.pbarcode_canonical
 group by all
)
,
 dcss_base AS (
   SELECT
     w.partner_warehouse_code AS warehouse,
     i.barcode as pbarcode_canonical,
i.id_partner,
     SUM(li.qty - li.qty_reserved) AS dcss_qty
   FROM noondwh.mxdcss_dcss.location_item li
   JOIN noondwh.mxdcss_dcss.item i
     USING (id_item)
   LEFT JOIN noondwh.mxdcss_dcss.location l
     USING (id_location)
   LEFT JOIN noondwh.mxdcss_dcss.zone z
     USING (id_zone)
   LEFT JOIN noondwh.mxdcss_dcss.warehouse w
     ON w.id_warehouse = z.id_warehouse
   WHERE
     id_country = 2
     AND z.zone_code NOT IN(
                   'STOCK-N-MGW-NL2',
                   'STOCK-N-MGW-NL',
                   'STOCK-N-RT2-NL',
                   'STOCK-N-RT-NL'
```

```
                )
      AND is_saleable = 0
    GROUP BY all
 )
, dcss_1 as (
 select warehouse,
case when COALESCE(b.id_partner) in (9411,9311) then "94311" else "non_94311" end  as
id_partner,
 sum(dcss_qty) as dcss_qty,
 sum(coalesce(p1.price, p2.price) * dcss_qty) as dcss_value
   from dcss_base b
 LEFT JOIN p1
 ON
   b.pbarcode_canonical = p1.pbarcode_canonical
   AND b.id_partner = p1.id_partner
LEFT JOIN p2
 ON b.pbarcode_canonical = p2.pbarcode_canonical
group by all
),

final as (SELECT distinct
 COALESCE(w.id_partner, d.id_partner) AS id_partner,
 COALESCE(w.warehouse, d.warehouse) AS warehouse,
 sum(IFNULL(d.dcss_qty, 0)) AS dcss_qty,
 sum(IFNULL(w.wms_qty, 0)) AS wms_qty,
 sum(IFNULL(d.dcss_qty, 0)) + sum(IFNULL(w.wms_qty, 0)) AS total_inventory,
 sum(IFNULL(d.dcss_value, 0)) AS dcss_value,
 sum(IFNULL(w.wms_value, 0)) AS wms_value,
 sum(IFNULL(d.dcss_value, 0)) + sum(IFNULL(w.wms_value, 0)) AS total_inventory_value
FROM wms_1 w
FULL OUTER JOIN dcss_1 d
 ON w.warehouse = d.warehouse and d.id_partner=w.id_partner
group by all
ORDER BY warehouse
)
```

```sql
select * from final
```

# ID Stock

## Easy Guide

**Purpose:** Creates an ID/DCSS stock snapshot with gross, reserved, and net quantities at location level.
**Key aliases / tables:** li = location_item, i = item, l = location, z = zone, w = warehouse
**Main fields / variables:** snapshot_date, pbarcode_canonical, gross_qty, net_qty

```
SELECT
  current_date() as snapshot_date,
  w.id_country,
  w.partner_warehouse_code,
  z.zone_code,
  l.location_code,
  i.barcode AS pbarcode_canonical,
  i.barcode_level_code,
  i.id_partner,
  i.expiry_date,
  li.qty AS gross_qty,
  li.qty_reserved,
  li.qty - li.qty_reserved AS net_qty,
  is_saleable,
  l.is_virtual
FROM
  noondwh.mxdcss_dcss.location_item li
JOIN
  noondwh.mxdcss_dcss.item i
USING
  (id_item)
LEFT JOIN
  noondwh.mxdcss_dcss.location l
USING
  (id_location)
LEFT JOIN
  noondwh.mxdcss_dcss.zone z
USING
  (id_zone)
LEFT JOIN
  noondwh.mxdcss_dcss.warehouse w
ON
  (w.id_warehouse = z.id_warehouse)
```

# WMS Stock

## Easy Guide

**Purpose:** Creates a current WMS stock snapshot by warehouse, zone, location, and barcode.
**Key aliases / tables:** s = stock, l = location, w = warehouse, z = zone, a = area, it = item, id = item_detail
**Main fields / variables:** warehouse, zone, location, qty, snap

```
SELECT DISTINCT
  id.id_partner,
  id.wms_barcode,
  id.pbarcode_canonical,
  w.code AS warehouse,
  z.code AS zone,
  l.code AS location,
  SUM(s.qty) AS qty,
  current_date() as snap
FROM noondwh.wms.stock s
LEFT JOIN noondwh.wms.location l
  USING (id_location)
LEFT JOIN noondwh.wms.warehouse w
  USING (id_warehouse)
LEFT JOIN noondwh.wms.zone z
  USING (id_zone)
LEFT JOIN noondwh.wms.area a
  USING (id_area)
LEFT JOIN noondwh.wms.item it
  USING (id_item)
LEFT JOIN noonbicedata.wms_v2.item_detail id
  USING (id_item)
WHERE
  1 = 1
  AND w.code LIKE ANY('%AUHID%', '%DXBNIM%')
  AND a.code = 'STOCK'
  AND s.qty > 0
GROUP BY ALL
```

# Store <> Putaway Pendency <> Buckets

### Easy Guide

**Purpose:** Buckets pending putaway inventory into store/temperature groups so the backlog is easy to segment.
**Key aliases / tables:** b = box, b_st = status, bi = box_item, pujl = putaway_job_line, dst_w = warehouse, src_w = warehouse
**Main fields / variables:** country, partner_warehouse_code, area, storage_condition, pending_qty_between_0_3hr, pending_qty_between_3_6hr, pending_qty_above_6hr, total_pending_qty
**Filter hints:** BETWEEN 0 AND 3 THEN 1; created_at >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY)

```
SELECT
  CASE WHEN dst_w.id_country = 1 THEN 'ae' ELSE 'sa' END AS country,
  dst_w.partner_warehouse_code AS partner_warehouse_code,
  dst_w.area AS area,

  CASE
    WHEN src_w.partner_warehouse_code IN (
      'DXBID01','DXBID02','AUHID01','AUHID02',
      'JEDID01','JEDID02','RUHID01','RUHID02'
    ) THEN 'pending_putaway_ambient'

    WHEN src_w.partner_warehouse_code IN (
      'AUHFKZ01','AUHFKZ02','AUHFKZ03','RUHID05','JEDID05'
    ) THEN 'pending_putaway_frozen'

    WHEN src_w.partner_warehouse_code IN (
      'DXBFNV01','RUHID07','JEDID07'
    ) THEN 'pending_putaway_fnv'

    WHEN src_w.partner_warehouse_code IN (
      'DXBID03','DXBID04','AUHID03','AUHID04',
      'RUHID03','RUHID04','JEDID03','JEDID04',
      'DXBID07','RUHID06','JEDID06'
    ) THEN 'pending_putaway_chiller'

    ELSE 'unknown'
  END AS storage_condition,

  SUM(CASE
    WHEN TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), b.created_at, HOUR) BETWEEN 0
AND 3 THEN 1
    ELSE 0
  END) AS pending_qty_between_0_3hr,

  SUM(CASE
    WHEN TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), b.created_at, HOUR) > 3
     AND TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), b.created_at, HOUR) <= 6 THEN 1
```

```
    ELSE 0
  END) AS pending_qty_between_3_6hr,

  SUM(CASE
    WHEN TIMESTAMP_DIFF(CURRENT_TIMESTAMP(), b.created_at, HOUR) > 6 THEN 1
    ELSE 0
  END) AS pending_qty_above_6hr,

  COUNT(*) AS total_pending_qty

FROM noondwh.mxfulfillment_norms.box b
JOIN noondwh.mxfulfillment_norms.status b_st USING(id_status)
LEFT JOIN noondwh.mxfulfillment_norms.box_item bi USING(id_box)
LEFT JOIN noondwh.mxfulfillment_norms.putaway_job_line pujl USING(id_box_item)
LEFT JOIN noondwh.mxfulfillment_norms.warehouse dst_w ON b.id_warehouse =
dst_w.id_warehouse
LEFT JOIN noondwh.mxfulfillment_norms.warehouse src_w ON b.id_src_warehouse =
src_w.id_warehouse

WHERE b.created_at >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY)
  AND b_st.code IN ('pending', 'putting')
  AND (pujl.id_putaway_job_line IS NULL OR pujl.id_status = 1)
  AND src_w.partner_warehouse_code IN (
    'DXBID01','DXBID02','AUHID01','AUHID02',
    'JEDID01','JEDID02','RUHID01','RUHID02',
    'AUHFKZ01','AUHFKZ02','AUHFKZ03','RUHID05','JEDID05',
    'DXBFNV01','RUHID07','JEDID07',
    'DXBID03','DXBID04','AUHID03','AUHID04',
    'RUHID03','RUHID04','JEDID03','JEDID04',
    'DXBID07','RUHID06','JEDID06'
  )

GROUP BY 1,2,3,4
```

# Putaway Pendency at DS - Ageging - Raw Level

## Easy Guide

**Purpose:** Builds dark-store putaway aging so backlog can be monitored by date, warehouse, and age bucket.
**CTEs:** base
**Key aliases / tables:** b = box, b_st = status, btm = box_tote_mapping, bi = box_item, pujl = putaway_job_line, rs = reason, dst_w = warehouse, src_w = warehouse
**Main fields / variables:** awb_nr, date, created_at, storage_condition, exdoc_nr, pbarcode_canonical, src_wh_code, ds_code, ds_name, country_code

```
WITH
 base AS (
 SELECT
   b.awb_nr AS awb_nr,
   CASE
     WHEN ctry.code = 'ae' THEN DATE(b.created_at, 'Asia/Dubai')
     WHEN ctry.code = 'sa' THEN DATE(b.created_at, 'Asia/Riyadh')
     ELSE DATE(b.created_at)
 END
   AS date,
   CASE
     WHEN ctry.code = 'ae' THEN DATETIME(b.created_at, 'Asia/Dubai')
     WHEN ctry.code = 'sa' THEN DATETIME(b.created_at, 'Asia/Riyadh')
     ELSE DATETIME(b.created_at)
 END
   AS created_at,
   CASE
       WHEN src_w.partner_warehouse_code IN (
           'DXBID01','DXBID02','AUHID01','AUHID02',
           'JEDID01','JEDID02','RUHID01','RUHID02'
       ) THEN 'pending_putaway_ambient'

       WHEN src_w.partner_warehouse_code IN (
           'AUHFKZ01','AUHFKZ02','AUHFKZ03','RUHID05','JEDID05'
       ) THEN 'pending_putaway_frozen'

       WHEN src_w.partner_warehouse_code IN (
           'DXBFNV01','RUHID07','JEDID07'
       ) THEN 'pending_putaway_fnv'
```

```
      WHEN src_w.partner_warehouse_code IN (
          'DXBID03','DXBID04','AUHID03','AUHID04',
          'RUHID03','RUHID04','JEDID03','JEDID04',
          'DXBID07','RUHID06','JEDID06'
      ) THEN 'pending_putaway_chiller'

      ELSE 'unknown'
  END AS storage_condition,
  b.exdoc_nr AS exdoc_nr,
  bi.wms_barcode,
  bi.pbarcode_canonical AS pbarcode_canonical,
  src_w.partner_warehouse_code AS src_wh_code,
  dst_w.partner_warehouse_code AS ds_code,
  dst_w.area AS ds_name,
  ctry.code AS country_code,
  SUM(
  IF
    (pujl.id_putaway_job_line IS NULL
      OR pujl.id_status=1, 1, 0)) AS items_pending,
FROM
  noondwh.mxfulfillment_norms.box b
LEFT JOIN
  noondwh.mxfulfillment_norms.status b_st
ON
  b.id_status = b_st.id_status
LEFT JOIN
  noondwh.mxfulfillment_norms.box_tote_mapping btm
USING
  (awb_nr)
LEFT JOIN
  noondwh.mxfulfillment_norms.box_item bi
ON
  bi.id_box=b.id_box
LEFT JOIN
  noondwh.mxfulfillment_norms.putaway_job_line pujl
ON
```

```
    pujl.id_box_item=bi.id_box_item
  LEFT JOIN
    noondwh.mxfulfillment_norms.reason rs
  ON
    rs.id_reason=pujl.id_reason
  LEFT JOIN
    noondwh.mxfulfillment_norms.warehouse dst_w
  ON
    b.id_warehouse = dst_w.id_warehouse
  LEFT JOIN
    noondwh.mxfulfillment_norms.warehouse src_w
  ON
    b.id_src_warehouse = src_w.id_warehouse
  LEFT JOIN
    noondwh.mxfulfillment_norms.country ctry
  ON
    ctry.id_country = src_w.id_country
  LEFT JOIN
    noondwh.mxfulfillment_norms.exdoc e
  ON
    e.exdoc_nr = b.exdoc_nr
  LEFT JOIN
    noondwh.mxfulfillment_norms.exdoc_type et
  ON
    et.id_exdoc_type = e.id_exdoc_type
  WHERE
    DATE(b.created_at) >= CURRENT_DATE()-2
    AND b_st.code IN ('pending',
      'putting')
    AND et.code = 'goms'
  GROUP BY
    ALL )
SELECT
  date,
  ds_code,
  ds_name,
```

```
 src_wh_code,
 storage_condition,
 exdoc_nr,
 awb_nr,
 country_code,
 wms_barcode,
 pbarcode_canonical,
 CASE
   WHEN country_code = 'ae' THEN TIMESTAMP_DIFF(CURRENT_DATETIME('Asia/Dubai'),
created_at, MINUTE)
   WHEN country_code = 'sa' THEN TIMESTAMP_DIFF(CURRENT_DATETIME('Asia/Riyadh'),
created_at, MINUTE)
END
 AS ageing_in_mins,
 SUM(items_pending) AS items_pending
 FROM
 base
GROUP BY
 ALL
```

# DS - Complains_

## Easy Guide

**Purpose:** Lists the raw dark-store defects source tables used for UAE and KSA.

`complains_raw_all`

Source table containing all customer complaint records from the darkstore namespace. Each row is one complaint event.

**Complain category:**

1. **complain_category**
2. **Defect_delivery (late / wrong / no delivery)**
3. **Defect_category (item malfunction / warranty)**
4. **Defect_delivery (damage)**
5. **Defect_category (content_mismatch)**
6. **Defect_quality**
7. **Defect_fulfillment (others)**
8. **Defect_fulfillment (missing / wrong item / order)**

### UAE, EGY

```
Select *
From `noonbinimksa.darkstore.complains_raw_all`
```

### KSA

```
Select *
From `noonbinimksa.darkstore.complains_raw_sa`
```

# Specific Complains

1. complaints on expired items

```
SELECT *
FROM `noonbinimksa.darkstore.complains_raw_all`
where complain_reason IN ( 'expired_item'  )
```

2. complaints on near expiry items

```
SELECT *
FROM `noonbinimksa.darkstore.complains_raw_all`
where complain_reason IN ( 'item_near_expiry','near_expiry','warranty_near_expiry' )
```

3.  complaints on quality issues for dairy and milk

```
SELECT distinct *
FROM `noonbinimksa.darkstore.complains_raw_all`
where complain_reason IN ( 'quality_not_fresh',
'ProductQuality_FungusorMold','bad_quality_item'  )
and lower(minutes_category_new) in ('milk', 'dairy & eggs')
```

# DS - Live Inventory

## Easy Guide

**Purpose:** Points to the raw dark-store live inventory table.
**Key aliases / tables:** use country code filter

### UAE, EGY, KSA: Live Qty

Below table has all the countries.

```
Select *
From noonbinimksa.Stores.wh_loc_qty
```

# DS - Non Live Inventory

## Easy Guide

**Purpose:** Points to the raw dark-store non-live inventory tables for UAE, KSA, and Egypt.
**Key aliases / tables:** KSA = DAM_non_live, EGY = DAM_non_live_ksa

### UAE

```
Select *
From noonbinimksa.Stores.DAM_non_live
```

### KSA

```
Select *
From noonbinimksa.Stores.DAM_non_live_ksa
```

### EGY

```
Select *
From noonbinimksa.Stores.DAM_non_live_all_eg
```

# DS - FEFO Movements

## ***FEFO Movements :***

### Easy Guide

**Purpose: This Gives all the skus which have breached FEFO at darkstore, date and sku level**
**Key Columns :**

| fefo_breach | exp_nl_units | ex_nl_value |
|---|---|---|

```
with logs  as
(select distinct a.*
from noonbinimdwh.modelling.fifo_report_logs as a
join
  (select wh_code, sku, date_, max(updated_at) updated_at
  from noonbinimdwh.modelling.fifo_report_logs
  group by ALL) as b USING (wh_code, sku, date_, updated_at)
)


select distinct country_code, date_,hour_, wh_code, ds_code, ds_name, replen_mode,
category, sub_category,  sku,  brand,product_title, vendor_name,keep_life, shelf_life,
unit_cost,fefo_breach, exp_nl_units, ex_nl_value,expected_expiry_date
from logs
where date_ >= current_date - 3
and category not in ('Beverages')
order by date_ desc, ex_nl_value desc
```

## ***FEFO Adherence***

### Easy Guide

**Purpose: This Gives percentage of all FEFO  non- adhered skus at darkstore, category at date level**

```
with logs  as
(select distinct a.*
from noonbinimdwh.modelling.fifo_report_logs as a
join
  (select wh_code, sku, date_, max(updated_at) updated_at
  from noonbinimdwh.modelling.fifo_report_logs
```

```
  group by ALL) as b USING (wh_code, sku, date_, updated_at)
)

,base as (select country_code, date_,hour_, wh_code, ds_code, ds_name, replen_mode,
category, sub_category,  sku,  brand,product_title, vendor_name,keep_life, shelf_life,
unit_cost,fefo_breach, exp_nl_units, ex_nl_value,expected_expiry_date
from logs
where lower(sub_category) not in ('fresh juices')
and date_ >= current_date - 3),

agg_ as (select country_code, date_, category, sum(ex_nl_value) as ex_nl_value
FROM base
GROUP BY ALL),

total_nl as (
  SELECT country_code, date_, category, sum(tot_nl*unit_cost) as total_nl_val
  FROM noonbinimdwh.modelling.fifo_base
  where  date_ >= current_date - 3
  and category not in ('Fruit & Vegetables','Beverages')
  GROUP BY ALL
)

select a.*, total_nl_val, SAFE_DIVIDE(ex_nl_value,total_nl_val) as perc from agg_ a
LEFT JOIN total_nl b on a.date_ = b.date_ and a.country_code = b.country_code and
a.category = b.category
order by date_ DESC, ex_nl_value desc
```

# DS - Non Live Migraions

Non Live Migrations:

```
Select *
From `noonbinimksa.darkstore.DAM_line_date_new_final`
```

# DS - Expiry ( raw and Adjused )

## Easy Guide

**Purpose:** Points to the raw and adjusted dark-store expiry tables by country.
**Key aliases / tables:** KSA = central_expiry_raw, Adjusted = central_expiry_raw_sa, KSA = central_expiry_raw_adjusted
**Main fields / variables:** per

### Raw expiry table, system expiry:

UAE and EGY:

Select * from `noonbinimksa.Stores.central_expiry_raw`

KSA:

Select * from `noonbinimksa.Stores.central_expiry_raw_sa`

### Adjusted expiry, as per KL and SL:

UAE and EGY:

Select * from `noonbinimksa.Stores.central_expiry_raw_adjusted`

KSA:

Select * from `noonbinimksa.Stores.central_expiry_raw_adjusted_sa`

# DS - Picking Data

## Easy Guide

**Purpose:** to get at date, order nr and store level picking data of non-shots order
**Main fields / variables: Order Funnel** Timings

### Picking Data Today:

UAE : Select * from `noonbinimksa.darkstore.order_funnel_time`
KSA : Select * from `noonbinimksa.darkstore.order_funnel_time_sa`
EGY : Select * from `noonbinimksa.darkstore.order_funnel_time`

### Picking Data Historical ( L30 ) :

UAE : Select * from `noonbinimops.fulfillment.order_funnel_time_ae_hist`
KSA : Select * from `noonbinimops.fulfillment.order_funnel_time_sa_hist`
EGY : Select * from `noonbinimops.fulfillment.order_funnel_time_ae_hist`

# DS - Inbound and Stock Take Data

## Inbound Base:

### Easy Guide

**Purpose:** to get at date and store level inbound data
**Main fields / variables:** Inbound details along with user ids

UAE : Select * from `noonbinimops.fulfillment.ipp`
KSA : Select * from `noonbinimops.fulfillment.ipp_ksa`
EGY and BAH : Select * from `noonbinimksa.darkstore.ipp_all`

## Stock Take Base:

### Easy Guide

**Purpose:** Stock take Data at location and store level
**Main fields / variables:** stock_take_type ( incident / full location ) , expected , actual_qty, variance

Base Table:

UAE , KSA , EGY : `select distinct * from noonbinimksa.darkstore.stock_take_base`

`Full Location Adherence for` UAE , KSA , EGY :

```
 with base as (
SELECT a.*
from noonbinimksa.darkstore.stock_take_base a
left join noondwh.instant_instant_order.warehouse b
on a.partner_warehouse_code = b.partner_wh_code
where 1=1 --LOWER(b.country_code) = 'ae'
and (status_desc IN ('completed', 'pending_approval') OR status_desc IS NULL)
)

, overall as (
select country_code,created_date, partner_warehouse_code, area_name_en,id_warehouse,
stock_take_type
,  count(distinct id_stock_take_job_queue) as overall_job
from base group by all)

, completed as (
select country_code, created_date, partner_warehouse_code, stock_take_type
```

```
, coalesce(count(distinct id_stock_take_job_queue),0) as completed_job
from base
where lower(status_desc) = 'completed' group by all)

, output as (
select a.*, b.completed_job
, case
  when (b.completed_job = 0 or b.completed_job is null ) then 0
  else round(b.completed_job/a.overall_job,2)
  end as fill_rate
from overall a
left join completed b using(created_date, partner_warehouse_code, stock_take_type)
where a.overall_job > 0
)

select distinct * from output
```

# DS - MP - Planned Vs Actual - Biometric

## Easy Guide

**Purpose: Manpower planned vs actual ( biometric )**
**Table : noonbinimksa.Stores.Biometric_base_v2_3**
**( Biometric is not started in KSA and EGY )**

### UAE

```
WITH base AS (
  SELECT
    created_date,
    employee_id,
    punch_status,
    CASE
      WHEN punch_status IN ('Punched Properly') THEN 'Present'
      WHEN punch_status = 'Week Off' THEN 'Week Off'
      WHEN punch_status = 'Annual Leave' THEN 'Annual Leave'
      WHEN LOWER(punch_status) LIKE '%leave' THEN 'Other Leave'
      WHEN punch_status = 'No Punch' THEN 'Absent'
      WHEN punch_status = 'Single Punch' THEN 'Present'
      ELSE 'Others'
    END AS final_status
  FROM noonbinimksa.Stores.Biometric_base_v2_3
  --WHERE wh_code = 'DXBDS49' AND created_date = '2026-04-20'
)

SELECT
  created_date,
  COUNT(DISTINCT employee_id) AS total_employees,

  COUNT(DISTINCT CASE WHEN final_status = 'Present' THEN employee_id END) AS present,
  COUNT(DISTINCT CASE WHEN final_status = 'Week Off' THEN employee_id END) AS
week_off,
  COUNT(DISTINCT CASE WHEN final_status like '%Leave' THEN employee_id END) AS leave_,
  COUNT(DISTINCT CASE WHEN final_status = 'Absent' THEN employee_id END) AS absent,
  COUNT(DISTINCT CASE WHEN final_status = 'Others' THEN employee_id END) AS others

FROM base
GROUP BY created_date;
```

# DS - Actual Logins

## Easy Guide

**Purpose: Actual Job Logins across Inbound, Outbound and Stock Take**
**Columns : country_code, created_date, partner_warehouse_code, name, email**

**UAE, KSA, EGY**

```
WITH inbound AS (

  WITH ae AS (
    SELECT
      'AE' AS country_code,
      dist_partner_code AS partner_warehouse_code,
      CAST(id_user AS STRING) AS id_user,
      DATE(date) AS created_date,
      MIN(TIMESTAMP(start_time)) AS start_time,
      MAX(TIMESTAMP(end_time)) AS end_time,
      SUM(completed_count) AS num_of_items,

    FROM noonbinimops.fulfillment.ipp
    GROUP BY partner_warehouse_code, id_user, created_date
  ),

  sa AS (
    SELECT
          'SA' AS country_code,
      dist_partner_code AS partner_warehouse_code,
      CAST(id_user AS STRING) AS id_user,
      DATE(date) AS created_date,
      MIN(TIMESTAMP(start_time)) AS start_time,
      MAX(TIMESTAMP(end_time)) AS end_time,
      SUM(completed_count) AS num_of_items
    FROM noonbinimops.fulfillment.ipp_ksa
    GROUP BY partner_warehouse_code, id_user, created_date
  ),

  eg AS (
    SELECT
      'EG' AS country_code,
      dist_partner_code AS partner_warehouse_code,
      CAST(id_user AS STRING) AS id_user,
      DATE(date) AS created_date,
      MIN(TIMESTAMP(start_time)) AS start_time,
```

```
      MAX(TIMESTAMP(end_time)) AS end_time,
      SUM(completed_count) AS num_of_items,
    FROM noonbinimksa.darkstore.ipp_all
    WHERE country_code = 3
    GROUP BY partner_warehouse_code, id_user, created_date
  )

  SELECT * FROM ae
  UNION ALL
  SELECT * FROM sa
  UNION ALL
  SELECT * FROM eg
),

outbound AS (

  WITH base AS (
    SELECT
      CAST(w.country_code AS STRING) AS country_code,
      partner_wh_code AS partner_warehouse_code,
      CAST(pj.id_user AS STRING) AS id_user,
      DATE(TIMESTAMP_ADD(mo.created_at, INTERVAL 4 HOUR)) AS created_date,
      MIN(TIMESTAMP_ADD(pj.created_at, INTERVAL 4 HOUR)) AS start_time,
      MAX(TIMESTAMP_ADD(pj.updated_at, INTERVAL 4 HOUR)) AS end_time,
      ANY_VALUE(mos.num_of_items) AS num_of_items
    FROM noondwh.schoms_homs_orchestration.mp_order mo
    LEFT JOIN noondwh.instant_instant_order.warehouse w
      ON mo.warehouse_code = w.wh_code
    LEFT JOIN noondwh.schoms_homs_orchestration.mp_order_history moh
      ON mo.id_mp_order = moh.id_mp_order
    LEFT JOIN noondwh.schoms_homs.status mo_s
      ON mo_s.id_status = moh.id_status
    LEFT JOIN noondwh.schoms_homs.picking_job pj
      ON mo.id_mp_order = pj.id_mp_order
    LEFT JOIN noondwh.mxfulfillment_norms.user_active_job uaj
      ON pj.id_picking_job = uaj.job_id
    LEFT JOIN noondwh.schoms_homs_orchestration.mp_order_stats mos
      ON mos.id_mp_order = mo.id_mp_order
    WHERE DATE(TIMESTAMP_ADD(mo.created_at, INTERVAL 4 HOUR))
          >= CURRENT_DATE("Asia/Dubai") - 31
    GROUP BY country_code, partner_warehouse_code, id_user, created_date
```

```
  )

  SELECT
    country_code,
    partner_warehouse_code,
    id_user,
    created_date,
    MIN(TIMESTAMP(start_time)) AS start_time,
    MAX(TIMESTAMP(end_time)) AS end_time,
    SUM(num_of_items) AS num_of_items
  FROM base
  GROUP BY country_code, partner_warehouse_code, id_user, created_date
),

stock_take AS (

  WITH base AS (
    SELECT
      UPPER(ctr.code) AS country_code,
      wh.partner_warehouse_code,
      CAST(uaj.id_user AS STRING) AS id_user,
      DATE(TIMESTAMP_ADD(stjq.created_at, INTERVAL 4 HOUR)) AS created_date,
      MIN(TIMESTAMP_ADD(stjq.created_at, INTERVAL 4 HOUR)) AS start_time,
      MAX(TIMESTAMP_ADD(stjq.updated_at, INTERVAL 4 HOUR)) AS end_time,
      SUM(
        CASE
          WHEN st.code IN ('pending_approval', 'completed')
           AND stjl_st.code IN ('present', 'excess')
          THEN 1 ELSE 0
        END
      ) AS num_of_items
    FROM noondwh.mxfulfillment_norms.stock_take_job_queue stjq
    LEFT JOIN noondwh.mxfulfillment_norms.location l USING (id_location)
    LEFT JOIN noondwh.mxfulfillment_norms.warehouse wh
      ON l.id_warehouse = wh.id_warehouse
    LEFT JOIN noondwh.mxfulfillment_norms.stock_take_job stj
      USING (id_stock_take_job_queue)
    LEFT JOIN noondwh.mxfulfillment_norms.stock_take_job_line stjl
      ON stj.id_stock_take_job = stjl.id_stock_take_job
    LEFT JOIN noondwh.mxfulfillment_norms.user_active_job uaj
      ON stj.id_stock_take_job = uaj.job_id
```

```
    LEFT JOIN noondwh.mxfulfillment_norms.status st
      ON stj.id_status = st.id_status
    LEFT JOIN noondwh.mxfulfillment_norms.status stjl_st
      ON stjl.id_status = stjl_st.id_status
    LEFT JOIN noondwh.mxfulfillment_norms.country ctr USING (id_country)
    GROUP BY country_code, partner_warehouse_code, id_user, created_date
  )

  SELECT
    country_code,
    partner_warehouse_code,
    id_user,
    created_date,
    MIN(TIMESTAMP(start_time)) AS start_time,
    MAX(TIMESTAMP(end_time)) AS end_time,
    SUM(num_of_items) AS num_of_items
  FROM base
  GROUP BY country_code, partner_warehouse_code, id_user, created_date
),

-- unify all
output AS (
  SELECT 'Inbound' AS type_, p.*, u.email, CONCAT(u.first_name,' ',u.last_name) AS
name
  FROM inbound p
  LEFT JOIN noondwh.mxfulfillment_norms.user u
    ON CAST(u.id_user AS STRING) = p.id_user

  UNION ALL

  SELECT 'Outbound' AS type_, p.*, u.email, CONCAT(u.first_name,' ',u.last_name) AS
name
  FROM outbound p
  LEFT JOIN noondwh.schoms_homs.user u
    ON CAST(u.id_user AS STRING) = p.id_user

  UNION ALL

  SELECT 'Stock Take' AS type_, p.*, u.email, CONCAT(u.first_name,' ',u.last_name) AS
name
  FROM stock_take p
```

```sql
  LEFT JOIN noondwh.mxfulfillment_norms.user u
    ON CAST(u.id_user AS STRING) = p.id_user
)

-- FINAL OUTPUT
SELECT
  country_code, created_date,
  partner_warehouse_code,
  id_user AS Employee_ID,
  name,
  email,

  STRING_AGG(DISTINCT id_user) AS id_users,

  SUM(CASE WHEN type_ = 'Inbound' THEN num_of_items ELSE 0 END) AS ib_qty,
  SUM(CASE WHEN type_ = 'Outbound' THEN num_of_items ELSE 0 END) AS ob_qty,
  SUM(CASE WHEN type_ = 'Stock Take' THEN num_of_items ELSE 0 END) AS st_qty,

  SUM(num_of_items) AS total_qty

FROM output
GROUP BY all
ORDER BY created_date DESC;
```

# DS - Putaway Pendency Hourly Bucket

***put-away delays in stores***

```
SELECT
  eg.AM, eg.Assistant_manager ,  lower(country_code) as country_code,
  src_wh_code,
  pm.ds_code,
  pm.ds_name,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 0 AND 180 THEN items_pending
      ELSE 0
  END
    ) AS `0-3 hrs`,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 181 AND 360 THEN items_pending
      ELSE 0
  END
    ) AS `3-6 hrs`,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 361 AND 720 THEN items_pending
      ELSE 0
  END
    ) AS `6-12 hrs`,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 721 AND 1440 THEN items_pending
      ELSE 0
  END
    ) AS `12-24 hrs`,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 1441 AND 2880 THEN items_pending
      ELSE 0
  END
    ) AS `24-48 hrs`,
  SUM(CASE
      WHEN ageing_in_mins >= 2881 THEN items_pending
      ELSE 0
  END
    ) AS `48 hrs +`,
  SUM(items_pending) AS total_pending
FROM
```

```
  noonbinimops.fulfillment.putaway_pendency_v2 pm
LEFT JOIN noonbinimksa.Stores.ops_logistic_fulfillment_managers_eg eg
ON
  pm.ds_code = eg.partner_wh_Code
GROUP BY all
```

# DS - Skips

***UAE, KSA, EGY***

```
Select  *
From `noonbinimksa.darkstore.daily_manual_skips_hourly_uae_1`
```

# DS - Audit Scores

**For UAE, KSA :**

**( For Egypt it is not functional )**

```
select *
from noonbinimksa.darkstore.historic_score1
```

# DS - IPP

***put-away delays in stores***

```
SELECT
  eg.AM, eg.Assistant_manager ,  lower(country_code) as country_code,
  src_wh_code,
  pm.ds_code,
  pm.ds_name,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 0 AND 180 THEN items_pending
      ELSE 0
  END
    ) AS `0-3 hrs`,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 181 AND 360 THEN items_pending
      ELSE 0
  END
    ) AS `3-6 hrs`,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 361 AND 720 THEN items_pending
      ELSE 0
  END
    ) AS `6-12 hrs`,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 721 AND 1440 THEN items_pending
      ELSE 0
  END
    ) AS `12-24 hrs`,
  SUM(CASE
      WHEN ageing_in_mins BETWEEN 1441 AND 2880 THEN items_pending
      ELSE 0
  END
    ) AS `24-48 hrs`,
  SUM(CASE
      WHEN ageing_in_mins >= 2881 THEN items_pending
      ELSE 0
  END
    ) AS `48 hrs +`,
  SUM(items_pending) AS total_pending
FROM
```

```
  noonbinimops.fulfillment.putaway_pendency_v2 pm
LEFT JOIN noonbinimksa.Stores.ops_logistic_fulfillment_managers_eg eg
ON
  pm.ds_code = eg.partner_wh_Code
GROUP BY all
```

# IPP - Inbound

## UAE:

```
with ipp as ( select * from noonbinimops.fulfillment.ipp)

,base as (select
  date
  ,dist_partner_code, exdoc_nr, Box_no
  ,p.id_user,email as putter_mail,concat(u.first_name," ",u.last_name) as putter_name
  ,case when HR_Desig is null then "temp" else lower(HR_Desig) end as HR_Desig,
Employee_ID
  ,sum(total_sec) as time_taken_sec,sum(completed_count) as completed_item
from ipp p
left join noondwh.mxfulfillment_norms.user u on u.id_user = p.id_user
left join noonbinimops.fulfillment.ds_picker_details_uae pk on u.email=pk.email_final
where date >= current_date("Asia/Dubai")-31
group by all)

select date
,dist_partner_code, id_user, putter_mail,putter_name, HR_Desig, Employee_ID
, sum(time_taken_sec) as time_taken_sec,sum(completed_item) as completed_item
,case when sum(time_taken_sec)=0 then 0 else
round(sum(completed_item)*3600/sum(time_taken_sec)) end as iph
from base
group by all
```

## KSA:

```
    WITH ipp AS (
        SELECT *
        FROM noonbinimops.fulfillment.ipp_ksa
    ),
    base AS (
        SELECT
            date,
            dist_partner_code,
            exdoc_nr,
```

```sql
        Box_no,
        p.id_user,
        u.email AS putter_mail,
        CONCAT(u.first_name," ",u.last_name) AS putter_name,
        'temp' AS HR_Desig,
        emp_id AS Employee_ID,
        SUM(total_sec) AS time_taken_sec,
        SUM(completed_count) AS completed_item,
        CASE
            WHEN SUM(total_sec) = 0 THEN 0
            ELSE ROUND(SAFE_DIVIDE(SUM(completed_count) * 3600, SUM(total_sec)))
        END AS iph
    FROM ipp p
    LEFT JOIN noondwh.mxfulfillment_norms.user u
        ON u.id_user = p.id_user
    LEFT JOIN noonbinimksa.Stores.ds_picker_details_ksa pk
        ON u.email = pk.email
    WHERE date >= CURRENT_DATE("Asia/Riyadh") - 31
    GROUP BY ALL
)
SELECT
    date,
    dist_partner_code,
    id_user,
    putter_mail,
    putter_name,
    HR_Desig,
    Employee_ID,
    SUM(time_taken_sec) AS time_taken_sec,
    SUM(completed_item) AS completed_item,
    CASE
        WHEN SUM(time_taken_sec) = 0 THEN 0
        ELSE ROUND(SAFE_DIVIDE(SUM(completed_item) * 3600, SUM(time_taken_sec)))
    END AS iph
FROM base
GROUP BY ALL
```

**EGY: WIP**

# IPP - Putaway

***UAE:***

```
WITH emp_list as (
  select distinct Emp_ID,a.designation, a.role, store, b.DS_code as wh_Code,
b.New_Shif_Timing as Shift_Timing, DOJ
from noonbinimops.fulfillment.emp_details_new a
left join noonbinimksa.Stores.emp_functional_details_ae b on a.emp_Id = b.Employee_ID
)
, outbound as (
  SELECT
    date,
    partner_wh_code,
    id_user,
    ANY_VALUE(picker_mail) AS picker_mail,
    ANY_VALUE(picker_name) AS picker_name,
    ANY_VALUE(HR_Desig) AS HR_Desig,
    ANY_VALUE(Employee_ID) AS Employee_ID,

    SUM(total_orders) AS total_orders,
    SUM(total_time_picking) AS total_time_picking,
    SUM(total_time_packing) AS total_time_packing,
    SUM(total_time) AS total_time,
    SUM(total_item) AS total_item,

    SAFE_DIVIDE(SUM(total_time_picking), SUM(total_item)) AS picking_time_per_item,
    SAFE_DIVIDE(SUM(total_time_packing), SUM(total_item)) AS packing_time_per_item,
    SAFE_DIVIDE(SUM(total_time), SUM(total_item)) AS picking_packing_time_per_item

FROM `noonbinimksa.Stores.speed_ae_emp`
--WHERE partner_wh_code = 'DXBDS37' AND id_user = 9424 AND date = '2026-03-08'
GROUP BY all
)

, final as (

 select 'Outbound' as type_, date
 ,  partner_wh_code as dist_partner_code, id_user
, picker_mail as mail,  picker_name as name
, HR_Desig as desig, Employee_ID
, total_time_picking, total_time_packing
```

```
, total_time as time_taken_sec, total_item as completed_item, '' as inbound_,
cast(picking_time_per_item as string) as outbound
, total_orders
 from outbound
)

,inter2 AS (
  SELECT
    type_,
    date,
    dist_partner_code AS store,
    name AS user_name,
    Employee_ID,
    completed_item,
    SAFE_CAST(inbound_ AS FLOAT64) AS inbound_iph,
    SAFE_CAST(outbound AS FLOAT64) AS outbound_time_per_item,
    time_taken_sec, total_time_picking, total_time_packing, total_orders
  FROM final
),

output AS (
  SELECT
    b.date,
    b.store,
    b.user_name,
    b.Employee_ID,

    SUM(IF(type_ = 'Outbound', completed_item, 0)) AS outbound_qty,
    SUM(IF(type_ = 'Outbound', time_taken_sec, 0)) AS outbound_time_,
    SUM(IF(type_ = 'Outbound', total_time_picking, 0)) AS outbound_time_picking,
    SUM(IF(type_ = 'Outbound', total_time_packing, 0)) AS outbound_time_packing,
    SUM(IF(type_ = 'Outbound', total_orders, 0)) AS total_orders


  FROM inter2 b
  GROUP BY
    b.date, b.store, b.user_name, b.Employee_ID
)

,output_1 AS (
  SELECT
```

```
    b.date,
    b.store,
    b.user_name,
    b.Employee_ID,
    b.* EXCEPT (date, user_name, Employee_ID, store)
  FROM output b
  LEFT JOIN emp_list a
    ON a.Emp_ID = b.Employee_ID
)

SELECT * ,
case when outbound_qty = 0 then 0
else safe_divide(outbound_qty*3600, outbound_time_) end as outbound_iph,

FROM output_1
```

***KSA:***

```
WITH outbound AS (
    WITH base AS (
        SELECT
            mo.id_mp_order,
            mo.order_nr,
            partner_wh_code AS partner_warehouse_code,
            area_name_en,
            pj.id_user,
            ANY_VALUE(mo.warehouse_code) AS warehouse_code,
            DATE(TIMESTAMP_ADD(mo.created_at, INTERVAL 3 HOUR)) AS created_date,
            MIN(TIMESTAMP_ADD(pj.ready_to_start_at , INTERVAL 3 HOUR)) AS
fulfillemnt_ready_to_start_at,
            MIN(
                GREATEST(
                    TIMESTAMP_ADD(pj.ready_to_start_at , INTERVAL 3 HOUR),
                    COALESCE(
                        LEAST(
                            COALESCE(TIMESTAMP_ADD(prj.preparing_ended_at , INTERVAL 3
HOUR), TIMESTAMP('2100-01-01')),
```

```
                        TIMESTAMP_ADD(TIMESTAMP_ADD(prj.preparing_started_at ,
INTERVAL 3 MINUTE), INTERVAL 3 MINUTE)
                    ),
                    TIMESTAMP('1980-01-01')
                )
            )
        ) AS picking_ready_to_start_at,
        MIN(CASE WHEN mo_s.code = 'created' THEN TIMESTAMP_ADD(moh.modified_at,
INTERVAL 4 HOUR) END) AS created,
        MIN(CASE WHEN mo_s.code = 'fulfilling' THEN TIMESTAMP_ADD(moh.modified_at,
INTERVAL 4 HOUR) END) AS fulfilling,
        MIN(CASE WHEN mo_s.code = 'fulfilled' THEN TIMESTAMP_ADD(moh.modified_at,
INTERVAL 4 HOUR) END) AS fulfilled,
        MIN(CASE WHEN mo_s.code = 'delivered' THEN TIMESTAMP_ADD(moh.modified_at,
INTERVAL 4 HOUR) END) AS delivered,
        MIN(CASE WHEN mo_s.code = 'to_scan' THEN TIMESTAMP_ADD(moh.modified_at,
INTERVAL 4 HOUR) END) AS to_scan,
        MIN(CASE WHEN mo_s.code = 'scanned' THEN TIMESTAMP_ADD(moh.modified_at,
INTERVAL 4 HOUR) END) AS scanned,
        MIN(CASE WHEN mo_s.code = 'picked_up' THEN TIMESTAMP_ADD(moh.modified_at,
INTERVAL 4 HOUR) END) AS picked_up,
        MIN(CASE WHEN pj_s.code = 'pending' THEN TIMESTAMP_ADD(pjh.modified_at,
INTERVAL 4 HOUR) END) AS pending,
        MIN(CASE WHEN pj_s.code = 'picking' THEN TIMESTAMP_ADD(pjh.modified_at,
INTERVAL 4 HOUR) END) AS picking,
        MIN(CASE WHEN pj_s.code = 'packing' THEN TIMESTAMP_ADD(pjh.modified_at,
INTERVAL 4 HOUR) END) AS packing,
        MIN(CASE WHEN pj_s.code = 'completed' THEN TIMESTAMP_ADD(pjh.modified_at,
INTERVAL 4 HOUR) END) AS completed,
        ANY_VALUE(mos.num_of_items) AS num_of_items
    FROM noondwh.schoms_homs_orchestration.mp_order mo
    LEFT JOIN noondwh.schoms_homs.country ctry USING(id_country)
    LEFT JOIN noondwh.schoms_homs_orchestration.mp_order_history moh ON
mo.id_mp_order = moh.id_mp_order
    LEFT JOIN noondwh.schoms_homs.status mo_s ON mo_s.id_status = moh.id_status
    LEFT JOIN noondwh.schoms_homs.picking_job pj ON mo.id_mp_order = pj.id_mp_order
    LEFT JOIN noondwh.schoms_homs.picking_job_history pjh ON pj.id_picking_job =
pjh.id_picking_job
    LEFT JOIN noondwh.schoms_homs.status pj_s ON pjh.id_status = pj_s.id_status
    LEFT JOIN noondwh.schoms_homs.prepping_job prj ON pj.id_picking_job =
prj.id_picking_job
```

```
        LEFT JOIN noondwh.schoms_homs_orchestration.mp_order_stats mos ON
mos.id_mp_order = mo.id_mp_order
        LEFT JOIN noondwh.instant_instant_order.warehouse w ON mo.warehouse_code =
w.wh_code
        LEFT JOIN noondwh.instant_instant_order.sales_order so ON mo.order_nr =
so.order_nr
        WHERE DATE(TIMESTAMP_ADD(mo.created_at, INTERVAL 4 HOUR)) >=
CURRENT_DATE("Asia/Dubai") - 31
          AND w.country_code = 'SA'
        GROUP BY ALL
    ),
    DeliveryTimes AS (
        SELECT
            DISTINCT created_date AS date,
            partner_warehouse_code AS partner_wh_code,
            order_nr,
            id_user,
            TIMESTAMP_DIFF(packing, fulfilling, SECOND) AS picking_time,
            TIMESTAMP_DIFF(fulfilled, packing, SECOND) AS packing_time,
            TIMESTAMP_DIFF(fulfilled, fulfilling, SECOND) AS picking_packing_time
        FROM base
        WHERE DATE(created) >= CURRENT_DATE('Asia/Riyadh') - 31
    ),
    item_count AS (
        SELECT
            EXTRACT(DATE FROM TIMESTAMP_ADD(a.created_at, INTERVAL 3 HOUR)) AS date,
            order_nr,
            a.wh_Code,
            a.country_code,
            a.id_sales_order,
            w.area_name_en AS name,
            w.partner_wh_code,
            COUNT(DISTINCT item_nr) AS item_count
        FROM noondwh.instant_instant_order.sales_order a
        LEFT JOIN noondwh.instant_instant_order.sales_order_item z ON a.id_sales_order =
z.id_sales_order
        LEFT JOIN noondwh.instant_instant_order.warehouse w ON a.wh_code = w.wh_code
        WHERE status_code_order = "delivered" AND cancel_reason_code IS NULL
          AND EXTRACT(DATE FROM TIMESTAMP_ADD(a.created_at, INTERVAL 4 HOUR)) >=
CURRENT_DATE("Asia/Dubai") - 31
          AND a.country_code = "SA"
```

```
    GROUP BY ALL
),
picker_base AS (
    SELECT
        DISTINCT a.date,
        a.partner_wh_code,
        id_user,
        a.order_nr,
        SUM(a.picking_time) AS time_taken_picking,
        SUM(a.packing_time) AS time_taken_packing,
        SUM(a.picking_packing_time) AS time_taken_picking_packing,
        SUM(i.item_count) AS picked_items
    FROM DeliveryTimes a
    INNER JOIN item_count i ON a.order_nr = i.order_nr
    GROUP BY ALL
),
final AS (
    SELECT
        a.date,
        a.partner_wh_code,
        a.id_user,
        u.email AS picker_mail,
        CONCAT(u.first_name," ",u.last_name) AS picker_name,
        a.order_nr,
        time_taken_picking,
        time_taken_packing,
        time_taken_picking_packing,
        picked_items,
        "temp" AS HR_Desig,
        Emp_id AS Employee_ID
    FROM picker_base a
    LEFT JOIN noondwh.schoms_homs.user u ON a.id_user = u.id_user
    LEFT JOIN noonbinimksa.Stores.ds_picker_details_ksa pk ON u.email = pk.email
)
SELECT
    date,
    partner_wh_code,
    id_user,
    picker_mail,
    picker_name,
    HR_Desig,
```

```
        Employee_ID,
        COUNT(DISTINCT order_nr) AS total_orders,
        SUM(time_taken_picking) AS total_time_picking,
        SUM(time_taken_packing) AS total_time_packing,
        SUM(time_taken_picking_packing) AS total_time,
        SUM(picked_items) AS total_item,
        ROUND(SAFE_DIVIDE(SUM(time_taken_picking), SUM(picked_items)), 2) AS
picking_time_per_item,
        ROUND(SAFE_DIVIDE(SUM(time_taken_packing), SUM(picked_items)), 2) AS
packing_time_per_item,
        ROUND(SAFE_DIVIDE(SUM(time_taken_picking_packing), SUM(picked_items)), 2) AS
picking_packing_time_per_item
    FROM final
    GROUP BY ALL
),

final AS (
    SELECT
        'Outbound' AS type_,
        date,
        partner_wh_code AS dist_partner_code,
        STRING_AGG(DISTINCT CAST(id_user AS STRING)) AS id_user,
        picker_mail AS mail,
        picker_name AS name,
        HR_Desig AS desig,
        Employee_ID,
        total_time_picking,
        total_time_packing,
        total_time AS time_taken_sec,
        total_item AS completed_item,
        '' AS inbound_,
        CAST(picking_time_per_item AS STRING) AS outbound
    FROM outbound
    GROUP BY ALL
),

inter1 AS (
    SELECT *
    FROM final
),
```

```
inter2 AS (
    SELECT
        type_,
        date,
        dist_partner_code AS store,
        name AS user_name,
        STRING_AGG(DISTINCT id_user) AS id_user,
        STRING_AGG(DISTINCT Employee_ID) AS Employee_ID,
        completed_item,
        SAFE_CAST(inbound_ AS FLOAT64) AS inbound_iph,
        SAFE_CAST(outbound AS FLOAT64) AS outbound_time_per_item,
        time_taken_sec,
        total_time_picking,
        total_time_packing
    FROM inter1
    GROUP BY ALL
),

output AS (
    SELECT
        b.date,
        b.store,
        b.user_name,
        b.id_user,
        SUM(IF(type_ = 'Outbound', completed_item, 0)) AS outbound_qty,
        SUM(IF(type_ = 'Outbound', time_taken_sec, 0)) AS outbound_time_,
        SUM(IF(type_ = 'Outbound', total_time_picking, 0)) AS outbound_time_picking,
        SUM(IF(type_ = 'Outbound', total_time_packing, 0)) AS outbound_time_packing,
        Employee_ID
    FROM inter2 b
    GROUP BY b.date, b.store, b.user_name, b.id_user, Employee_ID
),

output_1 AS (
    SELECT
        b.date,
        b.store,
        b.user_name,
        b.Employee_ID,
        b.* EXCEPT(date, user_name, store, id_user, Employee_ID)
    FROM output b
```

```
)

SELECT *,
case when outbound_qty = 0 then 0
else safe_divide(outbound_qty*3600, outbound_time_) end as outbound_iph
FROM output_1
```

# DS - Adjustments

***Adjustments :***

***UAE:***

```
SELECT * FROM `noonbinimops.fulfillment.adjustments_master_uae`
```

***KSA:***

```
SELECT * FROM `noonbinimops.fulfillment.adjustments_master_ksa`
```

***EGY:***

```
SELECT * FROM `noonbinimops.fulfillment.adjustments_master_eg`
```

# DS - Missing

***DS-MISSING***

### Easy Guide

**Purpose: for each inbound timestamp, it gives bifurcation of qtys ( Expected, Pending, Completed, Missing, Expired, Damaged, Excess ) at country_code level**

***UAE, KSA, EGY***

*SELECT DISTINCT **

*FROM noonbinimops.fulfillment.missing_base_v3*

*GROUP BY ALL*

### Easy Guide

**Purpose: For each inbound date, it gives bifurcation of qtys ( Expected, Pending, Completed, Missing, Expired, Damaged, Excess ) at country_code level**

***UAE, KSA, EGY***

```
SELECT DISTINCT
  country_code, ds_code, ds_name, DATE,
  psku_code, wms_barcode, pbarcode_canonical, id_partner,
  SUM(qty_expected) AS qty_expected,
  SUM(qty_pending) AS qty_pending,
  SUM(qty_completed) AS qty_completed,
  SUM(qty_missing) AS qty_missing,
  SUM(qty_expired) AS qty_expired,
  SUM(qty_damaged) AS qty_damaged,
  SUM(qty_excess) AS qty_excess,
  AVG(cost_price) AS cost_price,
  STRING_AGG(DISTINCT category) AS category,
  STRING_AGG(DISTINCT src_wh_code) AS source_warehouse,
  STRING_AGG(DISTINCT CAST(id_box AS STRING)) AS id_box,
  STRING_AGG(DISTINCT CAST(awb_nr AS STRING)) AS awb_nr,
  STRING_AGG(DISTINCT CAST(exdoc_nr AS STRING)) AS exdoc_nr,
  STRING_AGG(DISTINCT CAST(tote_code AS STRING)) AS tote_code,
  STRING_AGG(DISTINCT CAST(ts AS STRING)) AS ts
FROM noonbinimops.fulfillment.missing_base_v3
GROUP BY ALL
```

DS noonbinimksa table <> DAG mapping <> Raw tables

# DS noonbinimksa tables <> Raw Table/Dag mapping

| Heading / Tab | Dag Link | Derived table name | Main / primary raw table | If primary raw is derived: raw DAG | If primary raw is derived: raw source |
|---|---|---|---|---|---|
| DS - Complains | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/minutes_fulfillment_uae/Defects/complain_details_uae_auto.toml?ref_type=heads | noonbinimksa.darkstore.complains_raw_all | noonbinimksa.darkstore.complains_raw_ae | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/daily_ops/UAE/complains_raw_ae.toml?ref_type=heads | noondwh.instant_instant_order.cs_order_adjustment<br>noondwh.instant_instant_order.sales_order<br>noondwh.instant_instant_order.warehouse<br>noonbiinstnt.mktg.sales_complete |
| DS - Complains | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/daily_ops/KSA/complaints/complains_raw_sa.toml?ref_type=heads | noonbinimksa.darkstore.complains_raw_sa | noonbiinstnt.minutes.minutes_commercial_category_mapping | | |
| Specific Complains | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/minutes_fulfillment_uae/Defects/complain_details_uae_auto.toml?ref_type=heads | noonbinimksa.darkstore.complains_raw_all | noonbinimksa.darkstore.complains_raw_ae | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/daily_ops/UAE/complains_raw_ae.toml?ref_type=heads | noondwh.instant_instant_order.cs_order_adjustment<br>noondwh.instant_instant_order.sales_order<br>noondwh.instant_instant_order.warehouse<br>noonbiinstnt.mktg.sales_complete |
| DS - Live Inventory | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Hourly_Tracking/wh_loc_qt | noonbinimksa.Stores.wh_loc_qty | noondwh.wms.stock | | |

| | | | | | |
|---|---|---|---|---|---|
| | y.toml?ref_type=heads | | | | |
| DS - Non Live Inventory | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Disposal/NonLive/DAM.toml?ref_type=heads | noonbinimksa.Stores.DAM_non_live | noonbinimksa.Stores.wh_loc_qty | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Hourly_Tracking/wh_loc_qty.toml?ref_type=heads | noondwh.wms.stock<br>noondwh.wms.location<br>noondwh.wms.warehouse<br>noondwh.wms.zone |
| DS - Non Live Inventory | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Disposal/NonLive/ksa_DAM.toml?ref_type=heads | noonbinimksa.Stores.DAM_non_live_ksa | noonbinimksa.Stores.wh_loc_qty | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Hourly_Tracking/wh_loc_qty.toml?ref_type=heads | noondwh.wms.stock<br>noondwh.wms.location<br>noondwh.wms.warehouse<br>noondwh.wms.zone |
| DS - Non Live Inventory | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Disposal/NonLive/DAM_non_live_all_eg.toml?ref_type=heads | noonbinimksa.Stores.DAM_non_live_all_eg | noonbinimksa.Stores.wh_loc_qty | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Hourly_Tracking/wh_loc_qty.toml?ref_type=heads | noondwh.wms.stock<br>noondwh.wms.location<br>noondwh.wms.warehouse<br>noondwh.wms.zone |
| DS - Non Live Migrations | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/Non_Live/Today/NL_DB.toml?ref_type=heads | noonbinimksa.darkstore.DAM_line_date_new_final | noondwh.mxfulfillment_norms.stock_migration_job | | |
| DS - Expiry (raw and adjusted) | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Expiry/CentralExpiryTable/raw_data.toml?ref_type=heads | noonbinimksa.Stores.central_expiry_raw | noondwh.mxfulfillment_norms.expirable_item | | |

| | | | | | |
|---|---|---|---|---|---|
| DS - Expiry (raw and adjusted) | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Expiry/CentralExpiryTable/central_expiry_raw_sa.toml?ref_type=heads | noonbinimksa.Stores.central_expiry_raw_sa | noondwh.mxfulfillment_norms.expirable_item | | |
| DS - Expiry (raw and adjusted) | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Expiry/CentralExpiryTable/central_expiry_raw_adjusted.toml?ref_type=heads | noonbinimksa.Stores.central_expiry_raw_adjusted | noondwh.instant_spanner.product_ops_attributes | | |
| DS - Expiry (raw and adjusted) | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Expiry/CentralExpiryTable/central_expiry_raw_adjusted_sa.toml?ref_type=heads | noonbinimksa.Stores.central_expiry_raw_adjusted_sa | noondwh.instant_spanner.product_ops_attributes | | |
| DS - Picking Data | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/order_funnel_time.toml?ref_type=heads | noonbinimksa.darkstore.order_funnel_time | noondwh.schoms_homs_orchestration.mp_order | | |
| DS - Picking Data | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/order_funnel_time_sa.toml?ref_type=heads | noonbinimksa.darkstore.order_funnel_time_sa | noondwh.schoms_homs_orchestration.mp_order | | |

| | | | | | |
|---|---|---|---|---|---|
| DS - Inbound and Stock Take Data | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/minutes_fulfillment_uae/ipp%20txw/ipp_txw.toml?ref_type=heads | noonbinimksa.darkstore.ipp_all | noondwh.mxfulfillment_norms.putaway_job_line | | |
| DS - Inbound and Stock Take Data | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/stock_take/stock_take_base.toml?ref_type=heads | noonbinimksa.darkstore.stock_take_base | noondwh.mxfulfillment_norms.stock_take_job_queue | | |
| DS - MP - Planned Vs Actual - Biometric | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/ManPower/Biometric/Biometric_base_v2_3.toml?ref_type=heads | noonbinimksa.Stores.Biometric_base_v2_3 | noonbinimksa.Stores.biometric_roaster_3 | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/ManPower/Biometric/Biometric_roaster_historical.toml?ref_type=heads | noonbinimksa.Stores.table_mp_ae<br>noonbinimksa.Stores.Biometric_base_v2_3 |
| DS - Actual Logins | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/minutes_fulfillment_uae/ipp%20txw/ipp_txw.toml?ref_type=heads | noonbinimksa.darkstore.ipp_all | noondwh.mxfulfillment_norms.putaway_job_line | | |
| DS - Putaway Pendency Hourly Bucket | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/together_ops_manager.toml?ref_type=heads | noonbinimksa.Stores.ops_logistic_fulfillment_managers_eg | noonbinimops.fulfillment.putaway_pendency_v2 | | |

| | | | | | |
|---|---|---|---|---|---|
| DS - Skips | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/daily_ops/UAE/daily_manual_skips_hourly_uae.toml?ref_type=heads | noonbinimksa.darkstore.daily_manual_skips_hourly_uae_1 | noondwh.schoms_homs.picking_incident | | |
| DS - Audit Scores | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Audit/table5.toml?ref_type=heads | noonbinimksa.darkstore.historic_score1 | noonbinimksa.darkstore.audit_staff_qus_ans | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/Audit/staff_answers.toml?ref_type=heads | noondwh.instant_instant_ds_audit.question_group<br>noondwh.instant_instant_ds_audit.question<br>noondwh.instant_instant_ds_audit.audit_submission<br>noondwh.instant_instant_order.warehouse |
| DS - IPP | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/together_ops_manager.toml?ref_type=heads | noonbinimksa.Stores.ops_logistic_fulfillment_managers_eg | noonbinimops.fulfillment.putaway_pendency_v2 | | |
| IPP - Inbound | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/ManPower/ds_picker_details_ksa.toml?ref_type=heads | noonbinimksa.Stores.ds_picker_details_ksa | noondwh.schoms_homs.user | | |
| IPP - Putaway | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/KSA/Chiller/Excel/emp_functional_details_ae.toml?ref_type=heads | noonbinimksa.Stores.emp_functional_details_ae | noonbinimops.fulfillment.emp_details_new | | |

| | | | | | |
|---|---|---|---|---|---|
| IPP - Putaway | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/central_ops/minutes_fulfillment_uae/Speed/speed_ae_emp.toml?ref_type=heads | noonbinimksa.Stores.speed_ae_emp | noondwh.schoms_homs_orchestration.mp_order | | |
| IPP - Putaway | https://dp-gitlab.noon.team/dagman-group/dp-dagman-nimksa/-/blob/master/project/ops/ManPower/ds_picker_details_ksa.toml?ref_type=heads | noonbinimksa.Stores.ds_picker_details_ksa | noondwh.schoms_homs.user | | |